
MindVideo AI
بايثون 3.14
من الصفر إلى الاحتراف
دليل عملي متكامل لبناء عقلية المبرمج وإتقان اللغة
من أول سطر كود حتى نشر أول منتج حقيقي
تعلم
طبق
ابني مشاريع
انشر منتجك
Python
Data Analysis
Web Development
Automation
Machine Learning
Build & Deploy
Learn
Practice
Build Projects
Publish
Think
Code
Build
Deploy
م/أحمد صالح المغز
2026

>_

# بايثون 3.14
# من الصفر إلى الاحتراف

دليل عملي متكامل لبناء عقلية المبرمج وإتقان اللغة — من أول سطر كود حتى نشر أول منتج حقيقي.

م/ أحمد صالح المغز

الإصدار الأول · 2026

# مقدمة الكتاب

> *"أفضل وقت لتبدأ كان قبل عشر سنوات. ثاني أفضل وقت هو الآن."*

هذا الكتاب وُلد من ملاحظة بسيطة تكرّرت أمامي مئات المرات: معظم من يتعلّم البرمجة لا يفشل لأن اللغة صعبة، بل لأن أول كتاب وضعه أمامه كان مكتوبًا لمن يعرف البرمجة أصلًا.

لاحظ الفرق. كتاب يقول لك: «المتغيّر هو مساحة مُسمّاة في الذاكرة تُستخدم لتخزين قيمة». جملة صحيحة تمامًا، وميتة تمامًا بالنسبة لشخص لم يكتب سطرًا في حياته. لن يفهم منها شيئًا، وسيظن أن المشكلة فيه.

هذا الكتاب مكتوب بالاتجاه المعاكس. يبدأ من السؤال «لماذا نحتاج هذا الشيء أصلًا؟»، ثم يبني الفهم بالتشبيه والتجربة، ثم يصل إلى التعريف الرسمي — بعد أن يكون قد صار بديهيًا.

## لمن كُتب هذا الكتاب؟

ثلاثة أنواع من القرّاء سيجدون فيه ما يبحثون عنه:

- **المبتدئ التام** الذي لم يكتب سطر كود في حياته، ولا يعرف الفرق بين المترجم والمفسّر، ولا يهمّه أن يعرف الآن. هذا الكتاب يفترض أنك تبدأ من الصفر المطلق، ولا يفترض أي خلفية رياضية أو هندسية.
- **القادم من لغة أخرى** الذي يعرف كيف يفكّر برمجيًا، لكنه يريد أن يتعلّم *أسلوب* بايثون الحديث — لا مجرد ترجمة أفكاره القديمة إلى صيغة جديدة.
- **الممارس الذي يريد ترتيب معرفته** بعد سنوات من التعلّم المتقطّع من هنا وهناك، ويريد مرجعًا واحدًا متماسكًا يعود إليه.

## ما الذي يميّز هذا الكتاب؟

**أولًا: الترتيب مقصود.** الفصول ليست كومة مواضيع، بل سلسلة اعتماديات. كل فصل يستخدم ما قبله فقط، ولا يفترض ما بعده. لن تجد نفسك في منتصف مثال يستخدم مفهومًا لم تصل إليه بعد.

**ثانيًا: كل مثال يعمل فعلًا.** لا توجد أمثلة «توضيحية» ناقصة يصعب تشغيلها. الأكواد في هذا الكتاب مكتوبة لتُنسخ وتُشغَّل، ومجمّعة لك في مجلد مرافق منظّم حسب الفصول، مع سكربت يشغّلها كلها ويتحقق منها.

**ثالثًا: الأخطاء جزء من الدرس.** كل فصل يحتوي على قسم صريح للأخطاء الشائعة، لا لتتجنّبها فقط، بل لتفهم *لماذا* تحدث. من يفهم سبب الخطأ لا يكرّره؛ ومن يحفظ الحل فقط سيكرّره بصيغة أخرى.

**رابعًا: بايثون الحديثة، لا بايثون الأمس.** الكتاب مبنيّ على Python 3.14+ ويستخدم الأدوات التي يعتمدها المهندسون اليوم فعلًا: `uv` بدل `pip` التقليدي، `ruff` و`black` للتنسيق، `pytest` للاختبار، Type Hints كأسلوب كتابة

افتراضي، و`match-case` بدل سلاسل `if` الطويلة. لن تتعلم هنا أسلوبًا عفا عليه الزمن ثم تضطر إلى إعادة تعلّمه.

**خامسًا: الطريق إلى ما بعد الأساسيات.** الجزء الخامس من الكتاب يأخذك في جولة عملية داخل خمسة مسارات مهنية — الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر — كل مسار بمشروع حقيقي مكتمل. لن تخرج خبيرًا في أي منها، لكنك ستخرج وأنت تعرف *طعم* كل مسار، وقرارك بشأن تخصّصك سيكون مبنيًا على تجربة لا على مقال.

# كيف تستفيد منه إلى أقصى حد؟

قاعدة واحدة تلخّص كل شيء: **لا تقرأ الكتاب، نفّذه.**

البرمجة مهارة حركية بقدر ما هي معرفة. قراءة فصل عن الحلقات تشبه قراءة فصل عن السباحة؛ ستفهمه، وستنساه، وستغرق. اكتب الكود بيدك — لا تنسخه — واكسرْه عن قصد، واقرأ رسالة الخطأ، وأصلحه. هذه الدورة القصيرة (اكتب ← شغّل ← افهم الخطأ ← أصلح) هي ما يبني المهارة فعليًا.

ولهذا السبب، كل فصل ينتهي بثلاثة أشياء لا تجاوزها: **تمارين متدرجة**، ثم **اختبار فهم سريع**، ثم **ملخص مرجعي**. إن لم تحلّ التمارين، فأنت لم تنهِ الفصل بعد — أنت فقط قرأته.

# ملاحظة على المصطلحات

ستجد في الكتاب مزيجًا مقصودًا بين العربية والإنجليزية: نشرح المفهوم بالعربية، ثم نُثبّت المصطلح الإنجليزي بين قوسين. هذا ليس ترفًا. في عملك الحقيقي — التوثيق، رسائل الأخطاء، الاجتماعات، مقابلات العمل — ستقابل المصطلح الإنجليزي وحده. تعلّمه من الآن، ولا تنتظر حتى تحتاج إليه.

# شكر وتقدير

هذا الكتاب يقف على أكتاف مجتمع بايثون كله: من كتبوا التوثيق الرسمي مجانًا، ومن أجابوا عن ملايين الأسئلة في المنتديات، ومن بنوا المكتبات التي تجعل هذه اللغة قادرة على إدارة أنظمة يعتمد عليها مليارات البشر يوميًا. إن كنت ستتعلّم من عملهم، فردّ الجميل لاحقًا: أجب عن سؤال مبتدئ، أو اكتب توثيقًا واضحًا لكودك.

الآن، اقلب الصفحة. رحلتك تبدأ من الفصل الأول — وقبل أن تكتب أي كود.

# كيف تقرأ هذا الكتاب

قبل أن تبدأ، خُذ دقيقتين لتفهم كيف بُني هذا الكتاب. القراءة الصحيحة توفّر عليك أسابيع.

## تشريح الفصل الواحد

كل فصل — بلا استثناء — مبنيّ على التسلسل نفسه. عندما تعتاد عليه، ستعرف تلقائيًا أين أنت وماذا ينتظرك:

1. **مقدمة تحفيزية** — لماذا يستحق هذا الموضوع وقتك أصلًا.
2. **الشرح النظري** — الفكرة من أولها، بالتشبيه قبل المصطلح.
3. **أمثلة متدرجة** — من سطر واحد إلى برنامج كامل، خطوة بخطوة.
4. 💡 **نصائح الخبراء** — ما يعرفه من مارس العمل فعليًا ولم يُكتب في الكتب.
5. ⚠️ **تحذيرات وأخطاء شائعة** — الفخاخ التي يسقط فيها الجميع، ولماذا هي فخاخ.
6. 🔄 **مقارنات** — بين الطرق المختلفة، ومتى تختار كل واحدة.
7. **تمارين متدرجة** — ثلاثة مستويات: 🟢 مبتدئ، 🟡 متوسط، 🔴 تحدٍّ.
8. **مشروع تطبيقي** — يجمع كل ما تعلّمته في الفصل في شيء واحد يعمل.
9. 📌 **ملخص مرجعي** — جدول سريع تعود إليه لاحقًا بدل إعادة قراءة الفصل.

## دليل الرموز

خلال الكتاب ستقابل هذه الصناديق الملوّنة. كل واحد له وظيفة محددة، لا مجرد زينة:

> 💡 **نصيحة خبرة** — معلومة من الممارسة العملية، توفّر عليك وقتًا أو ألمًا.
>
> ⚠️ **تحذير** — خطأ يقع فيه المبتدئون بكثرة. اقرأه ببطء، فغالبًا ستقع فيه أنت أيضًا.
>
> ✅ **أفضل ممارسة** — الطريقة التي يكتب بها المحترفون هذا النوع من الكود.
>
> 🔄 **مقارنة** — جدول أو شرح يقارن بديلين، ليساعدك على الاختيار لا الحفظ.
>
> 🎯 **هدف أو تمرين** — مهمة تنفيذية. لا تتجاوزها.
>
> 📌 **ملاحظة** — تفصيل إضافي مفيد، لكنه ليس جوهريًا للمتابعة.

كذلك ستجد صناديق قابلة للفتح مثل هذا:

🔍 مثال: اضغط للكشف عن التفاصيل

هذه صناديق «اكشف الحل» — تحتوي على حلول التمارين وتلميحاتها وتفاصيل إضافية. في النسخة التفاعلية (HTML) تُفتح بالنقر. في نسخة PDF تظهر مفتوحة بالكامل حتى لا يفوتك شيء أثناء الطباعة.

# أكواد الكتاب

كل مثال برمجي في الكتاب موجود في مجلد `code` المرافق، منظّمًا حسب الفصول:

```
code/
├── chapters/
│   ├── ch01/ … ch26/      # أمثلة كل فصل بالترتيب
│   └── appendix-a/ …      # أمثلة الملاحق
├── run_all.py             # يشغّل كل الأمثلة ويتحقق منها
├── MANIFEST.json          # فهرس: كل ملف وقسمه وموضوعه
└── requirements.txt       # المكتبات الخارجية
```

اسم كل ملف يتبع النمط `رقم-الفصل` + `رقم-المثال` + `اسم_الدالة`، وفي أعلى كل ملف تجد عنوان القسم الذي جاء منه. لتشغيل مثال واحد:

**Terminal**

```
python code/chapters/ch03/03-004-variable_name.py
```

ولتشغيل كل الأمثلة والتحقق من سلامتها دفعة واحدة:

**Terminal**

```
python code/run_all.py
```

⚠ **ملاحظة مهمة:** بعض الأمثلة في الكتاب مكتوبة *بقصد* لتُظهر خطأً — مثل مثال يوضّح ماذا يحدث عند جمع نص برقم. هذه الأمثلة ستفشل عند التشغيل، وهذا هو المطلوب منها. سكربت `run_all.py` يصنّف كل مثال تلقائيًا: ناجح، أو مقتطف يعتمد على مثال سابق، أو تفاعلي يطلب إدخالًا، أو خطأ مقصود — فلا تختلط عليك الأمور.

# ثلاثة مسارات قراءة

لست مضطرًا لقراءة الكتاب بالترتيب من الغلاف إلى الغلاف. اختر المسار الذي يناسب وضعك:

**المسار الأول — المبتدئ التام (الافتراضي).**
اقرأ كل شيء بالترتيب، دون استثناء، وحلّ كل تمرين. المدة الواقعية: 8–12 أسبوعًا بمعدل ساعة يوميًا. لا تتخطَّ الفصول 1–10 مهما بدت بسيطة؛ هي الأساس الذي سيُحاسبك عليه كل ما بعدها.

**المسار الثاني — القادم من لغة أخرى.**
ابدأ بالفصل الثالث مباشرة، واقرأ بسرعة الفصول 3–7 (ستجدها مألوفة)، ثم توقّف عند الفصل الثامن والتاسع (هياكل البيانات في بايثون لها تفاصيل تختلف عن غيرها)، واقرأ الفصول 10–18 بتركيز كامل. هذا هو جوهر ما يميّز بايثون عن غيرها.

**المسار الثالث — المرجع السريع.**
إن كنت تعرف بايثون وتريد مرجعًا: ابدأ من جدول المحتويات، واقرأ ما تحتاجه فقط، ثم استخدم الملحق ج (الجدول المرجعي الشامل) والملحق أ (قاموس المصطلحات) كمرجع دائم.

# نصائح عملية قبل البدء

- **جهّز بيئتك أولًا.** لا تقرأ الفصل الثالث قبل أن تُثبّت بايثون وتشغّل أول ملف. الفصل الثاني هو بوابتك.
- **اكتب بيدك، لا تنسخ.** النسخ يخلق وهم الفهم. الكتابة تُجبرك على مواجهة التفاصيل.
- **لا تُطارد الكمال.** لن تفهم 100% من أي فصل في المرة الأولى، وهذا طبيعي ومقصود. الفهم البرمجي حلزوني: تعود للمفهوم لاحقًا بفهم أعمق.
- **خصص وقتًا للتمارين أكبر من وقت القراءة.** النسبة الصحية: 40% قراءة، 60% تطبيق.
- **عند التعطّل، لا تستسلم سريعًا ولا تُطل.** القاعدة العملية: 20 دقيقة محاولة جادة، ثم ابحث عن تلميح، ثم اسأل.

> 🎯 **التزام صغير، أثر كبير:** اكتب في ورقة «سأنهي الفصل الأول بحلول [تاريخ]» وعلّقها أمامك. الالتزام المكتوب يضاعف احتمال الإكمال — هذه ليست نصيحة تحفيزية، بل نتيجة تتكرر في كل دراسة عن التعلّم.

# خريطة الكتاب

الكتاب مقسّم إلى ستة أجزاء متتابعة وثلاثة ملاحق مرجعية. كل جزء يبني على ما قبله، وكل جزء ينتهي بقدرة جديدة تستطيع استخدامها فورًا.

01

### ما قبل الكود

قبل أن تلمس لوحة المفاتيح، تحتاج إلى عقلية جديدة وورشة عمل جاهزة. هذان الفصلان يبنيان الأساس الذهني والتقني الذي يستند إليه كل ما بعدهما.

الفصول 1–2

02

### الأساسيات المتينة

المتغيّرات، النصوص، الأرقام، القرار، التكرار، هياكل البيانات، والدوال. ثمانية فصول تُتقن فيها مفردات اللغة التي ستكتب بها كل برامجك.

الفصول 3–10

03

### البرمجة الوظيفية والكائنية

الانتقال من «كود يعمل» إلى «كود مُهندَس»: معالجة الأخطاء، الدوال المتقدمة، والبرمجة الكائنية بأساسها وتعمّقها.

الفصول 11–14

04

### بايثون الحديثة

المهارات التي تفصل المبرمج العادي عن المحترف: تلميحات الأنواع، الملفات وتنسيقات البيانات، البرمجة غير المتزامنة، وإدارة المشاريع بأدوات العصر.

الفصول 15–18

05

### جسور التخصص

جولة عملية في خمسة مسارات مهنية: الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر — كل مسار بمشروع حقيقي مكتمل.

الفصول 19–23

06

### الاحتراف والنشر

كيف تكتب كودًا يقرأه الآخرون بسرور، كيف تثق فيه باختبارات آلية، وكيف تُحوّله من ملف على جهازك إلى منتج حقيقي.

الفصول 24–26

07

**مراجع دائمة**

ثلاثة ملاحق للرجوع إليها مرارًا: قاموس المصطلحات، حلول جميع التمارين، وجدول مرجعي سريع يلخّص الكتاب كله.

الـفـصـول 1–3

# ما ستبنيه بنهاية الكتاب

الكتاب لا يعلّمك مفردات اللغة فقط، بل يجعلك تبني أشياء حقيقية. هذه قائمة بما ستنجزه فعليًا — كل واحد منها مشروع كامل يعمل:

- **بطاقة تعريف تفاعلية** تأخذ بيانات المستخدم وتحسبها وتطبعها بتنسيق احترافي.
- **مولّد كلمات مرور** يبني كلمات قوية بقواعد قابلة للتخصيص.
- **حاسبة علمية** من الطرفية تتعامل مع العمليات والأولويات بأمان.
- **لعبة تخمين الأرقام** بقواعد تحقق وحلقات تحكم كاملة.
- **قائمة مهام (To-Do)** تُدار من الطرفية بكل عمليات الإضافة والحذف والبحث.
- **نظام جرد مخزون** يتعامل مع القواميس والمجموعات بكفاءة.
- **مكتبة دوال رياضية** خاصة بك، موثّقة ومُختبرة.
- **نظام إدارة حسابات بنكية** مبني على البرمجة الكائنية.
- **نظام حجز مكتبة كامل** يجمع الوراثة وتعدد الأشكال والـ Dataclasses.
- **محلّل ملفات بيانات** يقرأ JSON وCSV ويستخرج منه رؤى.
- **واجهة API كاملة لإدارة المهام** بـ FastAPI مع تحقق واختبار.
- **تحليل بيانات مبيعات حقيقية** بالرسوم البيانية.
- **بوت كشط بيانات** يجمع المعلومات من الويب ويحفظها منظّمة.
- **نموذج تصنيف أول** بـ scikit-learn.
- **أداة سطر أوامر احترافية** لتنظيم الملفات تستخدمها يوميًا.
- **حزمة بايثون قابلة للنشر** مع اختباراتها وتوثيقها.

# ما لا يغطيه هذا الكتاب

الصدق جزء من الاحتراف. هذا الكتاب لا يحاول أن يكون موسوعة، ولن يجعلك خبيرًا في:

- **تطوير الويب المتقدّم** (قواعد البيانات الموزّعة، المصادقة، النشر السحابي على نطاق واسع).
- **التعلّم العميق** (PyTorch، الشبكات العصبية، ضبط النماذج الكبيرة) — الجزء الخاص بالذكاء الاصطناعي هو نافذة أولى فقط.
- **هندسة البيانات الضخمة** (Spark، تدفّقات المعالجة، مستودعات البيانات).
- **تطوير تطبيقات الموبايل** — بايثون ليست الخيار الأول هنا، والكتاب يقول ذلك بصراحة.

هذه ليست ثغرات في الكتاب، بل حدود مقصودة. كل واحد منها يستحق كتابًا كاملًا، والملحق الأخير في نهاية رحلتك سيدلّك على الطريق الصحيح لكل مسار.

# عن هذه النسخة


| | |
|---|---|
| المؤلف | م/ أحمد صالح المغز |
| العنوان | بايثون 3.14 — من الصفر إلى الاحتراف |
| الإصدار | الإصدار الأول · 2026 |
| لغة البرمجة | Python 3.14 أو أحدث |
| المحتوى | 26 فصلًا في 6 أجزاء، و3 ملاحق، و430 قسمًا |
| الأمثلة | 794 مثالًا برمجيًا، مجمّعة في مجلد `code` المرافق |
| الصيغ | PDF (للطباعة والقراءة)، HTML (تفاعلي)، Markdown (قابل للتعديل والنشر) |
| الخطوط | Cairo · IBM Plex Sans Arabic · JetBrains Mono |


هذا الكتاب مرجع تعليمي. الأمثلة والأكواد مرخّصة للاستخدام والتعديل في مشاريعك الشخصية والتجارية دون إذن مسبق. يُمنع إعادة نشر النص نفسه تجاريًا دون اتفاق.

صُمّم وهُندس هذا الكتاب آليًا من مصدر Markdown واحد، لتبقى الصيغ الثلاث متزامنة دائمًا.

# جدول المحتويات

26 فصلًا · 3 ملاحق · 6 أجزاء


## الجزء الأول — ما قبل الكود

1 كيف يفكّر المبرمج؟ 15

2 تجهيز ورشة العمل 27

## الجزء الثاني — الأساسيات المتينة

3 المتغيرات - أول لبنة 46

4 التعامل مع النصوص (Strings) 67

5 الأرقام والعمليات الحسابية 87

6 اتخاذ القرار (Control Flow) 107

7 التكرار (Loops) 130

8 هياكل البيانات - القوائم (Lists) 151

9 القواميس والمجموعات (Dict & Set) 176

10 الدوال - تنظيم الكود 201

## الجزء الثالث — البرمجة الوظيفية والكائنية

11 معالجة الأخطاء (Error Handling) 231

12 الدوال المتقدمة (Lambda, Decorators, Generators, Closures) 258

13 البرمجة الكائنية - الأساس (Class, Object, __init__, self) 290

14 البرمجة الكائنية 320

## الجزء الرابع — بايثون الحديثة

15 Type Hints — نحو برمجة أكثر وضوحًا ودقة (باستخدام mypy) 352

16 الملفات وتنسيقات البيانات 392

17 البرمجة غير المتزامنة - مقدمة عملية (async/await) 429

18 إدارة المشاريع كمحترف 458

## الجزء الخامس — جسور التخصص

19 مسار تطوير الويب - بناء واجهة API لإدارة المهام (FastAPI) 493
20 مسار تحليل البيانات (pandas, numpy, matplotlib) 518
21 مسار الأتمتة — استخراج البيانات من الويب (Web Scraping) 556
22 مسار الذكاء الاصطناعي — نافذة أولى (scikit-learn) 586
23 مسار بناء أدوات سطر الأوامر (CLI) 610

## الجزء السادس — الاحتراف والنشر

24 كتابة كود نظيف (Clean Code) 639
25 الاختبار (Testing) باستخدام pytest 667
26 من الكود إلى المنتج (Packaging, Deployment، وخارطة الطريق) 698

## الملاحق — مراجع دائمة

1 قاموس المصطلحات البرمجية (عربي-إنجليزي) 730
2 حلول جميع التمارين 745
3 جدول مرجعي سريع (Cheat Sheet) شامل 780

الجزء الأول · فصلان

01

# ما قبل الكود

قبل أن تلمس لوحة المفاتيح، تحتاج إلى عقلية جديدة وورشة عمل جاهزة. هذان الفصلان يبنيان الأساس الذهني والتقني الذي يستند إليه كل ما بعدهما.

1 · كيف يفكّر المبرمج؟

2 · تجهيز ورشة العمل

فصل 01 / 26 · الجزء الأول

01

# كيف يفكّر المبرمج؟

9 دقيقة قراءة | 11 قسمًا | 1 مثالًا | 1,847 كلمة

**في هذا الفصل**

1.1 مقدمة: لماذا هذا الفصل قبل أي سطر كود؟
1.2 ما هي البرمجة فعليًا؟
1.3 الخوارزمية: الوصفة قبل الكود
1.4 كيف «يفهم» الكمبيوتر الكود الذي تكتبه؟
1.5 لماذا Python تحديدًا؟ (مقارنة صادقة)
1.6 عقلية حل المشكلات: المهارة الحقيقية
1.7 أخطاء ذهنية شائعة عند المبتدئين (تجنّبها من اليوم الأول)
1.8 بناء «عضلة» حل المشكلات - تمارين عملية
1.9 مشروع تطبيقي صغير للفصل: «مصمم الخوارزميات»
1.10 اختبر فهمك (Quiz سريع)
1.11 ملخص الفصل (Cheat Sheet)

> *«البرمجة ليست عن تعلّم لغة، بل عن تعلّم طريقة تفكير جديدة تمامًا.»*

## 1.1 📖 مقدمة: لماذا هذا الفصل قبل أي سطر كود؟

أغلب الكتب التعليمية ترتكب خطأ جوهريًا: تقذف بالمبتدئ مباشرة في بحر الأقواس والـ syntax قبل أن يفهم **ماذا يحدث فعليًا** عندما «يبرمج». النتيجة؟ يحفظ القارئ الأوامر دون أن يفهمها، فينهار عند أول مشكلة لم يرها من قبل.

هذا الفصل مختلف. لن تكتب فيه سطر كود واحد بلغة Python، لكنك ستخرج منه بشيء أهم: **عقلية المبرمج**. وهذه العقلية هي ما سيبقى معك حتى لو تعلمت غدًا لغة برمجة أخرى غير Python بالكامل.

## 1.2 ما هي البرمجة فعليًا؟

لنبدأ بتعريف بسيط وصادق:

> ***البرمجة = إعطاء تعليمات دقيقة جدًا لجهاز «غبي» حرفيًا، لكنه سريع جدًا في التنفيذ.***

الكلمة المفتاحية هنا هي **«دقيقة»**. الكمبيوتر لا يفهم السياق، ولا يخمّن قصدك، ولا يقول «أعتقد أنك تقصد كذا». هو ينفّذ **بالضبط** ما كتبته، لا أكثر ولا أقل.

## 🍳 تشبيه وصفة الطبخ

تخيل أنك تكتب وصفة لشخص لم يدخل مطبخًا في حياته، ولا يملك أي حس سليم، لكنه ينفذ التعليمات بدقة روبوتية:

**وصفة سيئة (غامضة):**

```
اصنع بيضة مقلية
```

الشخص الحرفي لا يعرف: هل يكسر البيضة أولاً؟ هل يشغّل الموقد قبل أم بعد وضع المقلاة؟ كم من الوقت يقليها؟

**وصفة جيدة (دقيقة كالكود):**

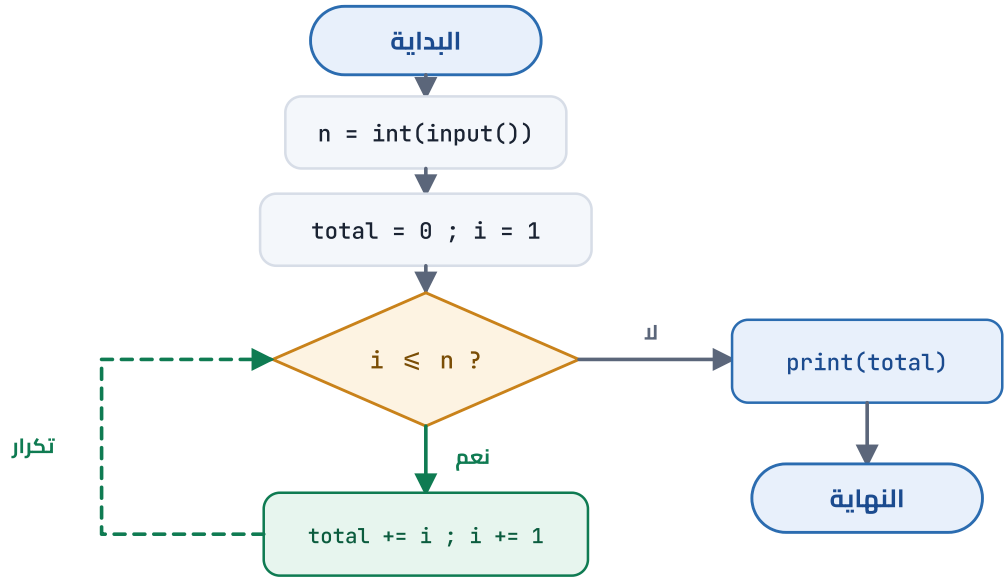
```
1. أحضر مقلاة من الخزانة
2. ضع المقلاة على الموقد
3. شغّل الموقد على حرارة متوسطة
4. أحضر بيضة واحدة من الثلاجة
5. اكسر البيضة في المقلاة
6. انتظر 3 دقائق
7. أطفئ الموقد
8. انقل البيضة إلى طبق
```

هذا بالضبط ما تفعله عندما تكتب كودًا. أنت لا «تطلب» من الكمبيوتر شيئًا، أنت **تصف له كل خطوة** بترتيب منطقي صارم.

## 💡 نصيحة الخبراء

> *أفضل المبرمجين ليسوا من يحفظون أكبر عدد من الأوامر، بل من يستطيعون تفكيك مشكلة كبيرة ومعقدة إلى سلسلة من خطوات صغيرة وبسيطة جدًا. هذه المهارة تسمى **Decomposition** (التفكيك)، وهي أهم مهارة ستطورها في هذا الكتاب.*

## 1.3 الخوارزمية: الوصفة قبل الكود




تُنفَّذ الخطوات بالترتيب، ويتكرّر الجسم طالما بقي الشرط صحيحًا.

شكل 1.1 — خوارزمية مجموع الأعداد من 1 إلى n

الكلمة التقنية لـ«الوصفة» هذه هي **Algorithm (خوارزمية)**.

> ***الخوارزمية*** = *سلسلة محددة ومنتهية من الخطوات لحل مشكلة معينة.*

لاحظ ثلاث خصائص مهمة في هذا التعريف:

| الخاصية | المعنى | مثال مخالف (خطأ) |
|---|---|---|
| **محددة** | كل خطوة واضحة بلا غموض | «أضف القليل من الملح» ❌ |
| **منتهية** | لها بداية ونهاية واضحتان | حلقة لا تتوقف أبدًا ❌ |
| **قابلة للتنفيذ** | يمكن فعلاً القيام بها | «فكّر بعمق في الحل» ❌ |

### 🎯 تمرين ذهني (بدون كمبيوتر بعد)

قبل أن نكتب أي كود، دعنا نتدرب على **التفكير الخوارزمي**. اكتب خوارزمية (بالعربية، كنقاط مرقّمة) لحل هذه المشكلة:

> ***المشكلة:*** *لديك 3 أرقام، وتريد معرفة أي رقم منها هو الأكبر.*

**حل مقترح (اضغط للكشف)**

```
1. سمِّ الأرقام: رقم_أ، رقم_ب، رقم_ج
2. افترض أن رقم_أ هو الأكبر مبدئيًا، وسمِّه "الأكبر_الحالي"
3. قارن رقم_ب بـ "الأكبر_الحالي":
   - إذا كان رقم_ب أكبر، اجعل "الأكبر_الحالي" = رقم_ب
4. قارن رقم_ج بـ "الأكبر_الحالي":
   - إذا كان رقم_ج أكبر، اجعل "الأكبر_الحالي" = رقم_ج
5. أعلن أن "الأكبر_الحالي" هو الإجابة النهائية
```

لاحظ: هذا بالضبط ما سنكتبه بلغة Python لاحقًا، لكن الآن بصيغة بشرية. هذه الصيغة الوسطى تُسمى **Pseudocode** (كود زائف) وسنستخدمها كثيرًا في هذا الكتاب قبل كل مشكلة جديدة.

---

## 1.4 كيف «يفهم» الكمبيوتر الكود الذي تكتبه؟




لا يُترجَم الكود إلى لغة الآلة مباشرة، بل إلى شفرة بايتية تُنفّذ على آلة افتراضية.

شكل 1.2 — رحلة الكود من المصدر إلى التنفيذ

هذا سؤال يتجنبه كثير من الكتب، لكن فهمه يزيل غموضًا كبيرًا من ذهن المبتدئ.

الكمبيوتر في الحقيقة **لا يفهم** Python إطلاقًا. المعالج (CPU) يفهم لغة واحدة فقط: **اللغة الثنائية** (0 و 1)، تسمى **Machine Code**.

السؤال إذن: كيف يتحول الكود الذي نكتبه بكلمات شبه إنجليزية ( `print` , `if` , `for` ) إلى لغة الأصفار والآحاد؟

### 🔄 مترجم (Compiler) مقابل مفسّر (Interpreter)

هناك طريقتان رئيسيتان لحل هذه المشكلة في عالم البرمجة:

**الطريقة الأولى - الترجمة الكاملة مسبقًا (Compiled Languages)**
مثل لغة C++. يأخذ برنامج يسمى **Compiler** كل الكود دفعة واحدة، ويترجمه بالكامل إلى ملف تنفيذي بلغة الآلة، **قبل** أي تشغيل فعلي. تشبّه هذه العملية بترجمة كتاب كامل إلى لغة أخرى قبل طباعته.

**الطريقة الثانية - الترجمة الفورية أثناء التشغيل (Interpreted Languages)**
وهذه طريقة Python. برنامج يسمى **Interpreter** يقرأ الكود **سطرًا بسطر**، ويترجم وينفّذ كل سطر فورًا قبل الانتقال للسطر التالي. يشبه هذا مترجمًا فوريًا يترجم كلام متحدث مباشرة أثناء إلقائه الخطاب.

## 🔄 جدول مقارنة: لماذا هذا يهمك عمليًا؟

| الجانب | لغات مُفسَّرة (Python) | لغات مُترجَمة (C++, Rust) |
|---|---|---|
| **السرعة في التطوير** | ✅ سريعة جدًا - اكتب وشغّل فورًا | ❌ أبطأ - يجب الترجمة أولاً |
| **سرعة التنفيذ** | ❌ أبطأ نسبيًا | ✅ أسرع بكثير |
| **اكتشاف الأخطاء** | عند الوصول للسطر الخاطئ فقط | ✅ قبل التشغيل بالكامل |
| **الأفضل لـ** | التعلم، السكربتات، النماذج الأولية | الأنظمة الحساسة للأداء |

> 💡 **لماذا هذا مهم لك كمبتدئ؟**
> لأن Python ستسامحك! إن كان لديك خطأ في السطر 50، فستُنفَّذ الأسطر من 1 إلى 49 بنجاح تام قبل أن تتوقف عند الخطأ. هذا رائع للتعلم التدريجي، لكنه يعني أيضًا أن اختبار الكود بشكل كامل قبل الاعتماد عليه أمر ضروري (سنتعلم هذا في فصل الاختبارات لاحقًا).

## ⚙️ ما يحدث فعليًا خلف الكواليس في Python

بدقة أكبر قليلاً (دون إغراق تقني):

```
الكود المصدري (ملف .py)
        ↓
   [يقرأه Interpreter المُفسِّر]
        ↓
يُحوَّل إلى Bytecode (صيغة وسيطة، ليست صفر وواحد مباشرة)
        ↓
   [تنفّذه Python Virtual Machine (PVM)]
        ↓
   النتيجة الفعلية على شاشتك
```

> ⚠️ **تحذير من الخلط الشائع:**
> كثير من المبتدئين يظنون أن Python «بطيئة لأنها سيئة». الحقيقة أدق: Python تضحي ببعض السرعة التنفيذية مقابل سهولة هائلة في الكتابة والقراءة وتقليل الأخطاء. هذه مقايضة (trade-off) مقصودة وذكية، وليست عيبًا. ولهذا بالضبط تجد Python تدير أنظمة عملاقة مثل Instagram وNetflix رغم «بطئها» النظري.

---

## 1.5 لماذا Python تحديدًا؟ (مقارنة صادقة)

لن أعطيك دعاية مجانية لـ Python. دعنا نكون موضوعيين تمامًا.

### ✅ نقاط قوة Python الحقيقية

1. **قابلية القراءة (Readability)**: صُممت لتُقرأ كالإنجليزية شبه الطبيعية
2. **منحنى تعلّم لطيف**: من أسهل اللغات للبدء بها فعليًا
3. **مجتمع ومكتبات هائلة**: تقريبًا أي مشكلة لها حل جاهز (مكتبة)
4. **شمولية استثنائية**: نفس اللغة تُستخدم في الويب، تحليل البيانات، الذكاء الاصطناعي، الأتمتة، الألعاب
5. **طلب سوق عمل ضخم** ومستمر في النمو

### ❌ حدود Python الحقيقية (بصدق تام)

1. **أبطأ تنفيذيًا** من لغات مثل C++, Rust, Go في المهام الحسابية الثقيلة جدًا
2. **ليست الخيار الأمثل** لتطبيقات الموبايل الأصلية (native mobile apps)
3. **استهلاك ذاكرة أعلى** نسبيًا مقارنة باللغات المُترجمة

### 🔄 مقارنة سريعة مع لغات أخرى شهيرة

| اللغة | متى تُفضَّل عن Python |
|---|---|
| **JavaScript** | تطوير واجهات المواقع (Frontend) - Python لا تعمل في المتصفح |
| **C++/Rust** | أنظمة تشغيل، محركات ألعاب، أداء حرج قصوى |
| **Java/Kotlin** | تطبيقات أندرويد الأصلية |
| **Swift** | تطبيقات iOS الأصلية |

🎯 **الخلاصة الصادقة:** Python ليست «الأفضل مطلقًا» (لا توجد لغة كذلك)، لكنها **الأفضل كنقطة بداية** لأنها تعلّمك التفكير البرمجي الصحيح بأقل قدر من الحواجز التقنية، ثم تفتح لك أبوابًا شبه لا نهائية بعد إتقانها.

---

## 1.6 عقلية حل المشكلات: المهارة الحقيقية

إليك سرًا يغفل عنه كثيرون: **90% من البرمجة الاحترافية ليست كتابة كود، بل التفكير قبل الكتابة.**

### 🧩 منهجية حل أي مشكلة برمجية (ستستخدمها طوال الكتاب)

عندما تواجه أي مشكلة، مهما بدت بسيطة أو معقدة، اتبع هذه الخطوات دائمًا:

```
1. افهم المشكلة تمامًا
   → ما هي المُدخلات (Inputs)؟
   → ما هي المُخرجات المطلوبة (Outputs)؟

2. فكّر بأمثلة يدوية بسيطة
   → جرّب حل المشكلة بيدك على ورقة بمثال صغير

3. اكتب الخوارزمية (Pseudocode)
   → بخطوات واضحة بلغتك الخاصة، بدون قلق من الـ syntax

4. حوّل الخوارزمية إلى كود حقيقي
   → هذه أسهل خطوة فعليًا إن أتقنت ما قبلها!

5. اختبر الكود بحالات متعددة
   → حالة عادية، حالة حدّية (edge case)، حالة خاطئة
```

### 🎯 مثال تطبيقي كامل على المنهجية

لنطبق هذه الخطوات على مشكلة واقعية:

> ***المشكلة:*** *تريد برنامجًا يخبرك إن كانت سنة معينة «سنة كبيسة» (366 يومًا) أم لا.*

**الخطوة 1: فهم المشكلة**

- المُدخل: رقم سنة (مثل 2024)
- المُخرج: نعم أو لا

### الخطوة 2: جرّب بأمثلة يدوية

- هل تعرف قاعدة السنة الكبيسة؟ (قابلة للقسمة على 4، لكن ليس على 100 إلا إذا كانت قابلة للقسمة على 400)
- 2024 ÷ 4 = 506 بدون باقي → كبيسة ✅
- 1900 ÷ 4 = بدون باقي، لكن ÷ 100 أيضًا بدون باقي، و÷400 بها باقي → **ليست** كبيسة ❌
- 2000 ÷ 400 = بدون باقي → كبيسة ✅

### الخطوة 3: الخوارزمية (Pseudocode)

```
1. خذ السنة كمُدخل
2. إذا كانت السنة قابلة للقسمة على 400 → كبيسة
3. وإلا، إذا كانت قابلة للقسمة على 100 → ليست كبيسة
4. وإلا، إذا كانت قابلة للقسمة على 4 → كبيسة
5. وإلا → ليست كبيسة
```

### الخطوة 4: الكود الحقيقي (نظرة تشويقية فقط - سنتعلم هذا بالتفصيل لاحقًا)

Python

```python
def is_leap_year(year: int) -> bool:
    """تحديد ما إذا كانت السنة كبيسة أم لا."""
    if year % 400 == 0:
        return True
    elif year % 100 == 0:
        return False
    elif year % 4 == 0:
        return True
    else:
        return False

# اختبار الدالة
print(is_leap_year(2024))  # True
print(is_leap_year(1900))  # False
print(is_leap_year(2000))  # True
```

### الخطوة 5: اختبار بحالات متعددة

- حالة عادية: 2023 → ليست كبيسة ✅
- حالة حدّية: 2000 (قرن لكن قابل للقسمة على 400) ✅
- حالة حدّية أخرى: 1900 (قرن غير قابل للقسمة على 400) ✅

> **لاحظ شيئًا مهمًا جدًا**: لم تفهم كل سطر في الكود أعلاه، وهذا طبيعي تمامًا! الهدف من هذا المثال ليس تعليمك `def` أو `%` الآن، بل إثبات أن **المنهجية** (الخطوات 1-3) هي الجزء الصعب والمهم فعليًا، وأن كتابة الكود النهائي (الخطوة 4) تصبح سهلة نسبيًا بمجرد أن تفكر بوضوح.

## 1.7 أخطاء ذهنية شائعة عند المبتدئين (تجنّبها من اليوم الأول)

### ⚠ الخطأ 1: «يجب أن أحفظ كل شيء عن ظهر قلب»

**الحقيقة:** حتى المبرمجون المحترفون ذوو خبرة 20 عامًا يبحثون في جوجل يوميًا. المهارة الحقيقية هي **فهم المفاهيم**، لا حفظ الأوامر. الحفظ يأتي تلقائيًا بالممارسة المتكررة.

### ⚠ الخطأ 2: «إذا ظهر لي خطأ (Error)، فأنا فاشل»

**الحقيقة:** الأخطاء (Errors) هي **أفضل أصدقائك** في رحلة التعلم. كل رسالة خطأ هي معلم يشرح لك بالضبط ماذا حدث. سنتعلم في الفصل القادم كيف «نقرأ» رسائل الأخطاء بدل الخوف منها.

### ⚠ الخطأ 3: «يجب أن أفهم كل التفاصيل قبل الانتقال للموضوع التالي»

**الحقيقة:** التعلم البرمجي **حلزوني** وليس خطيًا. ستتعلم مفهومًا بسيطًا، تستخدمه بشكل سطحي، ثم تعود له لاحقًا بفهم أعمق بعد أن ترى تطبيقات أكثر. لا تعلّق نفسك في فصل واحد لأسابيع.

### ⚠ الخطأ 4: «النسخ من مصادر أخرى حرام أو غش»

**الحقيقة:** كل المبرمجين المحترفين يبحثون ويرجعون لمصادر (التوثيق الرسمي، Stack Overflow). الفرق بين المبتدئ والمحترف ليس «عدم البحث»، بل **فهم ما تنسخه** قبل استخدامه.

---

## 1.8 بناء «عضلة» حل المشكلات - تمارين عملية

الآن حان دورك. هذه التمارين **لا تحتاج كتابة كود** - فقط ورقة وقلم، أو حتى تفكير ذهني. الهدف تدريب عقلك على التفكير الخوارزمي قبل الانتقال للكود الفعلي في الفصول القادمة.

### 🟢 تمرين 1 (مستوى مبتدئ)

اكتب خوارزمية (خطوات مرقّمة بالعربية) لتحضير كوب شاي، بافتراض أن من ينفّذها لا يعرف شيئًا إطلاقًا عن المطبخ.

### 🟡 تمرين 2 (مستوى متوسط)

لديك قائمة من 5 أسماء طلاب ودرجاتهم في امتحان. اكتب خوارزمية لإيجاد اسم الطالب صاحب أعلى درجة.

> 💡 تلميح: فكّر كيف حللنا مثال «أكبر رقم بين 3 أرقام» سابقًا، ثم وسّع نفس المنطق.

### 🔴 تمرين 3 (تحدي)

اكتب خوارزمية للتحقق مما إذا كانت كلمة معينة (مثل «مدام» أو "level") هي **Palindrome** (تُقرأ نفسها من اليمين واليسار).

<details>
<summary>🔍 تلميح للتمرين 3 (اضغط إذا احتجت مساعدة)</summary>

فكّر في مقارنة الحرف الأول بالحرف الأخير، ثم الثاني بما قبل الأخير، وهكذا... متى تتوقف هذه المقارنة؟ وماذا لو وجدت عدم تطابق في منتصف العملية؟

</details>

---

## 1.9 مشروع تطبيقي صغير للفصل: «مصمم الخوارزميات»

بما أننا لم نكتب كودًا بعد، مشروع هذا الفصل مختلف عن باقي الفصول: هو **تمرين تفكير منهجي كامل**.

### 📋 المهمة

اختر **موقفًا واقعيًا** من حياتك اليومية (مثال: «كيف أختار ماذا ألبس حسب حالة الطقس»، أو «كيف أرتب أولويات مهامي اليومية»)، ثم:

1. حدد **المُدخلات** بدقة (ما المعلومات التي تحتاجها لاتخاذ القرار؟)
2. حدد **المُخرج** المطلوب (ما القرار النهائي؟)
3. اكتب **خوارزمية كاملة** بخطوات مرقّمة وواضحة تمامًا
4. اختبر خوارزميتك ذهنيًا بـ 3 حالات مختلفة (تأكد أنها تعمل منطقيًا في كل حالة)

🎯 **لماذا هذا المشروع مهم؟** لأنه يحاكي **بالضبط** ما ستفعله في كل مشروع برمجي طوال حياتك المهنية: تحويل مشكلة واقعية غامضة إلى خطوات منطقية دقيقة، **قبل** لمس لوحة المفاتيح لكتابة كود فعلي. المبرمجون الذين يتخطون هذه الخطوة هم من يقضون ساعات في «تصحيح» كود مبني على تفكير غير واضح من الأساس.

---

## 1.10 اختبر فهمك (Quiz سريع)

أجب عن هذه الأسئلة ذهنيًا أو كتابيًا للتأكد من استيعابك قبل الانتقال للفصل التالي:

1. ما الفرق الجوهري بين لغة مُفسَّرة ولغة مُترجَمة؟

2. لماذا تُعتبر «دقة التعليمات» أهم خاصية في أي خوارزمية؟
3. اذكر خاصيتين من خصائص Python تجعلها مناسبة للمبتدئين، وخاصية واحدة تُعتبر نقطة ضعف حقيقية لها.
4. رتّب خطوات منهجية حل المشكلات البرمجية الخمس بالترتيب الصحيح (دون الرجوع للنص).
5. لماذا يُعتبر ظهور رسالة خطأ (Error) أمرًا إيجابيًا للمتعلم، لا سلبيًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. اللغة المُفسَّرة تُترجَم وتُنفَّذ سطرًا بسطر أثناء التشغيل (مثل Python)، بينما المُترجَمة تُترجَم بالكامل إلى لغة الآلة *قبل* أي تشغيل (مثل C++).
2. لأن الكمبيوتر ينفّذ حرفيًا ما يُكتب له دون تخمين النية أو السياق؛ أي غموض يؤدي لنتائج خاطئة أو توقف البرنامج.
3. نقاط القوة: قابلية القراءة العالية، منحنى تعلّم لطيف، مجتمع ومكتبات ضخمة. نقطة الضعف: أبطأ تنفيذيًا من اللغات المُترجَمة في المهام الحسابية الثقيلة.
4. فهم المشكلة ← التجربة بأمثلة يدوية ← كتابة الخوارزمية (Pseudocode) ← كتابة الكود الفعلي ← الاختبار بحالات متعددة.
5. لأن رسالة الخطأ تخبرك بدقة أين ولماذا فشل الكود، مما يسرّع عملية التعلم ويحوّل الفشل إلى معلومة بنّاءة بدل أن يكون نهاية مسدودة.

</details>

---

## 1.11 📌 ملخص الفصل (Cheat Sheet)

| المفهوم | التعريف المختصر |
|---|---|
| **Algorithm (خوارزمية)** | سلسلة محددة ومنتهية من الخطوات لحل مشكلة |
| **Pseudocode** | وصف خطوات الحل بلغة بشرية بسيطة قبل كتابة الكود الفعلي |
| **Interpreter (مُفسِّر)** | برنامج يترجم وينفّذ الكود سطرًا بسطر أثناء التشغيل (طريقة Python) |
| **Compiler (مُترجِم)** | برنامج يترجم كل الكود دفعة واحدة قبل أي تشغيل |
| **Decomposition (تفكيك)** | تقسيم مشكلة كبيرة معقدة إلى خطوات صغيرة وبسيطة |
| **Edge Case (حالة حدّية)** | حالة استثنائية أو نادرة يجب اختبار الكود ضدها |

## 🎯 ماذا بعد؟

في **الفصل القادم**، سننتقل أخيرًا للجانب العملي: تثبيت Python 3.14 فعليًا على جهازك، تجهيز بيئة تطوير احترافية بـ VS Code، وكتابة وتشغيل أول سطر كود حقيقي - مع تعلّم كيفية **قراءة رسائل الخطأ** بثقة بدل الخوف منها منذ اللحظة الأولى.

فصل 02 / 26 · الجزء الأول

02

# تجهيز ورشة العمل

11 دقيقة قراءة | 14 قسمًا | 7 مثالًا | 2,289 كلمة

**في هذا الفصل**

2.1 مقدمة: لماذا هذا الفصل حاسم؟
2.2 فهم المكوّنات الأساسية قبل التثبيت
2.3 تثبيت Python 3.14
2.4 أداة uv: إدارة حديثة وفائقة السرعة
2.5 تجهيز محرر الأكواد: Visual Studio Code
2.6 أول تواصل مع Python: الـ REPL
2.7 كتابة وتشغيل أول ملف كود حقيقي
2.8 فن قراءة رسائل الخطأ (مهارة تُغيّر رحلتك بالكامل)
2.9 لمحة عن البيئات الافتراضية (Virtual Environments)
2.10 أخطاء التهيئة الشائعة وحلولها السريعة
2.11 تمارين عملية للفصل
2.12 مشروع تطبيقي صغير: «بطاقة التعريف الذكية»
2.13 اختبر فهمك (Quiz سريع)
2.14 ملخص الفصل (Cheat Sheet)

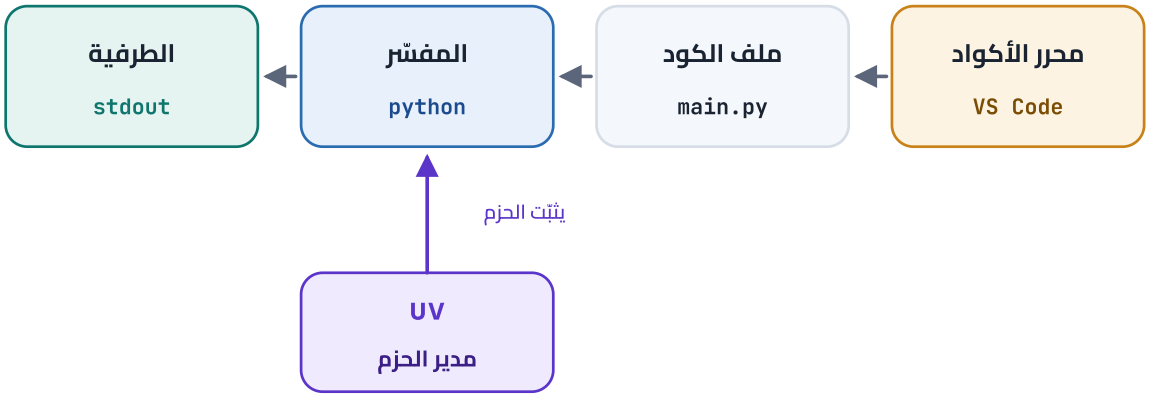
> *«النجار الماهر يعتني بأدواته قبل أن يصنع أي قطعة خشب. المبرمج الماهر يعتني ببيئة عمله قبل أن يكتب أي سطر كود.»*

## 2.1 📖 مقدمة: لماذا هذا الفصل حاسم؟

كثير من المبتدئين يواجهون **أول إحباط حقيقي** في رحلتهم البرمجية ليس بسبب صعوبة الكود، بل بسبب مشاكل التثبيت والإعداد. رسالة خطأ غامضة عند التثبيت، أو برنامج تحرير نصوص غير مناسب، كافية لإقناع الكثيرين أن «البرمجة ليست لهم» - وهذا خطأ فادح وغير صحيح إطلاقًا.

في هذا الفصل، سنبني بيئة عمل **احترافية وحديثة** خطوة بخطوة، بنفس الأدوات التي يستخدمها المبرمجون المحترفون في الشركات العالمية الكبرى اليوم. لن نستخدم طرقًا قديمة «كانت تُستخدم قبل 10 سنوات» - بل أحدث الممارسات المعتمدة في 2025.

# 2.2 فهم المكوّنات الأساسية قبل التثبيت




لكل أداة دور واحد: المحرر يكتب، والمفسّر ينفّذ، والطرفية تعرض، و uv يجهّز الحزم.

شكل 2.1 — أدوات ورشة العمل: من المحرر إلى الطرفية

قبل أن تضغط أي زر تثبيت، من الضروري أن تفهم **ماذا** تُثبّت ولماذا. هذا الفهم سيوفر عليك ساعات من الحيرة لاحقًا.

## 🧩 المكوّنات الثلاثة التي سنحتاجها

| المكوّن | الوظيفة | مثال تشبيهي |
|---|---|---|
| **Python Interpreter** | «المحرك» الذي يفهم وينفّذ كودك | محرك السيارة |
| **محرر أكواد (VS Code)** | البيئة التي تكتب فيها الكود براحة | المكتب ولوحة القيادة |
| **مدير الحزم (uv)** | أداة لتثبيت وإدارة مكتبات إضافية | ميكانيكي يحضر لك قطع غيار |

💡 **نصيحة مهمة:** لا تخلط بين «لغة Python» و"Python". VS Code هي اللغة نفسها، بينما VS Code مجرد برنامج تحرير نصوص ذكي يساعدك على كتابتها بسهولة أكبر. يمكنك تقنيًا كتابة بايثون بأي محرر نصوص بسيط (حتى Notepad!)، لكن هذا يشبه قيادة سيارة بدون مقود مساعد - ممكن، لكن غير عملي إطلاقًا.

# 2.3 تثبيت Python 3.14

## ⚠️ تحذير قبل البدء: مشكلة الإصدارات المتعددة

أحد أكثر الأخطاء شيوعًا بين المبتدئين هو انتهاء المطاف بعدة إصدارات من Python مُثبّتة على نفس الجهاز دون علمهم، مما يسبب ارتباكًا لاحقًا («لماذا الكود يعمل هنا ولا يعمل هناك؟»). سنتجنب هذه المشكلة من البداية باتباع خطوات دقيقة.

## 🪟 التثبيت على Windows

### الخطوة 1: التحقق المسبق

قبل التثبيت، تحقق إن كان لديك بالفعل إصدار قديم:

1. اضغط `Win + R`
2. اكتب `cmd` واضغط Enter
3. في النافذة السوداء التي تظهر، اكتب:

```
Terminal
python --version
```

- إذا ظهرت رسالة مثل `Python 3.9.0` أو أي رقم أقل من 3.12، ستحتاج للتحديث
- إذا ظهرت رسالة خطأ (`is not recognized`)، فهذا يعني Python غير مثبّت، وهذا جيد - ستبدأ نظيفًا

### الخطوة 2: تحميل Python 3.14

1. اذهب إلى الموقع الرسمي: `python.org/downloads`
2. حمّل أحدث إصدار من سلسلة 3.14.x (تجنب أي موقع آخر غير الرسمي)
3. شغّل ملف التثبيت الذي حمّلته

### الخطوة 3: خطوات التثبيت (الأهم!)

عند فتح برنامج التثبيت، ستظهر لك نافذة بها خيارات. هذه الخطوة حاسمة:

```
☑ Add python.exe to PATH    ← هذا الخيار يجب تفعيله!
☐ Install launcher for all users
```

> ⚠️ **تحذير حرج جدًا:** خيار "Add python.exe to PATH" **يجب** أن يكون مُفعّلاً (✅). هذا الخيار يسمح لجهازك بالتعرّف على أمر `python` من أي مكان في الطرفية. تجاهل هذه الخطوة هو **السبب رقم واحد** لمشاكل «Python غير معروف» التي يواجهها المبتدئون.

بعد تفعيل الخيار، اضغط **"Install Now"** وانتظر اكتمال التثبيت.

### الخطوة 4: التحقق من نجاح التثبيت

أغلق نافذة cmd القديمة وافتح واحدة **جديدة** (مهم: نافذة جديدة لتحديث الإعدادات)، ثم اكتب:

```
Terminal
python --version
```

يجب أن تظهر:

```
Python 3.14.0
```

## 🍎 التثبيت على macOS

### الخطوة 1: فحص الإصدار المُدمج

macOS يأتي أحيانًا بإصدار قديم من Python مُدمج في النظام. **لا تستخدمه أبدًا** للتطوير - هو مخصص لعمليات النظام الداخلية فقط.

افتح تطبيق **Terminal** (ابحث عنه في Spotlight بـ `Cmd + Space`)، واكتب:

```
Terminal
python3 --version
```

### الخطوة 2: التثبيت باستخدام Homebrew (الطريقة الموصى بها عالميًا)

إذا لم يكن لديك Homebrew (مدير الحزم الأشهر لـ macOS)، ثبّته أولاً:

```
Terminal
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

بعدها، ثبّت Python:

```
Terminal
brew install python@3.14
```

**الخطوة 3: التحقق**

```
Terminal
python3.14 --version
```

> **نصيحة**: على macOS، يُستخدم الأمر `python3` بدلاً من `python` غالبًا، لأن `python` وحدها قد تشير للإصدار القديم المُدمج بالنظام. سنتعامل مع هذا التفصيل بوضوح أكثر في قسم بيئات العمل الافتراضية لاحقًا.

## 🐧 التثبيت على Linux (Ubuntu/Debian كمثال)

معظم توزيعات Linux تأتي بـ Python مُثبّتًا مسبقًا، لكن غالبًا إصدار قديم.

```
Terminal
# تحديث قائمة الحزم
sudo apt update

# تثبيت الأدوات اللازمة للبناء
sudo apt install software-properties-common -y

# إضافة مستودع يحتوي أحدث إصدارات Python
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update

# تثبيت Python 3.14
sudo apt install python3.14 -y
```

**التحقق:**

```
Terminal
python3.14 --version
```

# 2.4 أداة uv: إدارة حديثة وفائقة السرعة

## 🔄 لماذا uv بدلاً من pip التقليدي؟

لسنوات، كان `pip` هو الأداة الوحيدة لتثبيت مكتبات Python الإضافية. لكن في عامي 2023-2024، ظهرت أداة ثورية اسمها **uv** (من شركة Astral، صانعة أداة Ruff الشهيرة) غيّرت قواعد اللعبة بالكامل.

| الخاصية | pip (التقليدية) | uv (الحديثة) |
|---|---|---|
| **السرعة** | بطيئة نسبيًا | ⚡ أسرع بـ 10-100 مرة |
| **إدارة البيئات الافتراضية** | تحتاج أداة منفصلة (venv) | ✅ مدمجة بالكامل |
| **إدارة إصدارات Python** | غير مدعومة | ✅ مدعومة مباشرة |
| **حل التعارضات بين الحزم** | بدائي | ✅ ذكي ومتقدم |
| **الاعتماد العالمي 2025** | يتراجع تدريجيًا | 📈 يتزايد بسرعة |

> 💡 **قرار تعليمي مهم**: سنعتمد في هذا الكتاب بالكامل على `uv` بدلاً من `pip` التقليدية، لأنها أصبحت المعيار الحديث الموصى به من قبل خبراء المجتمع، وستوفر عليك أخطاء ومشاكل كثيرة كان يعاني منها المبتدئون سابقًا مع الأدوات القديمة.

## 📥 تثبيت uv

### على Windows (PowerShell):

```PowerShell
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

### على macOS / Linux:

```Terminal
curl -LsSf https://astral.sh/uv/install.sh | sh
```

### التحقق من التثبيت (على أي نظام):

أغلق الطرفية وافتح واحدة جديدة، ثم:

```
Terminal
```

```
uv --version
```

يجب أن تظهر نتيجة مثل:

```
uv 0.5.x
```

✅ **ميزة إضافية رائعة:** بإمكان `uv` تثبيت إصدارات Python نفسها! مثلاً، يمكنك كتابة `uv python install 3.14` وستُثبَّت تلقائيًا دون الحاجة للذهاب لموقع Python الرسمي يدويًا. سنستخدم هذه الميزة بكثرة لاحقًا.

---

## 2.5 تجهيز محرر الأكواد: Visual Studio Code

### 🤔 لماذا VS Code تحديدًا؟

يوجد عشرات المحررات والبيئات (PyCharm, Sublime Text, Vim...)، لكن VS Code يبقى الخيار الأمثل للمبتدئين والمحترفين على حد سواء للأسباب التالية:

- ✅ **مجاني بالكامل** ومفتوح المصدر
- ✅ **خفيف** نسبيًا على موارد الجهاز
- ✅ **دعم استثنائي لـ Python** عبر الإضافات الرسمية
- ✅ **الأكثر استخدامًا عالميًا** (حسب استطلاعات Stack Overflow السنوية)
- ✅ يعمل بنفس الطريقة على Windows/Mac/Linux

### 📥 خطوات التثبيت

1. اذهب إلى `code.visualstudio.com`
2. حمّل النسخة المناسبة لنظامك
3. ثبّته باتباع التعليمات الافتراضية (بدون تعقيدات)

### 🧩 الإضافات الأساسية (لا تُثبّت أكثر من هذا في البداية!)

أحد الأخطاء الشائعة بين المبتدئين هو تثبيت عشرات الإضافات ظنًا أنها ستجعلهم «محترفين أسرع». النتيجة غالبًا جهاز بطيء وإرباك غير ضروري. إليك فقط **ما تحتاجه فعليًا**:

افتح VS Code، اذهب لأيقونة الإضافات (Extensions) في الشريط الجانبي (أو `Ctrl+Shift+X`)، وابحث عن:

| الإضافة | الوظيفة | لماذا ضرورية؟ |
|---|---|---|
| **Python** (من Microsoft) | الدعم الأساسي الكامل للغة | التلوين، الإكمال التلقائي، تشغيل الكود |
| **Pylance** (من Microsoft) | تحليل ذكي للكود | يكتشف الأخطاء قبل التشغيل |
| **Ruff** (من Astral) | فحص وتنسيق تلقائي للكود | يُعلّمك الأسلوب الصحيح من البداية |
| **Error Lens** | إبراز الأخطاء مباشرة في السطر | رؤية فورية للمشاكل |

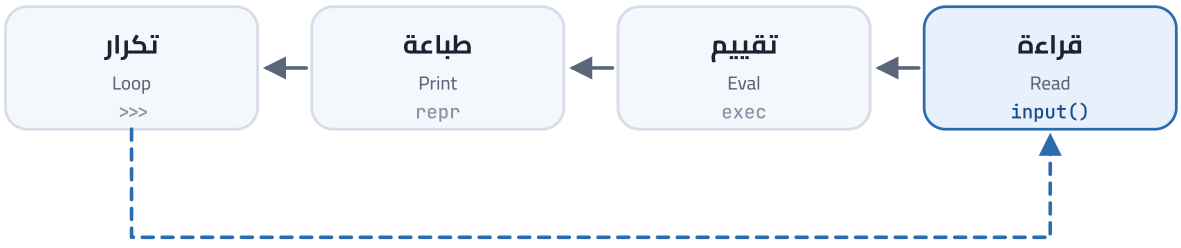
⚠ **تحذير:** عند تثبيت إضافة "Python"، ستُثبَّت "Pylance" تلقائيًا معها غالبًا كجزء من الحزمة - لا تقلق إن وجدتهما مرتبطتين.

### ⚙ إعداد سريع بعد التثبيت

افتح VS Code، واضغط `Ctrl + ,` (أو `Cmd + ,` على Mac) لفتح الإعدادات، وابحث عن `Format On Save`، وفعّله (✅). هذا سيجعل VS Code ينسّق كودك تلقائيًا بأسلوب احترافي معياري كل مرة تحفظ فيها الملف - عادة ستشكر نفسك عليها لاحقًا.

---

## 2.6 أول تواصل مع Python: الـ REPL




شكل 2.2 — دورة REPL: قراءة ثم تقييم ثم طباعة

### 💬 ما هو REPL؟

قبل أن نكتب ملفات كود حقيقية، دعنا نتعرّف على طريقة تفاعلية رائعة لتجربة Python فورًا: الـ **REPL** (اختصار لـ Read-Eval-Print-Loop).

فكّر في REPL كـ«محادثة مباشرة» مع Python: تكتب سطرًا، يُنفَّذ فورًا، وترى النتيجة مباشرة.

### 🎮 جرّب بنفسك

افتح الطرفية (Terminal في VS Code، أو cmd/Terminal في نظامك)، واكتب:

```
Terminal
```

```
python
```

(أو `python3` على Mac/Linux إن لزم الأمر)

ستظهر لك واجهة مثل هذه:

```
Python 3.14.0 (main, ...)
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

الرمز `>>>` يعني أن Python **جاهزة وتنتظر** أوامرك. جرّب كتابة:

```
Python
```

```
>>> 2 + 2
4
```

```
Python
```

```
>>> "مرحبا" + " " + "بالعالم"
'مرحبا بالعالم'
```

```
Python
```

```
>>> print("أنا أتعلم Python!")
أنا أتعلم Python!
```

> 💡 **نصيحة عملية:** استخدم REPL دائمًا لتجربة سطر كود سريع أو اختبار فكرة صغيرة، بدل إنشاء ملف كامل لكل تجربة. أغلب المبرمجين المحترفين يستخدمونه يوميًا لاختبارات سريعة.

للخروج من REPL في أي وقت:

```
Python
```

```
>>> exit()
```

# 2.7 كتابة وتشغيل أول ملف كود حقيقي

الـ REPL رائع للتجارب السريعة، لكن المشاريع الحقيقية تُكتب في **ملفات** تنتهي بامتداد `.py`.

### 📂 الخطوة 1: تنظيم مجلد العمل

قبل أي شيء، أنشئ مجلدًا مخصصًا لتمارين الكتاب:

```
Terminal

# أنشئ مجلدًا جديدًا
mkdir python-journey
cd python-journey
```

### 📝 الخطوة 2: فتح المجلد في VS Code

في الطرفية، داخل المجلد الذي أنشأته:

```
Terminal

code .
```

(النقطة `.` تعني «افتح هذا المجلد الحالي في VS Code»)

### ✍️ الخطوة 3: إنشاء أول ملف

في VS Code، أنشئ ملفًا جديدًا باسم `hello.py`. لكن بدلاً من مجرد `Hello World` التقليدي المُعاد استخدامه آلاف المرات، سنكتب شيئًا تفاعليًا بسيطًا يعطيك إحساسًا حقيقيًا بقوة البرمجة من اليوم الأول:

```python
Python  hello.py

# أول برنامج تفاعلي حقيقي

name = input("ما اسمك؟ ")
age = input("كم عمرك؟ ")

print(f"مرحبًا {name}! 👋")
print(f"عمرك {age} سنة.")

age_in_days = int(age) * 365
print(f"هذا يعني أنك عشت تقريبًا {age_in_days} يومًا على هذه الأرض!")
```

### ▶️ الخطوة 4: تشغيل الكود

في VS Code، يوجد عدة طرق لتشغيل الملف:

**الطريقة 1 (الأسهل):** اضغط الزر المثلث ▶️ في الزاوية العلوية اليمنى

**الطريقة 2 (الاحترافية - تعلّمها من الآن):** استخدم الطرفية المدمجة في VS Code ( `Ctrl+`` ) واكتب:

```
Terminal
```

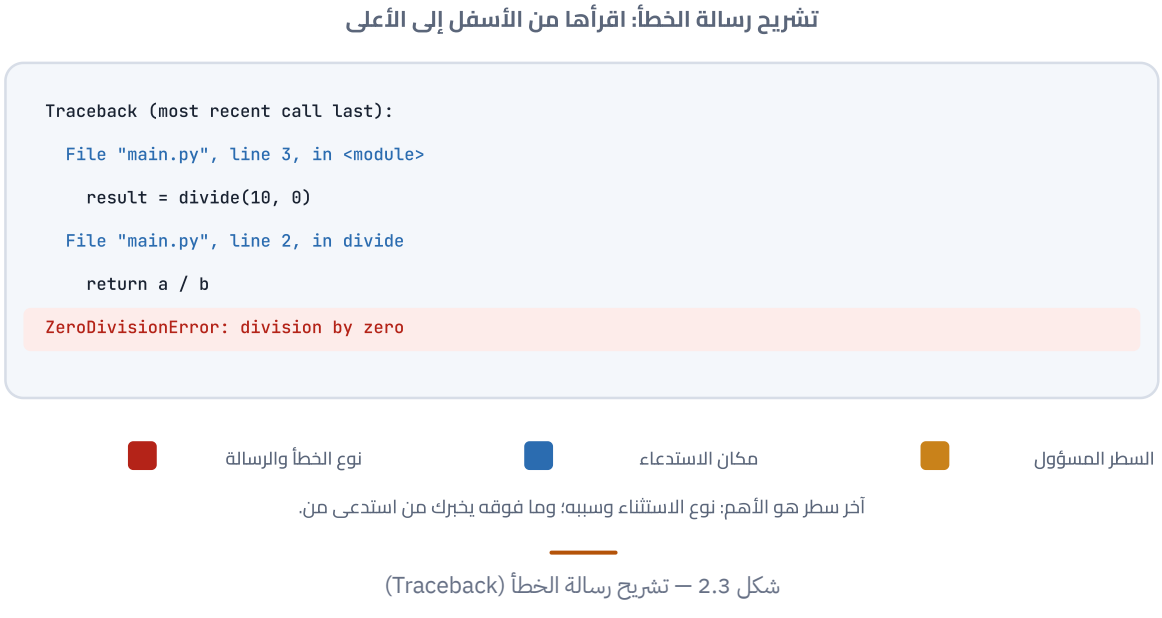
```
python hello.py
```

ستظهر النتيجة في الطرفية أسفل الشاشة، وسيطلب منك البرنامج إدخال اسمك وعمرك تفاعليًا.

✅ **لماذا الطريقة الثانية أفضل للتعلّم؟** لأنها تعكس بالضبط كيف يعمل المبرمجون المحترفون - عبر سطر الأوامر مباشرة - وستحتاج هذه المهارة بشدة لاحقًا عند العمل بمشاريع حقيقية خارج VS Code.

---

## 2.8 فن قراءة رسائل الخطأ (مهارة تُغيّر رحلتك بالكامل)




شكل 2.3 — تشريح رسالة الخطأ (Traceback)

### 🎯 لماذا هذا القسم هو الأهم في الفصل؟

تذكّر ما قلناه في الفصل الأول: الأخطاء ليست أعداءك، بل معلّموك. لكن لتستفيد منها، يجب أن تتعلّم **كيف تقرأها**. دعنا نرتكب خطأ مقصودًا ونحلّله معًا.

## 🔬 تجربة: خطأ مقصود

عدّل ملف `hello.py` وأضف هذا السطر الخاطئ عمدًا في النهاية:

```python
print("النتيجة النهائية: " + age_in_days)
```

شغّل الملف وستظهر رسالة خطأ تشبه هذه:

```
Traceback (most recent call last):
  File "hello.py", line 10, in <module>
    print("النتيجة النهائية: " + age_in_days)
                                  ~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

## 🧩 تشريح رسالة الخطأ خطوة بخطوة

هذه الرسالة تبدو مرعبة لأول وهلة، لكنها في الحقيقة **خريطة دقيقة** تخبرك بكل شيء تحتاجه:

```
Traceback (most recent call last):
  ↑ هذا فقط يعني "إليك تتبّع مسار الخطأ"

File "hello.py", line 10, in <module>
  ↑ اسم الملف ورقم السطر بالضبط حيث حدثت المشكلة

print("النتيجة النهائية: " + age_in_days)
  ↑ السطر الفعلي الذي سبّب الخطأ (معروض لك مباشرة!)

TypeError: can only concatenate str (not "int") to str
  ↑ شرح دقيق: "لا يمكن دمج نوع الخطأ + شرح دقيق: "لا يمكن دمج str مع int"
```

## 🔑 القاعدة الذهبية لقراءة أي خطأ في Python

```
1. اقرأ السطر الأخير أولاً (نوع الخطأ + الشرح) - هذا 80% من الحل
2. انظر لرقم السطر المذكور - اذهب إليه مباشرة في كودك
3. اسأل نفسك: "ماذا يحاول هذا السطر فعله، ولماذا يرفضه Python؟"
```

في مثالنا: `age_in_days` هو رقم (int)، بينما النص قبله هو str. Python لا تسمح بدمج (concatenate) نوعين مختلفين بـ `+` مباشرة. الحل:

```python
print("النتيجة النهائية: " + str(age_in_days))
```

> 💡 **لا تقلق إن لم تفهم `str` و `int` بعمق الآن** - سنشرحهما بالتفصيل الكامل في الفصل القادم. الهدف هنا فقط تعلّم **منهجية** قراءة الخطأ، لا حل هذا الخطأ تحديدًا.

### 📚 أنواع الأخطاء الأكثر شيوعًا للمبتدئين (مرجع سريع)

| نوع الخطأ | متى يظهر | مثال شائع |
|---|---|---|
| `SyntaxError` | خطأ في قواعد كتابة الكود نفسه | نسيان `:` بعد `if` |
| `NameError` | استخدام متغير غير موجود | خطأ إملائي في اسم متغير |
| `TypeError` | خلط أنواع بيانات غير متوافقة | جمع نص مع رقم مباشرة |
| `IndentationError` | مشكلة في المسافات البادئة | عدم محاذاة الأسطر بشكل صحيح |
| `ZeroDivisionError` | قسمة أي رقم على صفر | `10 / 0` |

> ⚠️ **تحذير من عادة سيئة شائعة جدًا:** كثير من المبتدئين عند رؤية خطأ طويل، يصابون بالذعر وينسخون **الرسالة بأكملها** إلى جوجل أو ChatGPT دون قراءتها أولاً. تدرّب على قراءة السطر الأخير بنفسك أولاً - ستندهش كم مرة ستفهم المشكلة بنفسك خلال ثوانٍ فقط.

---

## 2.9 لمحة عن البيئات الافتراضية (Virtual Environments)

### 🤔 لماذا نحتاج هذا المفهوم من البداية؟

تخيل أنك تعمل على مشروعين مختلفين: الأول يحتاج نسخة قديمة من مكتبة معينة، والثاني يحتاج أحدث نسخة من نفس المكتبة. إذا ثبّت كل شيء «عالميًا» على جهازك، ستحدث تعارضات حتمًا.

**البيئة الافتراضية (Virtual Environment)** هي حل بسيط وعبقري: «فقاعة» معزولة خاصة بكل مشروع، تحتوي نسخة Python والمكتبات الخاصة به فقط، دون التأثير على بقية مشاريعك أو نظامك.

> 💡 سنتعمّق في هذا المفهوم بالتفصيل الكامل في فصل لاحق مخصص لإدارة المشاريع الاحترافية. الآن، فقط تذكّر: **كل مشروع حقيقي يحتاج بيئته الافتراضية الخاصة** - هذه قاعدة ذهبية لا تُخالف أبدًا في الممارسة الاحترافية.

### 🎮 تجربة سريعة بـ uv

داخل مجلد مشروعك، جرّب:

```
Terminal
```

```
# إنشاء بيئة افتراضية جديدة
uv venv

# تفعيلها (الأوامر تختلف حسب نظامك)
# على Windows:
.venv\Scripts\activate

# على Mac/Linux:
source .venv/bin/activate
```

بعد التفعيل، ستلاحظ تغيّرًا بسيطًا في الطرفية (عادة اسم البيئة يظهر بين قوسين)، وهذا يعني أنك الآن «داخل» بيئة معزولة. سنستخدم هذا بكثرة بدءًا من مشاريعنا الأولى في الفصول القادمة.

---

## 2.10 أخطاء التهيئة الشائعة وحلولها السريعة

### ⚠ المشكلة 1: `'python' is not recognized as an internal or external command`

**السبب**: لم يتم تفعيل خيار "Add to PATH" أثناء التثبيت (على Windows).

**الحل**: أعد تثبيت Python، وتأكد من تفعيل ذلك الخيار هذه المرة، أو أضف المسار يدويًا لمتغيرات البيئة (خطوة متقدمة، البحث عنها بـ"Add Python to PATH manually" يعطيك حلولاً مفصلة).

### ⚠ المشكلة 2: VS Code لا «يرى» Python المُثبَّت

**السبب**: VS Code لم يكتشف تلقائيًا أين Python مُثبَّت.

**الحل**: اضغط `Ctrl+Shift+P` (أو `Cmd+Shift+P`)، اكتب `Python: Select Interpreter`، واختر الإصدار الصحيح من القائمة الظاهرة.

### ⚠ المشكلة 3: الأحرف العربية تظهر كرموز غريبة

**السبب**: مشكلة في ترميز الملف (Encoding).

**الحل**: تأكد في VS Code (أسفل يمين الشاشة) أن الترميز هو `UTF-8`. إذا لم يكن كذلك، اضغط عليه واختر `Reopen with Encoding` ثم `UTF-8`.

---

# 2.11 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

تأكد من نجاح تثبيت uv، Python، وVS Code بتنفيذ الأوامر التالية والتقاط لقطة شاشة (أو كتابة النتيجة) لكل منها:

```
Terminal
```

```
python --version
uv --version
code --version
```

### 🟡 تمرين 2 (مستوى متوسط)

عدّل ملف `hello.py` ليسأل المستخدم أيضًا عن **مدينته**، ثم يطبع جملة كاملة تجمع كل المعلومات (الاسم، العمر، المدينة) في رسالة ترحيبية واحدة منسّقة بشكل جميل باستخدام f-strings.

### 🔴 تمرين 3 (تحدي)

تعمّد كتابة 3 أنواع مختلفة من الأخطاء (مثلاً: `SyntaxError` بنسيان قوس، `NameError` باستخدام متغير غير معرّف، و `TypeError` بجمع نص مع رقم)، ثم اكتب تحليلك الخاص لكل رسالة خطأ ظهرت، موضحًا: ما السبب؟ وأين بالضبط في الكود حدثت المشكلة؟

---

# 2.12 مشروع تطبيقي صغير: «بطاقة التعريف الذكية»

### 📋 المهمة

بالاعتماد على كل ما تعلمته في هذا الفصل (كتابة ملف، استخدام `input`، `print`، f-strings)، ابنِ برنامجًا يُسمى `id_card.py` يقوم بـ:

1. يسأل المستخدم عن: الاسم الكامل، سنة الميلاد، المدينة، والهواية المفضلة
2. يحسب العمر التقريبي (بافتراض أن السنة الحالية 2025)
3. يطبع «بطاقة تعريف» منسّقة بشكل جميل، تشمل كل المعلومات، باستخدام رموز تعبيرية (emojis) وحدود نصية بسيطة (مثل `=====` ) لتبدو احترافية

### 💻 مثال على الشكل المتوقع للنتيجة

```
==================================
🪪  بطاقة التعريف الشخصية
==================================
👤 الاسم: أحمد محمد
🎂 العمر التقريبي: 28 سنة
🏙️ المدينة: الرياض
🎯 الهواية: القراءة
==================================
```

> 🎯 لا تقلق إن استغرقت وقتًا في هذا التمرين - هو مصمم عمدًا ليدفعك لتطبيق كل ما تعلمته بحرية، دون إجابة واحدة «صحيحة». جرّب، أخطئ، اقرأ رسائل الأخطاء، وصحّح. هذه هي حلقة التعلم الحقيقية.

---

## 2.13 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين REPL وكتابة كود في ملف `.py` ؟ ومتى تستخدم كل طريقة؟
2. لماذا نحتاج لتفعيل خيار "Add Python to PATH" عند التثبيت على Windows تحديدًا؟
3. اذكر الخطوات الثلاث لقراءة أي رسالة خطأ في Python بمنهجية صحيحة.
4. ما الفرق بين `pip` و `uv` ؟ ولماذا اعتمدنا `uv` في هذا الكتاب؟
5. ما هي البيئة الافتراضية (Virtual Environment) باختصار، ولماذا هي مهمة؟

✅ الإجابات النموذجية

1. REPL يُنفّذ كل سطر فورًا ومناسب للتجارب السريعة، بينما ملفات `.py` تُحفظ وتُشغَّل كاملة دفعة واحدة، وهي الطريقة المناسبة للمشاريع والبرامج الحقيقية.
2. لأن هذا الخيار يسمح لنظام التشغيل بالتعرّف على أمر `python` من أي مكان في الطرفية دون كتابة المسار الكامل يدويًا في كل مرة.
3. أولاً: اقرأ السطر الأخير (نوع الخطأ وشرحه). ثانيًا: حدد رقم السطر المذكور في كودك. ثالثًا: اسأل نفسك ماذا كان هذا السطر يحاول فعله ولماذا رفضته Python.
4. `pip` هي الأداة التقليدية لإدارة الحزم، بينما `uv` أداة حديثة أسرع بكثير، وتدير أيضًا إصدارات Python والبيئات الافتراضية بشكل مدمج. اعتمدناها لأنها الاتجاه الحديث الموصى به عالميًا.
5. هي «فقاعة» معزولة خاصة بكل مشروع، تحتوي نسخة Python ومكتباته الخاصة فقط، لتجنب التعارضات بين متطلبات المشاريع المختلفة على نفس الجهاز.

---

## 2.14 📌 ملخص الفصل (Cheat Sheet)

| الأمر/المفهوم | الوظيفة |
|---|---|
| `python --version` | التحقق من إصدار Python المُثبَّت |
| `uv venv` | إنشاء بيئة افتراضية جديدة |
| `uv --version` | التحقق من تثبيت uv |
| `python filename.py` | تشغيل ملف كود بايثون |
| `code .` | فتح المجلد الحالي في VS Code |
| REPL | وضع تفاعلي لتجربة أكواد سريعة (`>>>`) |
| `Traceback` | تتبّع مسار حدوث الخطأ في الكود |
| Virtual Environment | بيئة معزولة لكل مشروع لتجنب تعارض المكتبات |

## 🎯 ماذا بعد؟

الآن بيئتك جاهزة بالكامل، وتعلّمت كيف «تتحدث» مع Python وتفهم ردودها (حتى عند الأخطاء). في **الفصل الثالث**، سنبدأ رحلتنا الفعلية في عالم اللغة: **المتغيرات** - اللبنة الأساسية الأولى لأي برنامج، مع فهم عميق لما يحدث فعليًا في ذاكرة الكمبيوتر عند كل سطر كود تكتبه.

الجزء الثاني · 8 فصول

02

# الأساسيات المتينة

المتغيّرات، النصوص، الأرقام، القرار، التكرار، هياكل البيانات، والدوال. ثمانية فصول تُتقن فيها مفردات اللغة التي ستكتب بها كل برامجك.

3 · المتغيرات
4 · التعامل مع النصوص
5 · الأرقام والعمليات الحسابية
6 · اتخاذ القرار
7 · التكرار
8 · هياكل البيانات
9 · القواميس والمجموعات
10 · الدوال

03

فصل 03 / 26 · الجزء الثاني

# المتغيرات - أول لبنة

12 دقيقة قراءة | 17 قسمًا | 36 مثالًا | 2,308 كلمة

**في هذا الفصل**

3.1 مقدمة: لماذا المتغيرات هي نقطة البداية الحقيقية؟
3.2 ما هو المتغير؟ (الفهم الصحيح، لا المبسّط الخاطئ)
3.3 إنشاء المتغيرات: عملية الإسناد (Assignment)
3.4 قواعد وأعراف تسمية المتغيرات
3.5 الأنواع الأساسية للبيانات (Data Types)
3.6 Python لغة ديناميكية التنميط (Dynamically Typed)
3.7 Type Hints: الكتابة الواضحة من اليوم الأول
3.8 تحويل الأنواع (Type Casting)
3.9 الإسناد المتعدد وتبديل القيم (تقنيات Python الأنيقة)
3.10 الثوابت (Constants): عُرف لا قاعدة صارمة
3.11 جدول مقارنة شامل: أنواع البيانات الأساسية
3.12 أخطاء شائعة يرتكبها المبتدئون (تجنّبها الآن)
3.13 أفضل الممارسات: ملخص شامل
3.14 تمارين عملية للفصل
3.15 مشروع تطبيقي: «بطاقة التعريف الذكية - النسخة المُطوَّرة»
3.16 اختبر فهمك (Quiz سريع)
3.17 ملخص الفصل (Cheat Sheet)

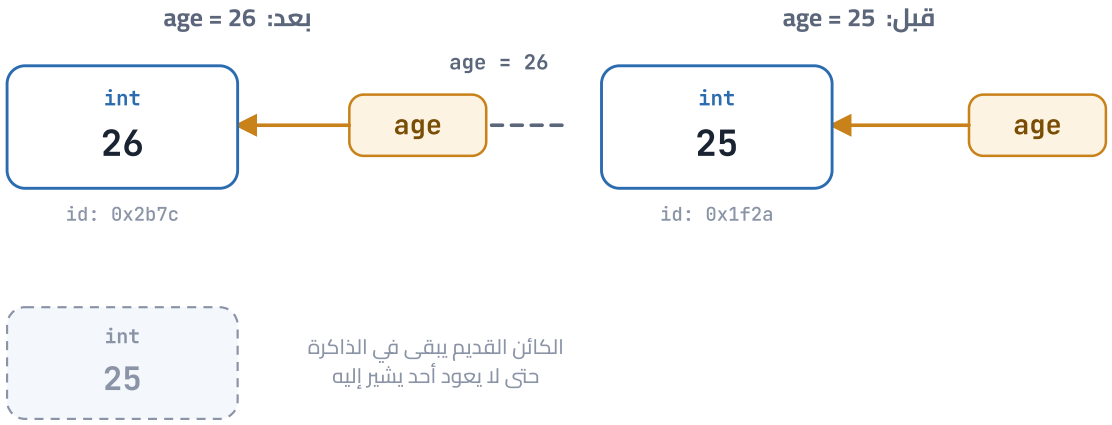
> *«المتغيرات هي الذاكرة التي تمنح برنامجك القدرة على 'التذكّر'. بدونها، كل برنامج سينسى كل شيء بمجرد الانتقال للسطر التالي.»*

## 3.1 📖 مقدمة: لماذا المتغيرات هي نقطة البداية الحقيقية؟

في الفصل السابق، كتبت برنامجك الأول الذي يسأل عن اسمك ويطبع ترحيبًا. لكن هل تساءلت: كيف «تذكّر» البرنامج اسمك بين سطر `input()` وسطر `print()`؟ الإجابة: **المتغيرات**.

المتغيرات هي أبسط وأهم مفهوم في أي لغة برمجة على الإطلاق. بدونها، لا يمكن للبرنامج تخزين أي معلومة، ولا معالجتها، ولا استخدامها لاحقًا. كل شيء ستتعلمه لاحقًا - من القوائم إلى الدوال إلى البرمجة الكائنية - مبني فوق هذا المفهوم الأساسي.

## 3.2 ما هو المتغير؟ (الفهم الصحيح، لا المبسّط الخاطئ)




المتغيّر ليس صندوقًا، بل اسم مربوط بكائن — والإسناد الجديد يعيد ربطه بكائن آخر.

شكل 3.1 — المتغيّر ملصق مربوط بكائن، والإسناد يعيد الربط

### التشبيه الشائع (ولماذا هو غير دقيق تمامًا في Python)

أغلب الكتب تشرح المتغير كـ«صندوق» تضع بداخله قيمة:

```
يحتوي 25 ← صندوق باسم "age"]
```

هذا التشبيه مفيد للبداية، لكنه **غير دقيق** بخصوص كيفية عمل Python فعليًا، وسيُسبب لك التباسًا لاحقًا عند تعلّم القوائم والكائنات. لذلك، سنتعلّم النموذج الصحيح من البداية.

### النموذج الصحيح: المتغيرات كـ«ملصقات» (Labels)

في Python، الصورة الأدق هي:

> ***المتغير ليس صندوقًا يحتوي قيمة، بل «ملصق» (اسم) يُشير إلى مكان في الذاكرة تُخزَّن فيه القيمة.***

تخيّل الأمر هكذا:

```python
age = 25
```

ما يحدث فعليًا في الذاكرة:

```
الذاكرة:
┌──────────────┐
│  كائن ←  │  25 القيمة (Object) مُخزَّن في مكان ما بالذاكرة
└──────────────┘

      ↑
      │ (يُشير إليه)
      │
   الملصق "age"
```

الاسم `age` هو مجرد **ملصق** يُشير إلى حيث يوجد الرقم 25 في الذاكرة. هذا الفرق يبدو بسيطًا الآن، لكنه سيكون حاسمًا لفهم سلوكيات معينة لاحقًا (خصوصًا عند التعامل مع القوائم في الفصل السابع).

## 🔬 إثبات عملي: دالة `id()`

يمكنك فعليًا «رؤية» أين تُخزَّن القيمة في الذاكرة باستخدام دالة `id()` المُدمجة:

**Python**

```
age = 25
print(id(age))  # 140712834567280 :سيطبع رقمًا طويلاً مثل
```

هذا الرقم هو عنوان الذاكرة (مبسّط) حيث يوجد الكائن `25`. جرّب هذا:

**Python**

```
age = 25
age_copy = age

print(id(age))       # 140712834567280
print(id(age_copy))  # 140712834567280  ← نفس العنوان بالضبط!
```

> **ملاحظة مهمة:** كلا المتغيرين `age` و `age_copy` يُشيران لنفس الكائن `25` في الذاكرة. لم يتم «نسخ» الرقم فعليًا، بل تم إنشاء ملصق جديد يُشير لنفس المكان. هذا السلوك طبيعي تمامًا للأرقام الصغيرة والقيم البسيطة، وسنتعمّق في تبعاته لاحقًا مع أنواع بيانات أكثر تعقيدًا.

# 3.3 إنشاء المتغيرات: عملية الإسناد (Assignment)

## ✍️ الصيغة الأساسية

```python
variable_name = value
```

علامة `=` هنا تُسمى **عامل الإسناد (Assignment Operator)**، وهي **لا تعني** «يساوي» بالمعنى الرياضي إطلاقًا، بل تعني: **«اجعل هذا الاسم يُشير إلى تلك القيمة»**.

```python
name = "سارة"
age = 30
height = 1.65
is_student = False
```

## ⚠️ تحذير حرج: الخلط بين `=` و `==`

هذا من أكثر الأخطاء شيوعًا بين المبتدئين على الإطلاق، ويستمر معهم أحيانًا لأسابيع قبل أن يستوعبوه تمامًا:

| الرمز | الاسم | الوظيفة | مثال |
|---|---|---|---|
| `=` | Assignment (إسناد) | **تخزين** قيمة في متغير | `age = 25` |
| `==` | Equality (مقارنة) | **مقارنة** إن كانت قيمتان متساويتين | `age == 25` |

```python
age = 25        # ✅ صحيح: تخزين القيمة 25 في age
print(age == 25)  # ✅ صحيح: مقارنة - هل age تساوي 25؟ → True

age = 25
print(age = 25)  # ❌ خطأ! SyntaxError - لا يمكن استخدام = داخل print()
```

🎯 **قاعدة ذهبية للتذكّر:** استخدم `=` عندما تريد «**تخزين**» قيمة، واستخدم `==` عندما تريد «**السؤال**» إن كانت قيمتان متساويتين. سنتعمّق في `==` أكثر في فصل التحكم بسير البرنامج (الفصل السادس).

# 3.4 قواعد وأعراف تسمية المتغيرات

## 📏 القواعد الإلزامية (يجب اتباعها، وإلا سيحدث خطأ)

Python تفرض قواعد صارمة لأسماء المتغيرات:

```python
# ✅ أسماء صحيحة
name = "أحمد"
age2 = 25
_private_value = 100
userName = "test"

# ❌ أسماء غير صحيحة (ستُسبب SyntaxError)
2age = 25          # لا يمكن البدء برقم
user-name = "x"    # الشرطة - غير مسموحة
class = "test"     # "class" كلمة محجوزة في Python
```

**القواعد الإلزامية:**

1. يجب أن يبدأ الاسم بحرف أو شرطة سفلية `_` (لا يبدأ برقم أبدًا)
2. يمكن أن يحتوي على حروف، أرقام، وشرطات سفلية فقط (لا مسافات، لا رموز خاصة)
3. حساس لحالة الأحرف (`Name` و `name` متغيران مختلفان تمامًا)
4. لا يمكن استخدام **الكلمات المحجوزة** (Reserved Keywords) مثل `if` , `for` , `class` , `True` , `None` ...

## 🎨 الأعراف الاحترافية (Convention) - ليست إلزامية، لكن اتبعها دائمًا

هنا يأتي الفرق بين كود «يعمل» وكود «احترافي». المجتمع العالمي لمبرمجي Python يتبع معيارًا موثقًا رسميًا يُسمى **PEP 8**، وأهم قاعدة فيه لتسمية المتغيرات:

> ***استخدم snake_case دائمًا:*** *كل الحروف صغيرة، والكلمات مفصولة بشرطة سفلية.*

```python
# ✅ الأسلوب الاحترافي الموصى به (snake_case)
first_name = "محمد"
total_price = 150.5
is_logged_in = True

# ❌ أساليب تعمل لكنها غير احترافية في Python
firstName = "محمد"      # camelCase - شائع في JavaScript وليس Python
FirstName = "محمد"      # PascalCase - يُستخدم للـ Classes فقط لاحقًا
FIRST_NAME = "محمد"     # كله كبير - يُستخدم للثوابت (Constants) فقط
```

## 🔄 مقارنة سريعة مع لغات أخرى

| اللغة | العُرف الشائع | مثال |
|---|---|---|
| **Python** | snake_case | `user_name` |
| **JavaScript** | camelCase | `userName` |
| **Java** | camelCase | `userName` |

⚠️ **تحذير للمبرمجين القادمين من لغات أخرى:** إذا كنت تعرف JavaScript أو Java مسبقًا، ستميل عفويًا لكتابة `camelCase`. تجنّب هذا في Python - الكود الذي لا يتبع snake_case يُعتبر «كودًا غير أصيل» (not Pythonic) في أي مراجعة كود احترافية.

## 👄 الأسماء ذات المعنى: فرق بين مبرمج مبتدئ ومحترف

```python
# ❌ أسماء غامضة (تعمل لكنها سيئة)
x = 25
y = "محمد"
z = x * 12

# ✅ أسماء واضحة ومعبّرة (احترافية)
age = 25
name = "محمد"
age_in_months = age * 12
```

💡 **نصيحة الخبراء:** الكود يُقرأ أكثر بكثير مما يُكتب. قد تعود لكودك بعد 6 أشهر ولن تتذكّر ماذا كان يعني `x`. استثمر ثانيتين إضافيتين الآن في اختيار اسم واضح، ستوفر عليك دقائق (أو ساعات) من الحيرة لاحقًا.

## 3.5 الأنواع الأساسية للبيانات (Data Types)

Python تحتوي أنواعًا كثيرة من البيانات، لكن في هذا الفصل سنتعرّف على **الأنواع الأساسية (Primitive Types)** التي ستستخدمها يوميًا.

### 🔢 الأعداد الصحيحة (Integer - `int`)

أي رقم صحيح، موجب أو سالب، بدون فاصلة عشرية:

```python
age = 25
temperature = -5
year = 2025

print(type(age))  # <class 'int'>
```

💡 **ميزة فريدة في Python**: على عكس لغات أخرى كثيرة، أعداد `int` في Python ليس لها حد أقصى للحجم (محدودة فقط بذاكرة جهازك). يمكنك فعليًا تخزين أرقام بمئات الخانات دون أي مشاكل:

```python
huge_number = 12345678901234567890123456789O
print(huge_number * 2)  # يعمل بدون أي مشاكل!
```

### 🔢 الأعداد العشرية (Float)

أي رقم يحتوي فاصلة عشرية:

```python
height = 1.75
price = 99.99
pi_approx = 3.14159

print(type(height))  # <class 'float'>
```

⚠️ **تحذير مهم - فخ الدقة العشرية**: ستُفاجأ لاحقًا (في الفصل الخامس) بأن Python (وكل لغات البرمجة تقريبًا) تعاني من مشكلة دقة في الأعداد العشرية:

```python
print(0.1 + 0.2)  # 0.30000000000000004 ← ليس 0.3 تمامًا!
```

هذا **ليس خطأ** في Python، بل طبيعة كيفية تخزين الكمبيوترات للأعداد العشرية في الذاكرة (نظام ثنائي). سنتعلّم كيف نتعامل مع هذا بذكاء في الفصل الخامس.

## 📝 النصوص (String - `str`)

أي سلسلة من الحروف، محاطة بعلامات اقتباس:

```python
name = "سارة"
city = 'الرياض'  # علامات مفردة أو مزدوجة - كلاهما صحيح
message = "اليوم هو يوم جميل!"

print(type(name))  # <class 'str'>
```

💡 **نصيحة اتساق:** Python لا تفرّق بين `'single quotes'` و `"double quotes"` وظيفيًا، لكن اختر أسلوبًا واحدًا والتزم به في كل مشروعك للاتساق. المعيار الأكثر شيوعًا عالميًا هو علامات الاقتباس المزدوجة `"..."`.

## ✅ القيم المنطقية (Boolean - `bool`)

قيمتان فقط: `True` أو `False` (لاحظ حرف كبير إلزامي في البداية):

```python
is_student = True
is_raining = False

print(type(is_student))  # <class 'bool'>
```

⚠️ **خطأ شائع جدًا:** كتابة `true` أو `false` بحروف صغيرة (كما في JavaScript أو Java) ستُسبب `NameError` في Python:

```python
is_active = true  # ❌ NameError: name 'true' is not defined
is_active = True  # ✅ صحيح
```

### 🚫 القيمة الفارغة ( `None` )

تمثّل «لا شيء» أو «غياب القيمة» - مفهوم مهم جدًا سنستخدمه كثيرًا لاحقًا:

```python
middle_name = None  # الشخص ليس لديه اسم أوسط

print(type(middle_name))  # <class 'NoneType'>
```

> 💡 **متى تُستخدم `None`؟** عندما تريد التعبير عن «لا توجد قيمة بعد» أو «فارغ عمدًا»، على عكس الصفر `0` أو النص الفارغ `""` اللذين هما قيم فعلية موجودة. سنرى استخدامات عملية قوية لهذا في فصول الدوال والبرمجة الكائنية.

---

## Python `3.6` لغة ديناميكية التنميط (Dynamically Typed)

### 🔄 الفهم الجوهري

في لغات مثل C++ أو Java، يجب عليك **تحديد نوع المتغير صراحة** عند إنشائه، ولا يمكن تغييره لاحقًا:

```java
// في Java (ليست Python!)
int age = 25;
age = "خمسة وعشرون";  // ❌ خطأ! لا يمكن تغيير النوع
```

لكن في Python، النوع **يُحدَّد تلقائيًا** حسب القيمة، ويمكن أن **يتغيّر** في أي وقت:

```python
age = 25          # الآن age من نوع int
print(type(age))  # <class 'int'>

age = "مسموح تمامًا في Python! خمسة وعشرون"  # ✅
print(type(age))      # <class 'str'>
```

### 🔄 مقارنة: ثابت التنميط مقابل ديناميكي التنميط

| الخاصية | Statically Typed (Java, C++) | Dynamically Typed (Python) |
|---|---|---|
| **تحديد النوع** | إلزامي وصريح عند الإنشاء | تلقائي حسب القيمة |
| **تغيير النوع لاحقًا** | ❌ غير مسموح | ✅ مسموح |
| **اكتشاف أخطاء الأنواع** | قبل التشغيل (compile time) | أثناء التشغيل فقط |
| **سرعة الكتابة** | أبطأ (تفاصيل أكثر) | ⚡ أسرع وأكثر مرونة |
| **احتمالية أخطاء مخفية** | أقل | ⚠️ أعلى نسبيًا |

> ⚠️ **تحذير احترافي مهم:** مرونة Python هذه سلاح ذو حدين. في مشاريع صغيرة، هي ميزة رائعة تُسرّع التطوير. لكن في مشاريع كبيرة بفرق متعددة، قد يُسبب هذا أخطاءً مخفية يصعب اكتشافها (مثل تمرير نص لدالة تتوقع رقمًا دون أي تحذير فوري). هذا بالضبط سبب ابتكار **Type Hints** التي سنتعلّمها في القسم القادم - وهي حل Python الأنيق لهذه المشكلة.

---

## Type Hints `3.7` : الكتابة الواضحة من اليوم الأول

### 🤔 لماذا نتعلّم هذا بهذا التبكير؟

معظم الكتب التعليمية تؤجّل هذا المفهوم لفصول متقدمة جدًا، أو تتجاهله كليًا. لكننا سنكسر هذا التقليد، لأن **Type Hints** أصبحت جزءًا أساسيًا من أسلوب كتابة Python الحديث والاحترافي، ومن الأفضل بناء العادة الصحيحة من البداية بدل تعلّم «الطريقة القديمة» ثم «تصحيحها» لاحقًا.

### ✍️ الصيغة الأساسية

```python
variable_name: type = value
```

مقارنة بين الأسلوبين:

```python
# بدون Type Hint (يعمل تمامًا، لكن أقل وضوحًا)
age = 25
name = "محمد"

# مع Type Hint (نفس الوظيفة، لكن أوضح للقارئ والأدوات)
age: int = 25
name: str = "محمد"
is_active: bool = True
height: float = 1.75
```

## 🎯 ما الفائدة الفعلية؟

Type Hints **لا تُغيّر** سلوك الكود إطلاقًا وقت التشغيل (Python ستتجاهلها عمليًا)، لكنها توفر فوائد ضخمة:

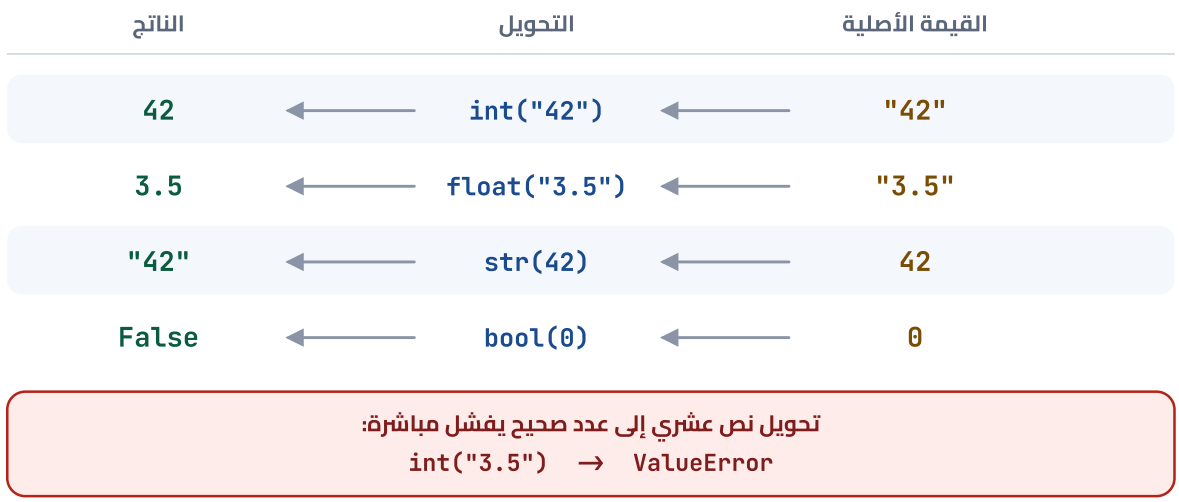
1. **توثيق ذاتي للكود**: أي شخص (حتى أنت بعد 6 أشهر) يفهم نوع البيانات المتوقع فورًا
2. **اكتشاف أخطاء مبكرًا**: محررات مثل VS Code (مع Pylance) تُنبّهك فورًا إن حاولت استخدام نوع خاطئ
3. **إكمال تلقائي أذكى**: المحرر يقترح عليك الدوال والخصائص الصحيحة بناءً على النوع المُعلن

## 🔬 تجربة عملية في VS Code

```python
age: int = 25
age = "خمسة وعشرون"  # لن يُسبب خطأ وقت التشغيل، لكن VS Code سيُظهر خط تحذير أحمر تحته!
```

> 💡 **نصيحة عملية**: من الآن فصاعدًا في هذا الكتاب، سنستخدم Type Hints في كل مثال تقريبًا - ليس لأنها إلزامية، بل لأنها **أفضل ممارسة عالمية حديثة** ستجعل كودك أوضح واحترافيًا من البداية، وستجنّبك إعادة تعلّم عادات لاحقًا.
>
> ⚠️ **توضيح مهم**: Type Hints **اختيارية بالكامل** في Python (على عكس لغات مثل Java حيث هي إلزامية). يمكنك كتابة برنامج كامل بدونها وسيعمل تمامًا. لكن في عالم العمل الاحترافي اليوم (خصوصًا في الشركات الكبرى)، أصبحت شبه معيار إلزامي لأي كود جاد.

## 3.8 تحويل الأنواع (Type Casting)




شكل 3.2 — التحويل الصريح بين الأنواع الأساسية

أحيانًا تحتاج لتحويل قيمة من نوع لآخر صراحة. Python توفر دوال مُدمجة لهذا:

```python
# تحويل من str إلى int
age_text: str = "25"
age_number: int = int(age_text)
print(age_number + 5)  # 30

# تحويل من int إلى str
score: int = 95
score_text: str = str(score)
print("درجتك: " + score_text)  # درجتك: 95

# تحويل من str إلى float
price_text: str = "19.99"
price: float = float(price_text)

# تحويل إلى bool
print(bool(0))      # False
print(bool(1))      # True
print(bool(""))     # False (نص فارغ = False)
print(bool("لا"))   # True  (أي نص غير فارغ = True، حتى لو كان "لا"!)
```

### ⚠️ الفخ الأشهر على الإطلاق للمبتدئين: `input()` تُعيد دائمًا نصًا

تذكر برنامجك في الفصل الثاني؟ هذا الفخ بالضبط وقعت فيه (أو كاد):

```python
age = input("كم عمرك؟ ")  # المستخدم يكتب: 25
print(type(age))            # <class 'str'> ← ليس int!

# محاولة استخدامه كرقم مباشرة:
age_next_year = age + 1  # ❌ TypeError: can only concatenate str (not "int") to str
```

**الحل الصحيح:**

```python
age: str = input("كم عمرك؟ ")
age_number: int = int(age)  # التحويل الصريح إلى int
age_next_year: int = age_number + 1
print(age_next_year)  # يعمل بنجاح الآن
```

🎯 **قاعدة ذهبية يجب حفظها:** دالة `input()` **تُعيد دائمًا نصًا (str)**، بغض النظر عمّا يكتبه المستخدم - حتى لو كتب رقمًا! يجب عليك دائمًا تحويله صراحة ( `int()` أو `float()` ) إن أردت استخدامه كرقم في عمليات حسابية.

### ⚠️ تحذير: محاولة تحويل نص غير رقمي

```python
age = int("خمسة وعشرون")  # ❌ ValueError: invalid literal for int()
```

سنتعلّم في الفصل الحادي عشر (معالجة الأخطاء) كيف «نمسك» هذا النوع من الأخطاء بأناقة قبل أن يُوقف البرنامج بالكامل.

---

## 3.9 الإسناد المتعدد وتبديل القيم (تقنيات Python الأنيقة)

### 🎁 الإسناد المتعدد

Python تسمح بإسناد عدة متغيرات في سطر واحد بأناقة:

```python
# الطريقة التقليدية
x = 5
y = 10
z = 15

# طريقة Python الأنيقة (Multiple Assignment)
x, y, z = 5, 10, 15
```

## 🎁 إسناد نفس القيمة لعدة متغيرات

```python
a = b = c = 0
print(a, b, c)  # 0 0 0
```

## 🔄 تبديل قيم متغيرين (دون متغير مؤقت!)

في أغلب اللغات، تبديل قيمتين يحتاج متغيرًا ثالثًا مؤقتًا:

```python
# الطريقة التقليدية (في لغات أخرى، أو حتى في Python)
temp = x
x = y
y = temp
```

لكن Python توفر طريقة أنيقة جدًا (سحرية بعض الشيء للمبتدئين):

```python
x, y = 5, 10
x, y = y, x  # تبديل القيم في سطر واحد!

print(x, y)  # 10 5
```

> 💡 **لماذا هذا مهم؟** هذا مثال ممتاز على فلسفة Python: «**القابلية للقراءة مهمة**» (Readability counts - من PEP 20، فلسفة Python الرسمية). بدلاً من 3 أسطر مع متغير مؤقت مُربك، سطر واحد واضح يُعبّر عن النية مباشرة.

# 3.10 الثوابت (Constants): عُرف لا قاعدة صارمة

### 🤔 هل توجد ثوابت حقيقية في Python؟

بصراحة تامة: **لا**. على عكس لغات مثل Java أو C++ التي لديها كلمة `const` تمنع تغيير القيمة فعليًا، Python **لا تملك آلية حقيقية** لمنع تغيير أي متغير.

لكن المجتمع البرمجي اتفق على **عُرف (Convention)**: أي متغير بأحرف كبيرة بالكامل ( `UPPER_CASE` ) يُعتبر «ثابتًا» **بالاتفاق**، ولا يجب تغييره، رغم أن Python تقنيًا لن تمنعك:

```python
PI: float = 3.14159
MAX_USERS: int = 100
APP_NAME: str = "نظام إدارة المكتبة"

# تقنيًا يمكنك فعل هذا، لكنه يُعتبر خطأ في الممارسة الاحترافية:
PI = 3.14  # ⚠️ ليس خطأ تقنيًا، لكنه مخالفة صارخة للعُرف المتفق عليه
```

✅ **أفضل الممارسات**: استخدم `UPPER_CASE` لأي قيمة تعرف أنها لن (ويجب ألا) تتغيّر طوال تشغيل البرنامج (مثل Pi، الحد الأقصى لشيء ما، اسم التطبيق). هذا يُرسل إشارة واضحة لأي مبرمج آخر (أو لك مستقبلًا) أن هذه القيمة «مقدّسة» ولا تُعدَّل.

---

# 3.11 جدول مقارنة شامل: أنواع البيانات الأساسية

| النوع | الكلمة المفتاحية | مثال | هل قابل للتعديل (Mutable)؟ |
|---|---|---|---|
| عدد صحيح | `int` | `25` | ❌ لا (Immutable) |
| عدد عشري | `float` | `3.14` | ❌ لا (Immutable) |
| نص | `str` | `"مرحبا"` | ❌ لا (Immutable) |
| منطقي | `bool` | `True` | ❌ لا (Immutable) |
| فارغ | `NoneType` | `None` | ❌ لا (Immutable) |

> **ملاحظة تمهيدية مهمة:** لاحظ أن كل الأنواع التي تعلمناها اليوم هي **Immutable** (غير قابلة للتعديل المباشر). هذا المفهوم سيصبح شديد الأهمية في الفصل الثامن عندما نتعرّف على القوائم (Lists) التي هي **Mutable** (قابلة للتعديل) - الفرق بينهما سيُفسّر سلوكيات غريبة ستواجهها لاحقًا إن لم تفهم هذا الأساس جيدًا من الآن.

---

## 3.12 أخطاء شائعة يرتكبها المبتدئون (تجنّبها الآن)

### ⚠️ الخطأ 1: استخدام متغير قبل تعريفه

```python
print(age)  # ❌ NameError: name 'age' is not defined
age = 25
```

**القاعدة:** يجب تعريف (إسناد قيمة لـ) المتغير **قبل** استخدامه، فـPython تُنفّذ الكود سطرًا بسطر من الأعلى للأسفل.

### ⚠️ الخطأ 2: أخطاء إملائية في أسماء المتغيرات

```python
user_name = "أحمد"
print(usre_name)  # ❌ NameError - خطأ إملائي! (usre بدل user)
```

> **نصيحة عملية:** استخدم ميزة الإكمال التلقائي في VS Code (تظهر تلقائيًا أثناء الكتابة) بدل كتابة الأسماء يدويًا بالكامل في كل مرة - هذا يقضي على أغلب الأخطاء الإملائية.

### ⚠️ الخطأ 3: الخلط بين الأنواع بدون تحويل

```python
age = input("عمرك: ")  # str دائمًا
total = age + 10  # ❌ TypeError
```

### ⚠️ الخطأ 4: استخدام الكلمات المحجوزة كأسماء متغيرات

```python
type = "طالب"  # ⚠️ يعمل تقنيًا، لكنه يُخفي دالة type() المُدمجة!
```

⚠ **تحذير دقيق:** هذا لن يُسبب خطأ فوريًا، لكنه سيُسبب مشاكل خفية لاحقًا إن احتجت استخدام `type()` الحقيقية في نفس الجزء من الكود. تجنّب استخدام أسماء الدوال والأنواع المُدمجة (`type` ، `list` ، `str` ، `input` ...) كأسماء لمتغيراتك.

---

## 3.13 أفضل الممارسات: ملخص شامل

Python

```python
# ✅ ممارسات جيدة يجب اتباعها دائمًا

# 1. أسماء واضحة ومعبّرة بصيغة snake_case
user_age: int = 25

# 2. استخدام Type Hints للوضوح
total_price: float = 99.99

# 3. الثوابت بأحرف كبيرة
MAX_ATTEMPTS: int = 3

# 4. تحويل صريح عند الحاجة
user_input: str = input("أدخل رقمًا: ")
number: int = int(user_input)

# 5. تعليق توضيحي عند الحاجة لشرح "لماذا" وليس "ماذا"
discount_rate: float = 0.15  # خصم 15% للعملاء المميزين
```

---

## 3.14 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

أنشئ ملفًا جديدًا، وعرّف 5 متغيرات تمثّل معلومات عن كتاب تحبّه (العنوان، المؤلف، سنة النشر، عدد الصفحات، هل قرأته بالكامل)، مستخدمًا Type Hints مناسبة لكل متغير، ثم اطبعها جميعًا بصيغة منسّقة باستخدام f-strings.

### 🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يسأل المستخدم عن رقمين (باستخدام `input()`)، يحوّلهما لنوع `int`، ثم يطبع: مجموعهما، الفرق بينهما، حاصل ضربهما. تأكد من معالجة مشكلة أن `input()` تُعيد نصًا دائمًا.

### 🔴 تمرين 3 (تحدي)

بدون استخدام متغير مؤقت ( `temp` )، اكتب كودًا يُبدّل قيم **ثلاثة** متغيرات بحيث: القيمة التي كانت في `a` تنتقل لـ `b` ، والتي كانت في `b` تنتقل لـ `c` ، والتي كانت في `c` تنتقل لـ `a` (تبديل دائري).

<details>
<summary>🔍 تلميح للتمرين 3</summary>

فكّر في الإسناد المتعدد الذي تعلمناه: `x, y = y, x` . هل يمكن توسيع نفس الفكرة لثلاثة متغيرات بترتيب معيّن؟

</details>

---

## 3.15 مشروع تطبيقي: «بطاقة التعريف الذكية - النسخة المُطوَّرة»

بما أننا تعمّقنا أكثر في المتغيرات والأنواع، لنُطوّر مشروع الفصل السابق ليكون أكثر احترافية.

### 📋 المهمة

أنشئ ملفًا باسم `advanced_id_card.py` يقوم بـ:

1. استقبال بيانات المستخدم: الاسم، سنة الميلاد، الطول (بالمتر)، هل هو موظف (نعم/لا)
2. **تحويل الأنواع بشكل صحيح** لكل قيمة (سنة الميلاد ← int، الطول ← float، حالة التوظيف ← bool)
3. استخدام **Type Hints** في كل متغير
4. حساب العمر الحالي تلقائيًا (بافتراض السنة الحالية 2025)
5. طباعة بطاقة تعريف منسّقة، مع التعامل الصحيح مع تحويل bool لنص عربي مفهوم («نعم»/«لا» بدل True/False)

## 💻 مثال على الكود المتوقع (حاول حله بنفسك أولاً قبل النظر!)

✅ حل نموذجي كامل (اضغط بعد المحاولة بنفسك)

Python advanced_id_card.py

```python
# استقبال البيانات
full_name: str = input("ما اسمك الكامل؟ ")
birth_year_text: str = input("ما سنة ميلادك؟ ")
height_text: str = input("ما طولك بالمتر؟ (مثال: 1.75) ")
is_employed_text: str = input("هل أنت موظف؟ (نعم/لا) ")

# تحويل الأنواع
birth_year: int = int(birth_year_text)
height: float = float(height_text)
is_employed: bool = is_employed_text.strip() == "نعم"

# الحسابات
CURRENT_YEAR: int = 2025
age: int = CURRENT_YEAR - birth_year

# تحويل bool لنص عربي مفهوم
employment_status: str = "نعم ✅" if is_employed else "لا ❌"

# طباعة البطاقة
print("=" * 35)
print("🪪 بطاقة التعريف الشخصية")
print("=" * 35)
print(f"👤 الاسم: {full_name}")
print(f"🎂 العمر: {age} سنة")
print(f"📏 الطول: {height} متر")
print(f"💼 موظف: {employment_status}")
print("=" * 35)
```

**ملاحظة على الحل:** استخدمنا `is_employed_text.strip() == "نعم"` لتحويل النص لـ bool بطريقة أذكى من `bool()` المباشرة (التي كانت ستُعيد `True` لأي نص غير فارغ، حتى لو كتب المستخدم «لا»!). هذا مثال عملي على التفكير النقدي حول تحويل الأنواع بدل الاعتماد الأعمى على الدوال الجاهزة.

## 3.16 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين `=` و `==`؟
2. لماذا يُفضَّل وصف Python كـ«متغيرات تُشير لقيم» بدلاً من «صناديق تحتوي قيم»؟
3. ما الذي تُعيده دالة `input()` دائمًا، بغض النظر عمّا يكتبه المستخدم؟
4. اذكر الفرق بين لغة ديناميكية التنميط (Python) ولغة ثابتة التنميط (Java).
5. لماذا نستخدم Type Hints رغم أنها لا تمنع الأخطاء وقت التشغيل فعليًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. `=` تُستخدم لإسناد (تخزين) قيمة في متغير، بينما `==` تُستخدم للمقارنة بين قيمتين للتحقق من تساويهما.
2. لأن المتغير في Python لا «يحتوي» القيمة فعليًا، بل هو اسم (ملصق) يُشير إلى مكان في الذاكرة حيث تُخزَّن القيمة الفعلية - وهذا يُفسّر سلوكيات مهمة لاحقًا خصوصًا مع البيانات القابلة للتعديل.
3. تُعيد دائمًا نوع البيانات `str` (نص)، حتى لو كتب المستخدم أرقامًا.
4. في اللغة الديناميكية (Python)، يُحدَّد نوع المتغير تلقائيًا ويمكن تغييره لاحقًا بحرية. في اللغة الثابتة (Java)، يجب تحديد النوع صراحة عند الإنشاء ولا يمكن تغييره بعدها.
5. لأنها تجعل الكود أوضح للقراءة، تساعد المحررات على اكتشاف الأخطاء المحتملة قبل التشغيل، وتُحسّن الإكمال التلقائي - رغم أن Python نفسها تتجاهلها وقت التشغيل الفعلي.

</details>

---

## 3.17 📌 ملخص الفصل (Cheat Sheet)

| المفهوم | مثال |
|---|---|
| إسناد قيمة | `age = 25` |
| Type Hint | `age: int = 25` |
| التحقق من النوع | `type(age)` |
| تحويل لـ int | `int("25")` |
| تحويل لـ float | `float("3.14")` |
| تحويل لـ str | `str(25)` |

| المفهوم | مثال |
| --- | --- |
| تحويل لـ bool | `bool(value)` |
| إسناد متعدد | `x, y = 5, 10` |
| تبديل قيم | `x, y = y, x` |
| ثابت (بالعُرف) | `MAX_SIZE = 100` |
| عنوان الذاكرة | `id(variable)` |

## ⚠️ تذكّر دائمًا

- `input()` تُعيد **str** دائمًا - حوّلها قبل الاستخدام كرقم
- استخدم **snake_case** لكل متغيراتك
- استخدم **UPPER_CASE** للثوابت فقط
- `=` للتخزين، `==` للمقارنة

## 🎯 ماذا بعد؟

الآن تفهم كيف تُخزّن Python البيانات الأساسية. لكن ماذا لو أردنا التعامل مع نصوص بشكل أعمق - دمجها، تقسيمها، البحث فيها، تنسيقها باحترافية؟ في **الفصل الرابع**، سنغوص بعمق في عالم **النصوص (Strings)**، أحد أكثر أنواع البيانات استخدامًا في أي برنامج حقيقي، مع استكشاف شامل لـ f-strings وكل العمليات النصية الأساسية التي ستحتاجها يوميًا.

فصل 04 / 26 · الجزء الثاني

04

# التعامل مع النصوص (Strings)

⏱ 8 دقيقة قراءة | 🧩 16 قسمًا | 💻 38 مثالًا | ✍ 1,664 كلمة

**في هذا الفصل**

4.1 مقدمة: لماذا النصوص بهذا الحجم من الأهمية؟
4.2 إنشاء النصوص: أكثر من مجرد اقتباسات
4.3 الحروف الخاصة (Escape Characters)
4.4 f-strings: الطريقة الحديثة والوحيدة التي يجب أن تتعلمها
4.5 الفهرسة (Indexing): الوصول لحرف معيّن
4.6 التقطيع (Slicing): استخراج جزء من النص
4.7 النصوص غير قابلة للتعديل (Immutable) - مفهوم حاسم
4.8 أهم Methods النصوص (الأكثر استخدامًا عمليًا)
4.9 دالة `len()`: طول النص
4.10 دمج النصوص: عمليات متعددة
4.11 جدول مرجعي شامل: أهم Methods النصوص
4.12 أخطاء شائعة ونصائح احترافية
4.13 تمارين عملية للفصل
4.14 مشروع تطبيقي: «محلل النصوص الذكي»
4.15 اختبر فهمك (Quiz سريع)
4.16 ملخص الفصل (Cheat Sheet)

> *«كل برنامج تقريبًا - من موقع ويب إلى نظام ذكاء اصطناعي - يتعامل مع النصوص بشكل أو بآخر. إتقان النصوص هو إتقان لغة التواصل بين الإنسان والآلة.»*

## 4.1 📖 مقدمة: لماذا النصوص بهذا الحجم من الأهمية؟

قد تظن أن النصوص (Strings) موضوع بسيط - بعد كل شيء، تعلمناها بالفعل في الفصل السابق كأحد الأنواع الأساسية. لكن الحقيقة أن النصوص هي **العمود الفقري** لمعظم البرمجة العملية: قراءة ملفات، معالجة مدخلات المستخدم، التواصل مع الإنترنت، تحليل البيانات، حتى الذكاء الاصطناعي الحديث (نماذج اللغة) مبني بالكامل على معالجة النصوص.

لهذا السبب، نُفرد فصلاً كاملاً لهذا النوع الواحد من البيانات - لأن العمق هنا سيُوفّر عليك أوقاتًا طويلة في كل مشروع مستقبلي.

# 4.2 إنشاء النصوص: أكثر من مجرد اقتباسات

## ✍️ الطرق الأساسية

```python
single_quoted: str = 'مرحبا'
double_quoted: str = "مرحبا"
```

كلاهما متطابق وظيفيًا. لكن متى تختار كل واحدة؟

## 🎯 متى تستخدم كل نوع اقتباس؟

```python
# استخدم مزدوجة عندما يحتوي النص على اقتباس مفرد
message: str = "اسمه كان 'محمد' في القصة"

# استخدم مفردة عندما يحتوي النص على اقتباس مزدوج
quote: str = '"النجاح يحتاج صبرًا" :قال المعلم'
```

✅ **أفضل الممارسات:** اختر نمطًا واحدًا (المزدوجة الأكثر شيوعًا عالميًا) والتزم به في كل مشروعك، واستخدم النوع الآخر فقط عند الحاجة الفعلية لتجنّب التعقيد. أداة `ruff` التي ثبّتناها في الفصل الثاني ستُنسّق هذا تلقائيًا لك بشكل متّسق.

## النصوص متعددة الأسطر (Triple Quotes)

```python
paragraph: str = """
هذا نص يمتد
على عدة أسطر
دون الحاجة لأي رموز خاصة
"""

print(paragraph)
```

💡 **استخدام شائع جدًا:** النصوص ثلاثية الاقتباس تُستخدم أيضًا لكتابة **التوثيق (Docstrings)** للدوال والكلاسات، وهو ما سنتعلمه بالتفصيل في فصل الدوال (الفصل العاشر).

## 4.3 الحروف الخاصة (Escape Characters)

أحيانًا تحتاج لتضمين رموز خاصة داخل النص يصعب كتابتها مباشرة. Python توفر «رموز هروب» لهذا:

| الرمز | المعنى | مثال |
|---|---|---|
| `\n` | سطر جديد (New Line) | `"سطر1\nسطر2"` |
| `\t` | مسافة تبويب (Tab) | `"اسم:\tمحمد"` |
| `\\` | شرطة مائلة فعلية | `"C:\\Users"` |
| `\"` | علامة اقتباس مزدوجة داخل نص مزدوج | `"قال: \"مرحبا\""` |
| `\'` | علامة اقتباس مفردة داخل نص مفرد | `'لا تقل: \'لا\''` |

```python
message: str = "السطر الأول\nالسطر الثاني"
print(message)
# الناتج:
# السطر الأول
# السطر الثاني
```

### النصوص الخام (Raw Strings)

أحيانًا تريد أن تُعامَل الشرطة المائلة `\` **حرفيًا** دون أي تفسير خاص - خصوصًا عند التعامل مع مسارات الملفات على Windows أو التعبيرات النمطية (Regex) لاحقًا:

```python
# بدون raw string - مشكلة!
path: str = "C:\new_folder"  # \n هنا تُفسَّر كسطر جديد! خطأ غير متوقع
print(path)
# الناتج: C:
# ew_folder  ← خطأ!

# مع raw string - الحل
path: str = r"C:\new_folder"
print(path)
# الناتج: C:\new_folder  ← صحيح تمامًا
```

⚠️ **تحذير عملي مهم:** هذا الخطأ شائع جدًا بين المبتدئين الذين يعملون على Windows ويتعاملون مع مسارات الملفات. دائمًا استخدم `r"..."` عند كتابة مسارات Windows، أو الأفضل - استخدم `pathlib` التي سنتعلمها في فصل لاحق وتتجنب هذه المشكلة بالكامل.

---

# f-strings `4.4` : الطريقة الحديثة والوحيدة التي يجب أن تتعلمها

## 🕰️ خلفية تاريخية سريعة (للفهم لا للحفظ)

عبر تاريخ Python، وُجدت 3 طرق لتنسيق النصوص ودمج المتغيرات فيها:

```python
name = "سارة"
age = 28

# 1 الطريقة: % formatting (قديمة جدًا، نادرًا ما تُستخدم اليوم)
print("اسمي %s وعمري %d" % (name, age))

# 2 الطريقة: .format() (كانت شائعة، أصبحت أقل استخدامًا)
print("اسمي {} وعمري {}".format(name, age))

# 3 الطريقة: f-strings (من - الحديثة Python 3.6+)
print(f"اسمي {name} وعمري {age}")
```

🎯 **قرار تعليمي واضح:** في هذا الكتاب، سنستخدم **f-strings حصريًا تقريبًا**، لأنها الأسلوب الموصى به عالميًا اليوم - أوضح، أسرع تنفيذيًا، وأقل عرضة للأخطاء. ستجد الطريقتين القديمتين في كود آخرين أو مشاريع قديمة، لذا من المهم أن تتعرّف عليهما، لكن لا تستخدمهما في كودك الخاص.

## ✍️ الصيغة الأساسية لـ f-strings

ضع حرف `f` قبل علامة الاقتباس، وضع أي متغير داخل أقواس `{}` :

```python
name: str = "محمد"
age: int = 25

greeting: str = f"مرحبًا {name}, عمرك {age} سنة"
print(greeting)  # مرحبًا محمد, عمرك 25 سنة
```

## 🚀 قوة f-strings: التعبيرات الكاملة داخل الأقواس

الميزة الحقيقية لـ f-strings أنها لا تقبل فقط أسماء متغيرات، بل **أي تعبير Python صالح**:

```python
price: float = 49.99
quantity: int = 3

# حسابات مباشرة
print(f"الإجمالي: {price * quantity}")  # الإجمالي: 149.97

# استدعاء دوال
name: str = "محمد"
print(f"الاسم بحروف كبيرة: {name.upper()}")  # محمد :الاسم بحروف كبيرة (ملاحظة: upper لا تؤثر على العربية)

# شروط مباشرة (Ternary expressions)
age: int = 15
print(f"الحالة: {'قاصر' if age < 18 else 'بالغ'}")  # الحالة: قاصر
```

## 🎨 تنسيق متقدم: التحكم في عرض الأرقام

f-strings توفر صياغة قوية للتحكم في كيفية عرض الأرقام - مهارة ستحتاجها كثيرًا في أي تطبيق حقيقي:

```python
price: float = 1234.5678

# تحديد عدد الخانات العشرية
print(f"{price:.2f}")  # 1234.57 (تقريب لخانتين عشريتين)

# إضافة فاصلة آلاف
print(f"{price:,.2f}")  # 1,234.57

# عرض كنسبة مئوية
ratio: float = 0.856
print(f"{ratio:.1%}")  # 85.6%

# محاذاة النص (مفيد جدًا للجداول في الطرفية)
name: str = "محمد"
print(f"[{name:>10}]")  # [      محمد] (محاذاة يمين، عرض 10)
print(f"[{name:<10}]")  # [محمد      ] (محاذاة يسار، عرض 10)
print(f"[{name:^10}]")  # [   محمد   ] (محاذاة وسط، عرض 10)
```

### 🐛 ميزة تشخيصية رائعة (Python 3.8+): `=` داخل f-string

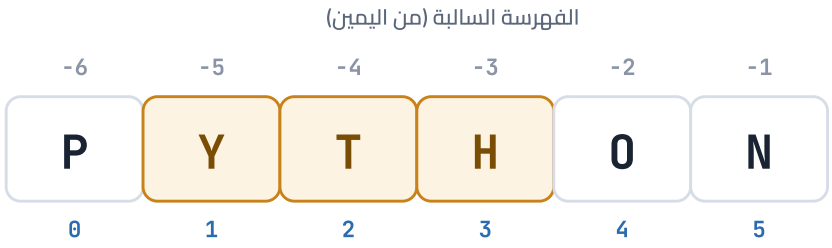
```python
x: int = 10
y: int = 20

print(f"{x=}, {y=}, {x + y=}")
# الناتج: x=10, y=20, x + y=30
```

💡 **نصيحة عملية قوية:** هذه الميزة ممتازة أثناء تصحيح الأخطاء (Debugging) - بدلاً من كتابة `print(f"x is {x}")` يدويًا، فقط اكتب `print(f"{x=}")` وستحصل على اسم المتغير وقيمته معًا تلقائيًا. استخدمها بكثرة أثناء تعلّمك لفهم ما يحدث في كودك.

---

## 4.5 الفهرسة (Indexing): الوصول لحرف معيّن




s[1:4] ← YTH (البداية مشمولة، والنهاية غير مشمولة)

الفهرسة الموجبة تبدأ من 0، والسالبة من -1 عند آخر حرف.

شكل 4.1 — الفهرسة الموجبة والسالبة والتقطيع في النصوص

### 🔢 فهم النظام: يبدأ العد من صفر دائمًا!

في Python (وأغلب لغات البرمجة)، كل نص هو **سلسلة من الحروف**، وكل حرف له «فهرس» (رقم موضعه) يبدأ من **0** وليس من 1.

```python
word: str = "مرحبا"
#           م ر ح ب ا
# الفهرس:   0 1 2 3 4
```

```python
word: str = "مرحبا"

print(word[0])  # (الحرف الأول)  م
print(word[1])  # ر
print(word[4])  # (الحرف الأخير - الفهرس 4، لأن الطول 5)  ا
```

⚠️ **تحذير حرج - خطأ شائع جدًا:** البدء من 0 وليس 1 يُربك المبتدئين باستمرار. تذكّر دائمًا: **الفهرس الأول هو 0**، والفهرس الأخير هو **طول النص - 1**.

## 🔄 الفهرسة السالبة: ميزة ذكية في Python

Python تسمح بالعد من **النهاية** باستخدام أرقام سالبة:

```python
word: str = "مرحبا"

print(word[-1])  # (آخر حرف)  ا
print(word[-2])  # (قبل الأخير)  ب
```

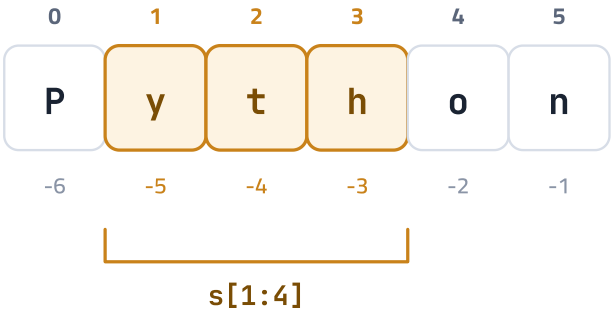
💡 **لماذا هذا مفيد؟** تخيّل أنك تريد آخر حرف في نص طويل لا تعرف طوله بالضبط. بدل حساب الطول يدويًا، فقط استخدم `[-1]` مباشرة - أنيق وبسيط.

## ⚠️ خطأ شائع: تجاوز حدود النص

```python
word: str = "مرحبا"
print(word[10])  # ❌ IndexError: string index out of range
```

**الحل:** دائمًا تأكد أن الفهرس الذي تطلبه موجود فعليًا ضمن طول النص (سنتعلم `len()` بعد قليل).

# 4.6 التقطيع (Slicing): استخراج جزء من النص




شكل 4.2 — التقطيع: البداية مشمولة والنهاية غير مشمولة

التقطيع يسمح لك باستخراج **جزء** من النص، وليس حرفًا واحدًا فقط:

## ✍️ الصيغة الأساسية

```python
text[start:end:step]
```

- `start` : الفهرس الذي يبدأ منه (مُضمَّن)
- `end` : الفهرس الذي ينتهي عنده (**غير** مُضمَّن - نقطة مهمة جدًا!)
- `step` : القفزة بين كل عنصر (اختياري، الافتراضي 1)

## 🎯 أمثلة متدرجة

```python
text: str = "البرمجة ممتعة"

print(text[0:8])    # البرمجة  (من الفهرس 0 حتى 7، الفهرس 8 غير مُضمَّن)
print(text[9:])     # ممتعة  (من الفهرس 9 حتى النهاية)
print(text[:8])     # البرمجة  (من البداية حتى الفهرس 7)
print(text[:])      # البرمجة ممتعة  (نسخة كاملة من النص)
```

⚠️ **تحذير مهم جدًا - أكثر الأخطاء شيوعًا في التقطيع:** الفهرس `end` **غير مُضمَّن** في النتيجة. هذا يُربك المبتدئين باستمرار:

```python
text: str = "Python"
print(text[0:3])  # Pyt  (ليس Pyth!) - لأن الفهرس 3 غير مُضمَّن
```

### 🔄 استخدام step للقفز بين الحروف

```python
text: str = "0123456789"

print(text[::2])    # 02468  (كل حرف ثانٍ)
print(text[1::2])   # 13579  (كل حرف ثانٍ، بدءًا من الفهرس 1)
print(text[::-1])   # 9876543210  (عكس النص بالكامل!)
```

💡 **نصيحة عملية مشهورة:** `text[::-1]` هي الطريقة الأكثر شيوعًا وأناقة في Python لعكس أي نص بالكامل - احفظ هذا النمط جيدًا، ستستخدمه كثيرًا.

---

## 4.7 النصوص غير قابلة للتعديل (Immutable) - مفهوم حاسم

### 🔒 الفهم الجوهري

تذكر من الفصل السابق: النصوص ( `str` ) هي نوع بيانات **Immutable** - أي **لا يمكن تعديلها مباشرة** بعد إنشائها.

```python
word: str = "مرحبا"
word[0] = "M"  # ❌ TypeError: 'str' object does not support item assignment
```

### 🤔 لكن ألسنا «نُعدّل» النصوص باستمرار؟

نعم، لكن ما يحدث فعليًا هو إنشاء نص **جديد بالكامل**، وليس تعديل النص الأصلي في مكانه:

```python
word: str = "مرحبا"
new_word: str = word.upper()  # لا يُعدّل word، بل يُنشئ نصًا جديدًا تمامًا

print(word)      # مرحبا  (لم يتغيّر!)
print(new_word)  # (نسخة معدّلة جديدة)
```

> **لماذا هذا مهم فعليًا؟** هذا يُفسّر لماذا دائمًا تحتاج لإعادة إسناد النتيجة إلى متغير (نفس المتغير أو جديد) عند استخدام methods تُغيّر النص - الـ method لا تُغيّر الأصل أبدًا:

```python
name: str = "محمد"
name.upper()        # ⚠️ النتيجة تُهدَر! لا يفعل شيئًا فعليًا
print(name)         # محمد  (لم يتغيّر!)

name = name.upper()  # ✅ الطريقة الصحيحة - إعادة الإسناد
print(name)          # محمد (بأحرف كبيرة إن كانت إنجليزية)
```

---

## 4.8 أهم Methods النصوص (الأكثر استخدامًا عمليًا)

Python توفر عشرات الـ methods للنصوص، لكن لن نُغرقك بكلها. إليك **الأكثر استخدامًا فعليًا** في المشاريع الحقيقية:

### 🔠 تغيير حالة الأحرف

```python
text: str = "Hello World"

print(text.upper())       # HELLO WORLD
print(text.lower())       # hello world
print(text.title())       # Hello World (كل كلمة تبدأ بحرف كبير)
print(text.capitalize())  # Hello world (أول حرف فقط كبير)
```

## ✂️ إزالة المسافات الزائدة

```python
text: str = "   مرحبا بالعالم   "

print(text.strip())   # "مرحبا بالعالم" (إزالة المسافات من الطرفين)
print(text.lstrip())  # "مرحبا بالعالم   " (من اليسار فقط)
print(text.rstrip())  #    "مرحبا بالعالم" (من اليمين فقط)
```

💡 **استخدام عملي شديد الأهمية:** `.strip()` أساسية جدًا عند معالجة مدخلات المستخدم من `input()`، لأن المستخدمين كثيرًا ما يكتبون مسافات زائدة عن غير قصد:

```python
name: str = input("اسمك: ").strip()
```

## 🔍 البحث داخل النص

```python
text: str = "مبرمجة بلغة Python ممتعة"

print("Python" in text)        # True
print("Java" in text)          # False
print(text.find("Python"))     # 11 (الفهرس حيث بدأت الكلمة)
print(text.find("Java"))       # -1 (غير موجودة)
print(text.count("ة"))         # عدد مرات تكرار الحرف
```

🔄 **مقارنة مهمة:** `in` مقابل `.find()`

| الطريقة | متى تستخدمها |
|---|---|
| `in` | عندما تحتاج فقط معرفة: موجود أم لا؟ (يُعيد bool) |
| `.find()` | عندما تحتاج معرفة **أين** بالضبط (يُعيد رقم الفهرس، أو -1) |

## 🔄 الاستبدال

```python
text: str = "أنا أحب Java"
new_text: str = text.replace("Java", "Python")
print(new_text)  # أنا أحب Python
```

## ✂️ التقسيم والدمج (من أهم الـ methods على الإطلاق)

```python
sentence: str = "التفاح,الموز,البرتقال"

# تقسيم النص إلى قائمة (سنتعلم القوائم بعمق في الفصل الثامن)
fruits: list[str] = sentence.split(",")
print(fruits)  # ['التفاح', 'الموز', 'البرتقال']

# العملية العكسية: دمج قائمة في نص واحد
joined: str = " - ".join(fruits)
print(joined)  # التفاح - الموز - البرتقال
```

💡 **تطبيق عملي حقيقي جدًا:** `.split()` و `.join()` هما من أكثر الـ methods استخدامًا في معالجة البيانات الحقيقية (مثل قراءة ملفات CSV التي سنتعلمها لاحقًا).

## ✅ التحقق من محتوى النص

```python
print("12345".isdigit())      # True - كلها أرقام؟
print("Hello".isalpha())      # True - كلها حروف؟
print("Hello123".isalnum())   # True - حروف أو أرقام فقط؟
print("   ".isspace())        # True - كلها مسافات؟
print("Hello".startswith("He"))  # True
print("Hello".endswith("lo"))    # True
```

🎯 **استخدام عملي شديد الشيوع:** التحقق من صحة مدخلات المستخدم:

```python
age_input: str = input("أدخل عمرك: ")

if age_input.isdigit():
    age: int = int(age_input)
    print(f"عمرك {age} سنة")
else:
    print("خطأ: يجب إدخال رقم فقط!")
```

---

## 4.9 دالة `len()`: طول النص

```python
text: str = "مرحبا"
print(len(text))  # 5
```

> ⚠️ **ملاحظة تقنية:** `len()` ليست method تابعة للنص (لا تُكتب كـ `text.len()`)، بل **دالة مستقلة** تأخذ النص كمُدخل. هذا تصميم مقصود في Python يُفرّق بين الدوال العامة (`len()`, `print()`) والـ methods الخاصة بنوع بيانات معين (`.upper()`, `.strip()`).

### 🎯 استخدام شائع: التحقق من طول كلمة مرور

```python
password: str = input("أدخل كلمة مرور: ")

if len(password) < 8:
    print("⚠️ كلمة المرور قصيرة جدًا! يجب أن تكون 8 أحرف على الأقل")
else:
    print("✅ كلمة المرور مقبولة")
```

---

## 4.10 دمج النصوص: عمليات متعددة

```python
first_name: str = "محمد"
last_name: str = "أحمد"

# الطريقة 1: عامل + (Concatenation)
full_name: str = first_name + " " + last_name
print(full_name)  # محمد أحمد

# الطريقة 2: f-string (الأفضل والأكثر مرونة)
full_name = f"{first_name} {last_name}"

# الطريقة 3: تكرار النص بعامل *
separator: str = "-" * 20
print(separator)  # --------------------
```

✅ **أفضل الممارسات:** استخدم f-strings دائمًا عند دمج أكثر من قيمتين أو عند الحاجة لتنسيق، واستخدم `+` فقط للدمج البسيط جدًا بين نصّين.

---

## 4.11 جدول مرجعي شامل: أهم Methods النصوص

| Method | الوظيفة | مثال |
|---|---|---|
| `.upper()` | تحويل لأحرف كبيرة | `"ABC"` ← `"abc".upper()` |
| `.lower()` | تحويل لأحرف صغيرة | `"abc"` ← `"ABC".lower()` |
| `.strip()` | إزالة المسافات من الطرفين | `"hi"` ← `" hi ".strip()` |
| `.split(sep)` | تقسيم لقائمة | `["a","b"]` ← `"a,b".split(",")` |
| `.join(list)` | دمج قائمة لنص | `"a-b"` ← `"-".join(["a","b"])` |
| `.replace(old,new)` | استبدال جزء | `"Hi"` ← `"hi".replace("h","H")` |
| `.find(sub)` | إيجاد موضع | `2` ← `"hello".find("l")` |
| `.count(sub)` | عدّ التكرارات | `3` ← `"aaa".count("a")` |
| `.startswith(s)` | يبدأ بـ؟ | `True` ← `"hi".startswith("h")` |

| Method | الوظيفة | مثال |
|---|---|---|
| `.endswith(s)` | ينتهي بـ؟ | `True` ← `"hi".endswith("i")` |
| `.isdigit()` | كلها أرقام؟ | `True` ← `"123".isdigit()` |
| `.isalpha()` | كلها حروف؟ | `True` ← `"abc".isalpha()` |
| `len(text)` | طول النص | `5` ← `len("hello")` |

---

## 4.12 أخطاء شائعة ونصائح احترافية

### ⚠️ الخطأ 1: نسيان أن methods لا تُعدّل الأصل

```python
name: str = "محمد"
name.upper()
print(name)  # محمد ← لم يتغيّر! يجب إعادة الإسناد
```

### ⚠️ الخطأ 2: الخلط بين فهرسة النصوص والتقطيع

```python
text: str = "Python"
print(text[0])    # P (حرف واحد)
print(text[0:1])  # P (لكن كنتيجة تقطيع - الفرق دقيق لكن مهم لاحقًا)
print(text[10])   # ❌ IndexError
print(text[10:20])  # "" نص فارغ (التقطيع لا يُسبب خطأ حتى لو تجاوز الحدود!)
```

> 💡 **ملاحظة دقيقة جدًا:** لاحظ الفرق: الفهرسة المباشرة `[10]` خارج الحدود تُسبب خطأ، بينما التقطيع `[10:20]` خارج الحدود يُعيد ببساطة نصًا فارغًا دون أي خطأ! هذا سلوك مقصود في Python يستحق الانتباه.

### ⚠️ الخطأ 3: استخدام `+` لدمج نص مع نوع آخر مباشرة

```python
age: int = 25
message: str = "عمري: " + age  # ❌ TypeError
message: str = "عمري: " + str(age)  # ✅ صحيح
message: str = f"عمري: {age}"       # ✅ الأفضل والأبسط
```

### ✅ نصيحة احترافية: أداء الدمج المتكرر

```python
# ❌ غير فعّال عند دمج نصوص كثيرة في حلقة (سنتعلم الحلقات في الفصل السابع)
result: str = ""
for i in range(1000):
    result += str(i)  # كل عملية += تُنشئ نصًا جديدًا بالكامل في الذاكرة!

# ✅ أكثر كفاءة (سنطبقها عمليًا لاحقًا)
parts: list[str] = [str(i) for i in range(1000)]
result: str = "".join(parts)
```

> 💡 لا تقلق إن لم تفهم مثال الحلقة الآن بالكامل - سنعود لهذا المفهوم بعمق في فصل القوائم والحلقات. فقط احتفظ بهذه النصيحة في ذهنك لاحقًا.

---

## 4.13 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

اطلب من المستخدم إدخال اسمه الكامل، ثم اطبع:
- الاسم بأحرف كبيرة بالكامل
- عدد الحروف في الاسم (بدون المسافات)
- الاسم معكوسًا

### 🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يطلب من المستخدم إدخال بريد إلكتروني، ويتحقق من:
- هل يحتوي على الرمز `@` ؟
- هل يحتوي على نقطة `.` بعد الـ `@` ؟

- إن تحققت الشروط، اطبع «بريد إلكتروني صالح الشكل»، وإلا اطبع رسالة خطأ مناسبة

### 🔴 تمرين 3 (تحدي)

اكتب برنامجًا يتحقق إن كانت جملة مُدخلة من المستخدم هي **Palindrome** (تُقرأ نفسها من الاتجاهين)، **متجاهلًا** المسافات وحالة الأحرف (كبيرة/صغيرة). مثال: "Was it a car or a cat I saw" يجب أن تُعتبر Palindrome صحيحة.

<details>
<summary>🔍 تلميح للتمرين 3</summary>

فكّر في الخطوات: (1) حوّل كل الأحرف لنفس الحالة باستخدام `.lower()`. (2) أزل المسافات باستخدام `.replace(" ", "")`. (3) قارن النص بنسخته المعكوسة `[::-1]`.

</details>

---

## 4.14 مشروع تطبيقي: «محلل النصوص الذكي»

### 📋 المهمة

أنشئ ملفًا باسم `text_analyzer.py` يقوم بتحليل أي نص يُدخله المستخدم، ويعرض تقريرًا كاملاً عنه:

**المتطلبات:**

1. طلب نص من المستخدم (جملة أو فقرة)
2. حساب: عدد الحروف (بدون مسافات)، عدد الكلمات، عدد المسافات
3. إيجاد أطول كلمة في النص
4. تحويل النص لصيغة «عنوان» (كل كلمة تبدأ بحرف كبير - إن كان إنجليزيًا)
5. عرض النص معكوسًا بالكامل
6. طباعة تقرير منسّق بشكل احترافي وجذاب

## 💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python text_analyzer.py

```python
"""برنامج تحليل النصوص - يعرض إحصائيات شاملة عن أي نص مُدخَل."""

# استقبال النص من المستخدم
text: str = input("أدخل نصًا لتحليله: ").strip()

# تقسيم النص لكلمات لحساب الإحصائيات
words: list[str] = text.split()

# الحسابات الأساسية
total_chars: int = len(text.replace(" ", ""))
word_count: int = len(words)
space_count: int = text.count(" ")

# إيجاد أطول كلمة
longest_word: str = max(words, key=len) if words else ""

# تحويلات النص
title_case: str = text.title()
reversed_text: str = text[::-1]

# طباعة التقرير
print("\n" + "=" * 40)
print("📊 تقرير تحليل النص")
print("=" * 40)
print(f"📝 النص الأصلي: {text}")
print(f"🔤 عدد الحروف (بدون مسافات): {total_chars}")
print(f"📦 عدد الكلمات: {word_count}")
print(f"␣ عدد المسافات: {space_count}")
print(f"🏆 أطول كلمة: {longest_word}")
print(f"🔠 بصيغة العنوان: {title_case}")
print(f"🔄 النص معكوسًا: {reversed_text}")
print("=" * 40)
```

**ملاحظة على الحل:** استخدمنا `max(words, key=len)` لإيجاد أطول كلمة - وهذا مثال تشويقي بسيط على دالة `max()` مع معامل `key`، وهو مفهوم سنتعمّق فيه لاحقًا في فصل الدوال المتقدمة. لا تقلق إن بدا غامضًا الآن.

## 4.15 اختبر فهمك (Quiz سريع)

1. لماذا تُعتبر f-strings الطريقة المُفضَّلة لتنسيق النصوص في Python الحديثة؟
2. ما الفرق بين `text[3]` و `text[3:4]`؟
3. لماذا لا يُسبب `text[100:200]` خطأ حتى لو تجاوز طول النص، بينما `text[100]` يُسبب خطأ؟
4. ما الفرق بين `in` و `.find()` عند البحث داخل نص؟
5. لماذا يجب إعادة إسناد نتيجة method مثل `.upper()` لمتغير بدلاً من استدعائها فقط؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. لأنها أوضح في القراءة، تدعم تعبيرات Python كاملة داخل الأقواس، أسرع تنفيذيًا من الطرق القديمة، وتوفر تحكمًا قويًا بالتنسيق (كالخانات العشرية والمحاذاة).
2. `text[3]` يُعيد حرفًا واحدًا (نوعه str بطول 1)، بينما `text[3:4]` يُعيد نصًا ناتجًا عن تقطيع (أيضًا بطول 1 في هذه الحالة تحديدًا، لكن المفهوم مختلف تقنيًا - التقطيع يُعيد دائمًا str حتى لو كان حرفًا واحدًا أو فارغًا).
3. لأن التقطيع (Slicing) مصمَّم في Python ليكون «متسامحًا» - إن تجاوزت الحدود، يُعيد ببساطة أقصى ما هو متاح أو نصًا فارغًا، بعكس الفهرسة المباشرة التي تتطلب وجود الفهرس فعليًا.
4. `in` تُعيد فقط `True` / `False` (هل موجود أم لا)، بينما `.find()` تُعيد رقم الفهرس الدقيق لأول ظهور، أو `-1` إن لم تُوجد.
5. لأن النصوص في Python غير قابلة للتعديل (Immutable)، فكل method تُعدّل النص فعليًا تُنشئ نسخة جديدة معدّلة بدلاً من تغيير الأصل، لذا يجب حفظ هذه النسخة الجديدة في متغير لاستخدامها.

</details>

## 4.16 📌 ملخص الفصل (Cheat Sheet)

```python
# الإنشاء
text: str = "مرحبا"
multiline: str = """نص متعدد الأسطر"""
raw: str = r"C:\path"

# f-strings
f"{variable}"
f"{value:.2f}"       # خانتان عشريتان
f"{value:,}"         # فاصلة آلاف
f"{variable=}"       # تشخيص سريع

# الفهرسة والتقطيع
text[0]          # الحرف الأول
text[-1]         # الحرف الأخير
text[2:5]        # من الفهرس 2 حتى 4
text[::-1]       # عكس النص

# أهم Methods
text.upper() / text.lower()
text.strip()
text.split(",") / "-".join(list)
text.replace(old, new)
text.find(sub)
"sub" in text
len(text)
```

### 🎯 ماذا بعد؟

تعلمنا الآن كيف نتعامل مع النصوص باحتراف. لكن ماذا عن الأرقام؟ في **الفصل الخامس**، سنتعمّق في **العمليات الحسابية**، نكتشف فخ الدقة العشرية الشهير عمليًا، نتعلّم التقريب الصحيح، ونبني حاسبة علمية بسيطة من الطرفية تطبّق كل هذه المفاهيم.

فصل 05 / 26 · الجزء الثاني

05

# الأرقام والعمليات الحسابية

8 دقيقة قراءة | 18 قسمًا | 37 مثالًا | 1,632 كلمة

### في هذا الفصل

5.1 مقدمة: أكثر من مجرد جمع وطرح
5.2 العمليات الحسابية الأساسية
5.3 العمليات الحسابية المتقدمة
5.4 أولوية العمليات (Operator Precedence)
5.5 فخ الدقة العشرية: المشكلة الأشهر في كل لغات البرمجة
5.6 التقريب بعمق: `round()` وتفاصيلها الدقيقة
5.7 دوال رياضية مُدمجة مفيدة
5.8 تحويلات الأنواع الرقمية بعمق
5.9 العمليات المختصرة (Augmented Assignment Operators)
5.10 مقارنة الأرقام: عوامل المقارنة
5.11 الأعداد الكبيرة جدًا: ميزة فريدة في Python
5.12 الأعداد العشوائية: مقدمة سريعة لوحدة `random`
5.13 أخطاء شائعة ونصائح احترافية
5.14 جدول مرجعي شامل للفصل
5.15 تمارين عملية للفصل
5.16 مشروع تطبيقي: «الحاسبة العلمية التفاعلية»
5.17 اختبر فهمك (Quiz سريع)
5.18 ملخص الفصل (Cheat Sheet)

> *«الحاسوب آلة حسابية في جوهرها. فهم كيف تتعامل Python مع الأرقام هو فهم لب عمل الكمبيوتر نفسه.»*

## 5.1 📖 مقدمة: أكثر من مجرد جمع وطرح

قد تظن أن التعامل مع الأرقام هو أبسط جزء في البرمجة - بعد كل شيء، تعلمته في المدرسة الابتدائية. لكن في هذا الفصل، ستكتشف فخاخًا حقيقية تُربك حتى المبرمجين ذوي الخبرة إن لم يفهموها جيدًا، وستتعلم تقنيات احترافية للتعامل مع الأرقام بدقة ونظافة.

## 5.2 العمليات الحسابية الأساسية

```python
a: int = 10
b: int = 3

print(a + b)   # 13   ← الجمع
print(a - b)   # 7    ← الطرح
print(a * b)   # 30   ← الضرب
print(a / b)   # 3.3333333333333335  ← القسمة (تُعيد دائمًا float!)
```

### ⚠️ تحذير مهم: القسمة `/` تُعيد دائمًا `float`

```python
result = 10 / 2
print(result)        # 5.0  ← float! ليس int كـ 5
print(type(result))  # <class 'float'>
```

> 💡 **لماذا هذا مهم؟** في لغات كثيرة أخرى، قسمة عددين صحيحين تُعيد عددًا صحيحًا. لكن Python 3 غيّرت هذا السلوك عمدًا (مقارنة بـ Python 2 القديمة) لتجنّب فقدان الدقة العشرية بشكل غير متوقع.

---

## 5.3 العمليات الحسابية المتقدمة

### ➗ القسمة الصحيحة (Floor Division) `//`

```python
print(10 // 3)   # 3   ← ناتج القسمة بدون الباقي (مُقرَّب للأسفل)
print(10 / 3)    # 3.3333...  ← القسمة العادية للمقارنة
```

### 🔢 باقي القسمة (Modulo) `%`

```python
print(10 % 3)    # 1   ← الباقي بعد القسمة
print(20 % 5)    # 0   ← قابل للقسمة تمامًا
```

🎯 **استخدام عملي شديد الشيوع:** `%` هي الأداة الأساسية للتحقق من كون رقم زوجيًا أو فرديًا:

```python
number: int = 7
if number % 2 == 0:
    print("زوجي")
else:
    print("فردي")
```

## 🔺 الأُس (Exponentiation) `**`

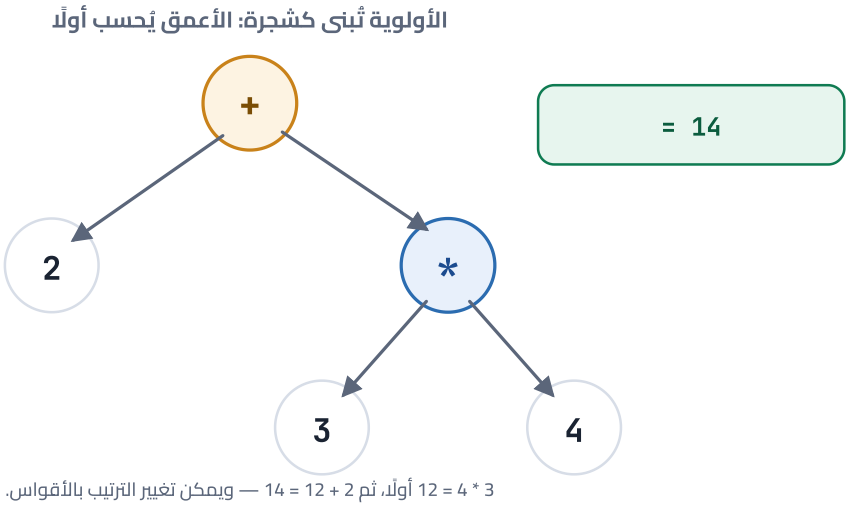
```python
print(2 ** 3)    # 8    ← 2 أُس 3
print(10 ** 2)   # 100  ← تربيع 10
print(4 ** 0.5)  # 2.0  ← الجذر التربيعي (أُس 0.5)
```

⚠️ **خطأ شائع عند القادمين من لغات أخرى:** في بعض اللغات (مثل JavaScript القديمة)، الأُس يُكتب كدالة `Math.pow()`. في Python، استخدم `**` مباشرة - لا تخلط مع `^` التي تعني شيئًا مختلفًا تمامًا في Python (XOR على مستوى البت - موضوع متقدم لن نحتاجه الآن).

## 📊 جدول شامل لكل العمليات الحسابية

| العامل | الاسم | مثال | النتيجة |
|---|---|---|---|
| `+` | جمع | `7 + 3` | `10` |
| `-` | طرح | `7 - 3` | `4` |
| `*` | ضرب | `7 * 3` | `21` |
| `/` | قسمة عادية | `7 / 3` | `2.333...` |
| `//` | قسمة صحيحة | `7 // 3` | `2` |
| `%` | باقي القسمة | `7 % 3` | `1` |
| `**` | أُس | `7 ** 3` | `343` |

# 5.4 أولوية العمليات (Operator Precedence)




شكل 5.1 — أولوية العمليات كشجرة

## القاعدة المعروفة من الرياضيات (PEMDAS)

Python تتبع نفس القواعد الرياضية التقليدية التي تعلمتها في المدرسة:

```
1. الأقواس ()
2. الأُس **
3. الضرب * والقسمة / // (من اليسار لليمين)
4. الجمع + والطرح - (من اليسار لليمين)
```

```python
result = 2 + 3 * 4
print(result)  # 14  ← (14=12+2) ثم الجمع، (12=4*3) الضرب أولاً

result = (2 + 3) * 4
print(result)  # 20  ← (20=4*5) ثم الضرب، (5=3+2) الأقواس أولاً
```

✅ **نصيحة احترافية:** حتى عندما تكون متأكدًا من أولوية العمليات، استخدم الأقواس `()` لجعل الكود **أوضح** لأي قارئ آخر (أو لك مستقبلاً). الوضوح دائمًا أهم من توفير حرفين من الكتابة:

```python
# يعمل لكن يحتاج تفكيرًا لفهمه
total = price * quantity - discount * quantity

# أوضح بكثير بنفس النتيجة
total = (price * quantity) - (discount * quantity)
```

---

## 5.5 فخ الدقة العشرية: المشكلة الأشهر في كل لغات البرمجة




شكل 5.2 — لماذا 0.1 + 0.2 لا تساوي 0.3

### 🔬 التجربة الصادمة

```python
print(0.1 + 0.2)
# الناتج: 0.30000000000000004
# ليس 0.3 كما تتوقع!
```

هذا ليس خطأ في Python - **كل** لغات البرمجة تقريبًا (C++, Java, JavaScript) تعاني من نفس المشكلة بالضبط.

### 🤔 لماذا يحدث هذا؟

الكمبيوتر يُخزّن كل شيء في النظام **الثنائي** (0 و 1 فقط). بعض الكسور العشرية البسيطة في نظامنا العشري (مثل 0.1) لا يمكن تمثيلها **بدقة تامة** في النظام الثنائي - تمامًا كما أن الكسر 1/3 لا يمكن كتابته بدقة تامة كعدد عشري (0.3333... إلى ما لا نهاية).

> **تشبيه مبسّط:** تخيل أنك تحاول قياس طول بمسطرة مُقسّمة لأثلاث فقط (لا أرباع ولا أخماس). بعض الأطوال ستقيسها بدقة تامة، لكن أطوالاً أخرى ستحتاج تقريبًا. هذا بالضبط ما يحدث مع الكمبيوتر والنظام الثنائي مقابل بعض الكسور العشرية.

## ⚠️ تبعات عملية خطيرة لهذه المشكلة

```python
price: float = 19.99
quantity: int = 3
total: float = price * quantity

print(total)  # 59.96999999999999  ← ليس 59.97 بالضبط!
```

تخيل لو كان هذا نظام حسابات بنكية حقيقي - هذا الفرق الصغير جدًا قد يُسبب مشاكل محاسبية خطيرة متراكمة.

## ✅ الحل الأول: التقريب عند العرض باستخدام `round()`

```python
total: float = 0.1 + 0.2
print(round(total, 2))  # 0.3  ← تقريب لخانتين عشريتين
```

```python
price: float = 19.99
quantity: int = 3
total: float = round(price * quantity, 2)
print(total)  # 59.97
```

> ⚠️ **تحذير مهم:** `round()` يحل مشكلة **العرض**، لكن لا يحل مشكلة **الدقة الداخلية** أثناء الحسابات المتتالية. في عمليات حسابية طويلة ومتكررة (كأنظمة مالية حقيقية)، لا تعتمد فقط على `round()`.

## ✅ الحل الثاني (الاحترافي): استخدام وحدة `decimal` للحسابات المالية الدقيقة

عندما تتعامل مع أموال حقيقية أو حسابات تتطلب دقة تامة، استخدم وحدة `decimal` المُدمجة في Python:

```python
from decimal import Decimal

price = Decimal("19.99")  # مباشرة لاحظ: كنص! وليس float
quantity = Decimal("3")
total = price * quantity

print(total)  # 59.97 ← دقيق تمامًا!
```

🎯 **قاعدة ذهبية للمشاريع الحقيقية**: أي نظام يتعامل مع أموال فعلية (تطبيق متجر، نظام محاسبة، بنك) **يجب** أن يستخدم `Decimal` بدلاً من `float` العادي. هذا ليس اختياريًا - هو معيار صناعي معتمد عالميًا لتجنّب كوارث حسابية حقيقية حدثت في أنظمة مالية بسبب هذه المشكلة بالضبط.

### 🔄 مقارنة سريعة: متى تستخدم float ومتى decimal؟

| الحالة | الاستخدام الموصى به |
|---|---|
| حسابات علمية، إحداثيات، قياسات تقريبية | ✅ `float` |
| حسابات مالية، أسعار، فواتير، رواتب | ✅ `Decimal` |
| عرض سريع لرقم تقريبي للمستخدم | `round()` كافية |

---

## 5.6 التقريب بعمق: `round()` وتفاصيلها الدقيقة

### ✍️ الاستخدام الأساسي

```python
print(round(3.14159, 2))   # 3.14
print(round(3.14159, 0))   # 3.0
print(round(3.14159))      # 3  (بدون معامل ثانٍ، يُعيد int)
```

### ⚠️ سلوك غريب يجب معرفته: Banker's Rounding

```python
print(round(2.5))   # 2  ← ليس 3 كما قد تتوقع!
print(round(3.5))   # 4
print(round(0.5))   # 0  ← ليس 1!
```

هذا ليس خطأ - Python تستخدم أسلوبًا يُسمى **«التقريب المصرفي» (Banker's Rounding)**: عندما يكون الرقم بالضبط في المنتصف (مثل 0.5)، يُقرَّب لأقرب رقم **زوجي**. هذا الأسلوب مُستخدم إحصائيًا لتقليل التحيّز التراكمي في التقريب عبر آلاف العمليات.

> ⚠️ **تحذير عملي**: إذا كنت تتوقع التقريب «التقليدي» (0.5 دائمًا للأعلى)، ستُفاجأ بهذا السلوك. لحسابات حساسة، استخدم وحدة `decimal` مع إعدادات تقريب صريحة بدلاً من الاعتماد على `round()` الافتراضية.

---

## 5.7 دوال رياضية مُدمجة مفيدة

### 🔢 دوال أساسية بدون استيراد

```python
print(abs(-5))        # 5    ← القيمة المطلقة
print(max(3, 7, 2))   # 7    ← أكبر قيمة
print(min(3, 7, 2))   # 2    ← أصغر قيمة
print(pow(2, 3))      # 8    ← نفس 2**3
print(sum([1, 2, 3])) # 6    ← مجموع عناصر (سنتعمق في القوائم لاحقًا)
```

### 📐 وحدة `math`: دوال رياضية متقدمة

لدوال أكثر تخصصًا، نحتاج استيراد وحدة `math` المُدمجة:

```python
import math

print(math.sqrt(16))      # 4.0   ← الجذر التربيعي
print(math.ceil(4.1))     # 5     ← التقريب للأعلى دائمًا
print(math.floor(4.9))    # 4     ← التقريب للأسفل دائمًا
print(math.pi)            # 3.141592653589793
print(math.factorial(5))  # 120   ← المضروب (!5)
```

> **أول تعرّف على `import`** : هذا أول استخدام لنا لمفهوم **الاستيراد (import)** في الكتاب - طريقة Python لاستخدام كود إضافي مُنظَّم في «وحدات» (modules) منفصلة. سنتعمّق في هذا المفهوم بالتفصيل لاحقًا، لكن تذكّر الآن: أي وظيفة متقدمة غير متوفرة مباشرة، غالبًا موجودة في وحدة تحتاج استيرادها أولاً.

### 🔄 مقارنة: `round()` مقابل `math.floor()` مقابل `math.ceil()`

```python
number: float = 4.5

print(round(number))        # 4  (تقريب مصرفي للأقرب زوجي)
print(math.floor(number))   # 4  (دائمًا للأسفل)
print(math.ceil(number))    # 5  (دائمًا للأعلى)
```

---

## 5.8 تحويلات الأنواع الرقمية بعمق

### 🔄 int ↔ float

```python
# من int إلى float
x: int = 5
y: float = float(x)
print(y)  # 5.0

# من float إلى int (تحذير: يقطع الجزء العشري، لا يُقرِّب!)
a: float = 9.99
b: int = int(a)
print(b)  # 9  ← القطع وليس التقريب! ليس 10
```

> ⚠️ **تحذير حرج جدًا:** تحويل `float` إلى `int` باستخدام `int()` **يقطع** الجزء العشري ببساطة (Truncation)، ولا يُقرِّب أبدًا:

```python
print(int(9.99))   # 9  (ليس 10)
print(int(-9.99))  # -9 (ليس -10)
```

إذا أردت التقريب الفعلي قبل التحويل، استخدم `round()` أولاً:

```python
print(int(round(9.99)))  # 10  ← صحيح الآن
```

---

## 5.9 العمليات المختصرة (Augmented Assignment Operators)

### ✍️ الصيغة المختصرة لتحديث متغير

بدلاً من كتابة:

```python
counter: int = 10
counter = counter + 5
```

Python توفر صيغة مختصرة وأنيقة:

```python
counter: int = 10
counter += 5  # نفس تمامًا counter = counter + 5
print(counter)  # 15
```

### 📊 جدول العمليات المختصرة الكاملة

| الصيغة المختصرة | تُكافئ |
|---|---|
| `x += 5` | `x = x + 5` |
| `x -= 5` | `x = x - 5` |
| `x *= 5` | `x = x * 5` |
| `x /= 5` | `x = x / 5` |
| `x //= 5` | `x = x // 5` |
| `x %= 5` | `x = x % 5` |
| `x **= 5` | `x = x ** 5` |

```python
score: int = 0
score += 10  # score أصبحت 10
score += 5   # score أصبحت 15
score *= 2   # score أصبحت 30
print(score)  # 30
```

✅ **أفضل الممارسات**: استخدم الصيغ المختصرة دائمًا عند تحديث قيمة متغير بناءً على قيمته الحالية - أقصر، أوضح نية، وهي الأسلوب القياسي المتوقع من أي مبرمج Python محترف.

---

## 5.10 مقارنة الأرقام: عوامل المقارنة

سنتعمّق في استخدامها الكامل في الفصل السادس (التحكم بسير البرنامج)، لكن من الضروري التعرّف عليها الآن:

```python
print(5 > 3)    # True
print(5 < 3)    # False
print(5 == 5)   # True
print(5 ≠ 3)    # True
print(5 ⩾ 5)    # True
print(5 ⩽ 4)    # False
```

### ⚠️ تحذير: مقارنة الأعداد العشرية مباشرة

```python
result: float = 0.1 + 0.2
print(result == 0.3)  # False!  ← بسبب فخ الدقة العشرية الذي شرحناه
```

**الحل الصحيح لمقارنة أعداد عشرية:**

```python
result: float = 0.1 + 0.2
is_close_enough: bool = abs(result - 0.3) < 0.0001
print(is_close_enough)  # True
```

> **نصيحة متقدمة:** في المشاريع الاحترافية، يُستخدم غالبًا `math.isclose()` الجاهزة بدل كتابة هذا المنطق يدويًا:

```python
import math
print(math.isclose(0.1 + 0.2, 0.3))  # True
```

---

## 5.11 الأعداد الكبيرة جدًا: ميزة فريدة في Python

### 🔢 لا حدود عملية لحجم الأعداد الصحيحة

على عكس لغات مثل C++ أو Java حيث الأعداد الصحيحة لها حد أقصى ثابت (overflow)، Python تتعامل مع الأعداد الكبيرة جدًا تلقائيًا دون أي إعداد إضافي:

```python
huge: int = 2 ** 1000
print(huge)  # يعمل بشكل طبيعي تمامًا - رقم ضخم جدًا بمئات الخانات!
```

### ✍️ الفواصل في الأرقام الكبيرة (Underscore Separators)

لتحسين قراءة الأرقام الكبيرة في كودك، Python تسمح باستخدام شرطة سفلية `_` كفاصل بصري (يُتجاهَل فعليًا من Python):

```python
population: int = 1_000_000_000  # مليار - أوضح بكثير من 1000000000
print(population)  # 1000000000  (الشرطات تُحذَف تلقائيًا)
```

> **نصيحة عملية:** استخدم هذه الميزة دائمًا مع الأرقام الكبيرة (أكثر من 4 خانات) لتحسين قابلية القراءة - فرق كبير بين `1000000` و `1_000_000` من حيث سهولة الفهم الفورية.

---

## 5.12 الأعداد العشوائية: مقدمة سريعة لوحدة `random`

لن نتعمق هنا، لكن من المفيد التعرّف على هذه الأداة المُستخدَمة بكثرة لاحقًا في المشاريع (الألعاب، المحاكاة):

```python
import random

random_number: int = random.randint(1, 10)  # رقم عشوائي بين 1 و10 (شاملاً الطرفين)
print(random_number)

random_float: float = random.random()  # رقم عشوائي بين 0 و1
print(random_float)
```

🎯 سنستخدم هذه الوحدة بكثرة في مشاريع لاحقة، خصوصًا في مشروع «لعبة تخمين الأرقام» في الفصل السابع.

---

## 5.13 أخطاء شائعة ونصائح احترافية

### ⚠️ الخطأ 1: القسمة على صفر

```python
result = 10 / 0  # ❌ ZeroDivisionError: division by zero
```

سنتعلّم كيف «نمسك» هذا الخطأ بأناقة في فصل معالجة الأخطاء (الفصل الحادي عشر). الآن، فقط تذكّر أن القسمة على صفر غير معرَّفة رياضيًا، وPython ترفضها بخطأ واضح بدل نتيجة غير منطقية.

### ⚠️ الخطأ 2: نسيان أن `/` تُعيد float دائمًا

```python
total_items: int = 10
groups: int = 3
items_per_group = total_items / groups  # 3.333... وليس 3

# إذا أردت عددًا صحيحًا من المجموعات:
items_per_group = total_items // groups  # 3  ← استخدم القسمة الصحيحة
```

### ⚠️ الخطأ 3: الثقة العمياء بنتائج float في المقارنات

```python
# كود خاطئ محتمل
if (0.1 + 0.2) == 0.3:
    print("متساويان")
else:
    print("غير متساويين")  # هذا ما سيُطبع فعليًا!
```

### ✅ نصيحة احترافية: استخدام Type Hints مع الأرقام

```python
def calculate_total(price: float, quantity: int) -> float:
    return round(price * quantity, 2)
```

> 💡 لا تقلق إن لم تفهم صيغة الدالة بالكامل الآن - سنتعمق فيها في الفصل العاشر. لاحظ فقط كيف توضح Type Hints بدقة متوقع كل نوع بيانات.

---

## 5.14 جدول مرجعي شامل للفصل

| العملية/الدالة | المثال | النتيجة |
|---|---|---|
| جمع/طرح/ضرب | `5 + 3`, `5 - 3`, `5 * 3` | `8`, `2`, `15` |
| قسمة عادية | `10 / 3` | `3.333...` |
| قسمة صحيحة | `10 // 3` | `3` |
| باقي القسمة | `10 % 3` | `1` |
| أُس | `2 ** 3` | `8` |
| تقريب | `round(3.14159, 2)` | `3.14` |
| قيمة مطلقة | `abs(-5)` | `5` |
| أكبر/أصغر قيمة | `max(1,5,3)`, `min(1,5,3)` | `5`, `1` |
| جذر تربيعي | `math.sqrt(16)` | `4.0` |

| العملية/الدالة | المثال | النتيجة |
|---|---|---|
| تقريب لأعلى/أسفل | `math.ceil(4.1)` , `math.floor(4.9)` | `5` , `4` |
| رقم عشوائي | `random.randint(1,10)` | رقم بين 1-10 |
| حسابات مالية دقيقة | `Decimal("19.99")` | دقة تامة |

---

## 5.15 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطلب من المستخدم رقمين، ثم يطبع نتائج كل العمليات الحسابية الأساسية بينهما (جمع، طرح، ضرب، قسمة، قسمة صحيحة، باقي القسمة، أُس) بتنسيق واضح ومرتّب.

### 🟡 تمرين 2 (مستوى متوسط)

اكتب برنامج «حاسبة فاتورة مطعم»: يطلب سعر الوجبة، عدد الأشخاص، ونسبة الإكرامية (Tip) كنسبة مئوية، ثم يحسب: نصيب كل شخص من السعر الأساسي، قيمة الإكرامية الإجمالية، والمبلغ الإجمالي شاملاً الإكرامية. استخدم `round()` بشكل صحيح لعرض نتائج مالية منطقية.

### 🔴 تمرين 3 (تحدي)

اكتب برنامجًا يحوّل درجة حرارة من فهرنهايت إلى مئوية والعكس (حسب اختيار المستخدم)، باستخدام الصيغ الرياضية الصحيحة:

- `celsius = (fahrenheit - 32) * 5/9`
- `fahrenheit = celsius * 9/5 + 32`

تأكد من تقريب النتيجة النهائية لخانة عشرية واحدة فقط.

---

## 5.16 مشروع تطبيقي: «الحاسبة العلمية التفاعلية»

### 📋 المهمة

أنشئ ملفًا باسم `scientific_calculator.py` يبني حاسبة متكاملة من الطرفية:

**المتطلبات:**

1. طلب رقمين من المستخدم
2. طلب العملية المطلوبة (كنص: "+"، "-"، "*"، "/"، "//"، "%"، "**")
3. تنفيذ العملية الصحيحة بناءً على اختيار المستخدم
4. معالجة حالة القسمة على صفر بشكل أنيق (رسالة واضحة بدل انهيار البرنامج - استخدام مبدئي بسيط، سنتعمّق في `try/except` لاحقًا)
5. عرض النتيجة بتنسيق احترافي، مع تقريب مناسب إن كانت النتيجة عشرية

## 💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python scientific_calculator.py

```python
"""حاسبة علمية تفاعلية بسيطة من سطر الأوامر."""

print("=" * 40)
print("🧮 الحاسبة العلمية التفاعلية")
print("=" * 40)

# استقبال المدخلات
num1: float = float(input("أدخل الرقم الأول: "))
operator: str = input("أدخل العملية (+ - * /// % **): ").strip()
num2: float = float(input("أدخل الرقم الثاني: "))

# معالجة حالة خاصة: القسمة على صفر
if operator in ("/", "//", "%") and num2 == 0:
    print("\n❌ خطأ: لا يمكن القسمة على صفر!")
else:
    result: float = 0.0
    valid_operation: bool = True

    if operator == "+":
        result = num1 + num2
    elif operator == "-":
        result = num1 - num2
    elif operator == "*":
        result = num1 * num2
    elif operator == "/":
        result = num1 / num2
    elif operator == "//":
        result = num1 // num2
    elif operator == "%":
        result = num1 % num2
    elif operator == "**":
        result = num1 ** num2
    else:
        valid_operation = False
        print(f"\n❌ خطأ: العملية '{operator}' غير معروفة!")

    if valid_operation:
        print("\n" + "=" * 40)
        print(f"📊 النتيجة: {num1} {operator} {num2} = {round(result, 4)}")
        print("=" * 40)
```

**ملاحظة على الحل:** استخدمنا هنا `if/elif` بشكل مبدئي (سنتعمّق فيها بالكامل في الفصل القادم مباشرة)، وهذا مثال تطبيقي ممتاز يُظهر كيف أن المفاهيم في البرمجة مترابطة ومتراكمة - الأرقام التي تعلمناها هنا ستُستخدم فعليًا بمجرد أن نتعلم أدوات اتخاذ القرار في الفصل التالي.

---

## 5.17 اختبر فهمك (Quiz سريع)

1. لماذا تُعيد `/` دائمًا نوع `float` حتى لو كانت القسمة «نظيفة» بدون باقي؟
2. ما الفرق بين `//` و `%`؟ وما الذي يُعيده كل منهما؟
3. لماذا تظهر `0.1 + 0.2` كـ `0.30000000000000004` بدلاً من `0.3`؟
4. متى يجب استخدام `Decimal` بدلاً من `float` العادي؟
5. ما الفرق بين `int(9.9)` و `round(9.9)`؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. هذا قرار تصميمي مقصود في Python 3 لتجنّب فقدان الدقة العشرية بشكل غير متوقع، على عكس بعض اللغات الأخرى التي تُعيد عددًا صحيحًا عند قسمة عددين صحيحين.
2. `//` تُعيد ناتج القسمة الصحيح (مُقرَّب للأسفل، بدون كسور)، بينما `%` تُعيد الباقي المتبقي بعد تلك القسمة.
3. بسبب طريقة تخزين الكمبيوتر للأعداد العشرية في النظام الثنائي، حيث بعض الكسور العشرية البسيطة في نظامنا العشري لا يمكن تمثيلها بدقة تامة في الثنائي.
4. عند التعامل مع حسابات مالية حقيقية (أسعار، فواتير، رواتب) تتطلب دقة تامة بدون أي أخطاء تقريب تراكمية.
5. `int(9.9)` يقطع الجزء العشري فورًا دون تقريب فينتج `9`، بينما `round(9.9)` يُقرِّب فعليًا للأقرب فينتج `10`.

</details>

---

## 5.18 📌 ملخص الفصل (Cheat Sheet)

```python
# العمليات الأساسية
a + b, a - b, a * b, a / b
a // b   # قسمة صحيحة
a % b    # باقي القسمة
a ** b   # أُس

# التقريب والدقة
round(3.14159, 2)     # 3.14
from decimal import Decimal
Decimal("19.99")      # دقة مالية تامة

# دوال مُدمجة
abs(-5), max(1,5,3), min(1,5,3)

# وحدة math
import math
math.sqrt(16), math.ceil(4.1), math.floor(4.9), math.pi

# عمليات مختصرة
x += 5   # x = x + 5
x *= 2   # x = x * 2

# فواصل الأرقام الكبيرة
population = 1_000_000

# مقارنة آمنة للأعداد العشرية
import math
math.isclose(0.1 + 0.2, 0.3)  # True
```

### ⚠️ تذكّر دائمًا

- `/` تُعيد **float** دائمًا، `//` تُعيد القسمة الصحيحة
- لا تقارن أعداد float بـ `==` مباشرة أبدًا
- استخدم `Decimal` للحسابات المالية الحقيقية
- `int()` يقطع الكسور، `round()` يُقرّبها فعليًا

### 🎯 ماذا بعد؟

تعلمنا حتى الآن كيف نُخزّن البيانات (متغيرات)، ونتعامل مع النصوص والأرقام. لكن كل برامجنا حتى الآن «تسير في خط مستقيم» - تُنفَّذ من الأعلى للأسفل دون أي قرارات. في **الفصل السادس**، سندخل عالمًا جديدًا تمامًا: **اتخاذ القرار (Control Flow)** - كيف نجعل البرنامج «يفكر» ويتخذ مسارات مختلفة بناءً على الشروط، شاملاً `if/elif/else`

التقليدية، وأسلوب `match-case` الحديث الثوري في Python.

فصل 06 / 26 · الجزء الثاني

06

# اتخاذ القرار (Control Flow)

9 دقيقة قراءة | 17 قسمًا | 35 مثالًا | 1,760 كلمة

في هذا الفصل

6.1 مقدمة: من التنفيذ الخطي إلى اتخاذ القرار
6.2 القيم المنطقية في صميم اتخاذ القرار
6.3 العبارة الشرطية الأساسية: `if`
6.4 التفريع: `if...else`
6.5 شروط متعددة: `if...elif...else`
6.6 العوامل المنطقية: `and`, `or`, `not`
6.7 القيم «الصادقة» و«الكاذبة» ضمنيًا (Truthy & Falsy)
6.8 الجُمل الشرطية المُدمجة: Ternary Expression
6.9 match-case: الأسلوب الحديث الثوري (Python 3.10+)
6.10 Walrus Operator `:=` - الأداة الذكية (Python 3.8+)
6.11 الشروط المتداخلة (Nested Conditions)
6.12 أخطاء شائعة ونصائح احترافية
6.13 جدول مرجعي شامل للفصل
6.14 تمارين عملية للفصل
6.15 مشروع تطبيقي: «نظام تقييم كلمة المرور الذكي»
6.16 اختبر فهمك (Quiz سريع)
6.17 ملخص الفصل (Cheat Sheet)

*«حتى الآن، برامجك كانت 'مطيعة' تمامًا - تُنفِّذ كل سطر بالترتيب دون تفكير. في هذا الفصل، ستتعلّم كيف تجعل برنامجك 'يُفكّر' ويختار مسارات مختلفة بناءً على الظروف - هذا هو جوهر الذكاء في البرمجة.»*

## 6.1 📖 مقدمة: من التنفيذ الخطي إلى اتخاذ القرار

كل البرامج التي كتبتها حتى الآن تشترك في صفة واحدة: **تنفيذ خطي (Linear Execution)** - تبدأ من السطر الأول، وتنتهي بالسطر الأخير، بدون أي انحراف. لكن البرامج الحقيقية ليست كذلك أبدًا. فكّر في أي تطبيق تستخدمه يوميًا: تطبيق البنك «يقرر» إظهار رسالة خطأ إن أدخلت كلمة مرور خاطئة، نظام الملاحة «يقرر» مسارًا بديلاً إن وُجد ازدحام. هذا «القرار» هو ما سنتعلمه الآن.

# 6.2 القيم المنطقية في صميم اتخاذ القرار

## 🔄 مراجعة سريعة: كل قرار يعتمد على `True` أو `False`

تذكر من الفصل الثالث: `bool` لها قيمتان فقط. كل جملة شرطية في Python تعتمد جوهريًا على تقييم تعبير ما إلى `True` أو `False`:

```python
age: int = 20
print(age ⩾ 18)  # True  ← هذا هو أساس كل قرار برمجي
```

## 📊 عوامل المقارنة الكاملة (مراجعة وتوسيع)

| العامل | المعنى | مثال | النتيجة |
|---|---|---|---|
| `==` | يساوي | `5 == 5` | `True` |
| `≠` | لا يساوي | `5 ≠ 3` | `True` |
| `>` | أكبر من | `5 > 3` | `True` |
| `<` | أصغر من | `5 < 3` | `False` |
| `⩾` | أكبر أو يساوي | `5 ⩾ 5` | `True` |
| `⩽` | أصغر أو يساوي | `5 ⩽ 4` | `False` |

---

# 6.3 العبارة الشرطية الأساسية: `if`

## ✍️ الصيغة الأساسية

```python
if condition:
    # الكود الذي يُنفَّذ فقط إذا كان الشرط True
```

```python
age: int = 20

if age ⩾ 18:
    print("مسموح لك بالدخول")
```

### ⚠️ تحذير حرج: المسافة البادئة (Indentation) ليست اختيارية في Python!

هذا من أهم الفروق الجوهرية بين Python ولغات أخرى كثيرة. في لغات مثل Java أو C++، المسافات البادئة للتنسيق فقط (تجميلية). في Python، هي **جزء من قواعد اللغة نفسها**:

```python
# ✅ صحيح - المسافة البادئة متّسقة (4 مسافات هي المعيار العالمي)
if age ⩾ 18:
    print("مسموح")
    print("أهلاً بك")

# ❌ خطأ - IndentationError
if age ⩾ 18:
print("مسموح")  # لا توجد مسافة بادئة!
```

```
IndentationError: expected an indented block after 'if' statement on line 1
```

🎯 **قاعدة ذهبية**: استخدم دائمًا **4 مسافات** (وليس Tab) لكل مستوى من المسافة البادئة - هذا هو المعيار الرسمي المُوصى به في PEP 8. VS Code يفعل هذا تلقائيًا عند ضغط مفتاح Tab (يُترجمه لـ4 مسافات تلقائيًا بالإعدادات الافتراضية لإضافة Python).

⚠️ **تحذير من خطأ خفي خطير**: لا تخلط بين المسافات (Spaces) ومفتاح Tab في نفس الملف! هذا يُسبب أخطاء محيّرة جدًا تبدو «صحيحة بصريًا» لكن Python ترفضها. VS Code يُظهر هذه المشاكل تلقائيًا، لكن انتبه جيدًا إن نسخت كودًا من مصادر مختلفة.

## 6.4 التفريع: `if...else`

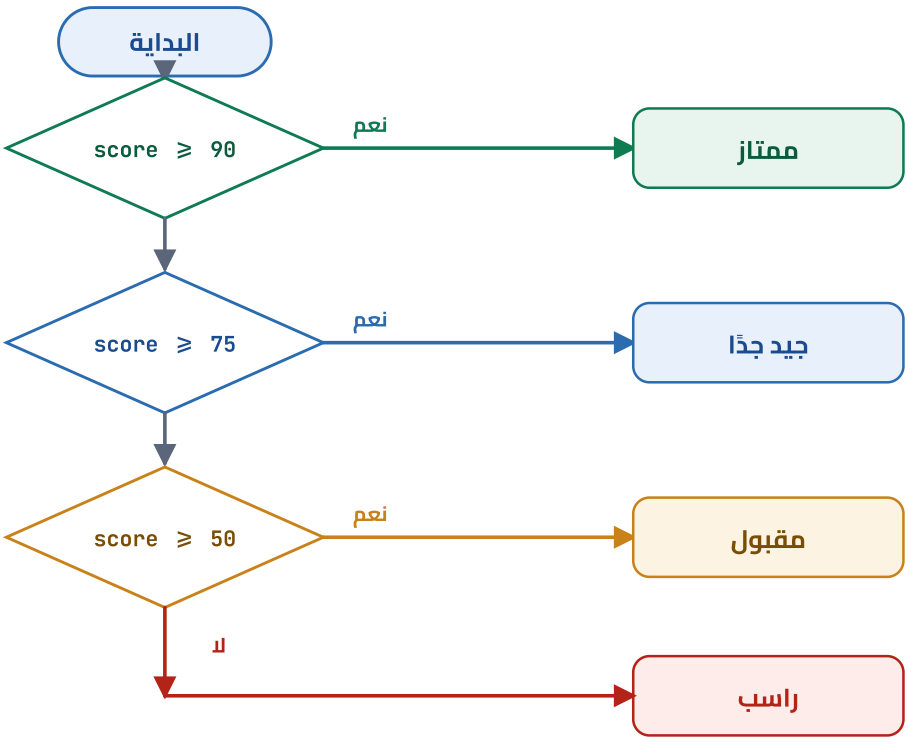
```python
age: int = 15

if age ⩾ 18:
    print("مسموح لك بالدخول")
else:
    print("عذرًا، يجب أن تكون 18 سنة أو أكثر")
```

**الفهم الجوهري:** `else` تعني «في أي حالة أخرى غير الشرط أعلاه» - لا تحتاج شرطًا خاصًا بها، فهي «كل ما تبقى».

---

## 6.5 شروط متعددة: `if...elif...else`




يُفحص الشرط من الأعلى للأسفل، ويُنفَّذ أول فرع يتحقّق فقط.

شكل 6.1 — تدفّق if / elif / else

```python
score: int = 75

if score ⩾ 90:
    grade: str = "ممتاز"
elif score ⩾ 80:
    grade = "جيد جدًا"
elif score ⩾ 70:
    grade = "جيد"
elif score ⩾ 60:
    grade = "مقبول"
else:
    grade = "راسب"

print(f"تقديرك: {grade}")  # تقديرك: جيد
```

## 🔬 كيف تُقيَّم `elif` فعليًا؟ (فهم دقيق مهم)

Python تفحص الشروط **بالترتيب من الأعلى للأسفل**، وبمجرد أن يكون أحدها `True` ، **تتوقف فورًا** عن فحص الباقي (حتى لو كانت شروط لاحقة صحيحة أيضًا):

```python
score: int = 95

if score ⩾ 90:
    print("ممتاز")      # هذا يُطبع ويتوقف هنا
elif score ⩾ 80:
    print("جيد جدًا")   # أيضًا صحيح ⩽ 80 ،رغم أن 95 لن يُفحص حتى!
```

> ⚠️ **خطأ شائع جدًا بسبب هذا المفهوم:** ترتيب الشروط **مهم جدًا**. إذا كتبت الشروط بترتيب خاطئ، ستحصل على نتائج غير منطقية:

```python
# ❌ ترتيب خاطئ - منطق معكوس
score: int = 95

if score ⩾ 60:
    print("مقبول")   # هذا سيُطبع لـ 95! لأنه أول شرط صحيح يُقابَل
elif score ⩾ 90:
    print("ممتاز")   # لن يصل هنا أبدًا لأي درجة أعلى من 60!
```

✅ **القاعدة الذهبية:** عند استخدام نطاقات متدرجة، رتّب الشروط دائمًا من **الأكثر تحديدًا (تشددًا) إلى الأقل**.

---

## 6.6 العوامل المنطقية: `and` ، `or` ، `not`

أحيانًا تحتاج لدمج أكثر من شرط واحد معًا.

### `and` : كلا الشرطين يجب أن يكونا صحيحين

```python
age: int = 25
has_license: bool = True

if age >= 18 and has_license:
    print("يمكنك القيادة")
```

### `or` : شرط واحد على الأقل يكفي

```python
is_weekend: bool = False
is_holiday: bool = True

if is_weekend or is_holiday:
    print("يوم إجازة!")
```

### `not` : عكس القيمة المنطقية

```python
is_raining: bool = False

if not is_raining:
    print("يمكننا الخروج للنزهة")
```

## 📊 جدول الحقيقة المبسّط (Truth Table)

| A | B | A and B | A or B | not A |
|---|---|---|---|---|
| True | True | True | True | False |
| True | False | False | True | False |
| False | True | False | True | True |
| False | False | False | False | True |

## 🎯 دمج عوامل متعددة مع الأقواس للوضوح

```python
age: int = 25
has_ticket: bool = True
is_vip: bool = False

if (age >= 18 and has_ticket) or is_vip:
    print("مسموح بالدخول")
```

✅ **أفضل الممارسات:** عند دمج `and` و `or` معًا في نفس الشرط، استخدم الأقواس دائمًا لتوضيح الأولوية - حتى لو كانت النتيجة نفسها بدون أقواس، فالوضوح للقارئ أهم بكثير.

## ⚠️ تحذير: Short-Circuit Evaluation (التقييم القصير)

Python «تتوقف مبكرًا» عن تقييم الشرط إن أمكنها معرفة النتيجة النهائية مسبقًا:

```python
def expensive_check() -> bool:
    print("تم استدعاء الفحص المكلف!")
    return True

# مع and: إذا كان الشرط الأول False، لن يُستدعى الثاني إطلاقًا
if False and expensive_check():
    pass
# لن تُطبع "تم استدعاء الفحص المكلف!" أبدًا!

# مع or: إذا كان الشرط الأول True، لن يُستدعى الثاني إطلاقًا
if True or expensive_check():
    pass
# لن تُطبع "تم استدعاء الفحص المكلف!" أبدًا!
```

> **فائدة عملية مهمة**: هذا السلوك يُستخدم بذكاء لتجنّب أخطاء محتملة:

```python
user_data: dict = {}  # قاموس فارغ (سنتعلمه بالفصل التاسع)

if "name" in user_data and user_data["name"] != "":
    print("الاسم موجود وغير فارغ")
# إذا لم يكن "name" موجودًا أصلاً، and تتوقف عند الشرط الأول
# وتتجنب خطأ محاولة الوصول لمفتاح غير موجود
```

---

## 6.7 القيم «الصادقة» و«الكاذبة» ضمنيًا (Truthy & Falsy)

### 🎭 مفهوم فريد ومهم جدًا في Python

في Python، ليس فقط `True` و `False` يُستخدمان في الشروط - أي قيمة يمكن تقييمها ضمنيًا كـ«صادقة» (Truthy) أو «كاذبة» (Falsy):

```python
name: str = ""

if name:
    print("الاسم موجود")
else:
    print("الاسم فارغ")  # هذا سيُطبع! لأن النص الفارغ يُعتبر Falsy
```

## 📊 القيم التي تُعتبر Falsy (كل ما عداها يُعتبر Truthy)

| القيمة | النوع |
|---|---|
| `False` | bool |
| `0` , `0.0` | أرقام |
| `""` (نص فارغ) | str |
| `None` | NoneType |
| `[]` (قائمة فارغة) | list - سنتعلمها لاحقًا |
| `{}` (قاموس فارغ) | dict - سنتعلمه لاحقًا |

**Python**

```python
value: int = 0

if value:
    print("له قيمة")
else:
    print("فارغ أو صفر")  # هنا سيُطبع
```

✅ **استخدام احترافي شائع جدًا:** التحقق من وجود قيمة مُدخلة من المستخدم:

**Python**

```python
name: str = input("(اختياري) أدخل اسمك: ").strip()

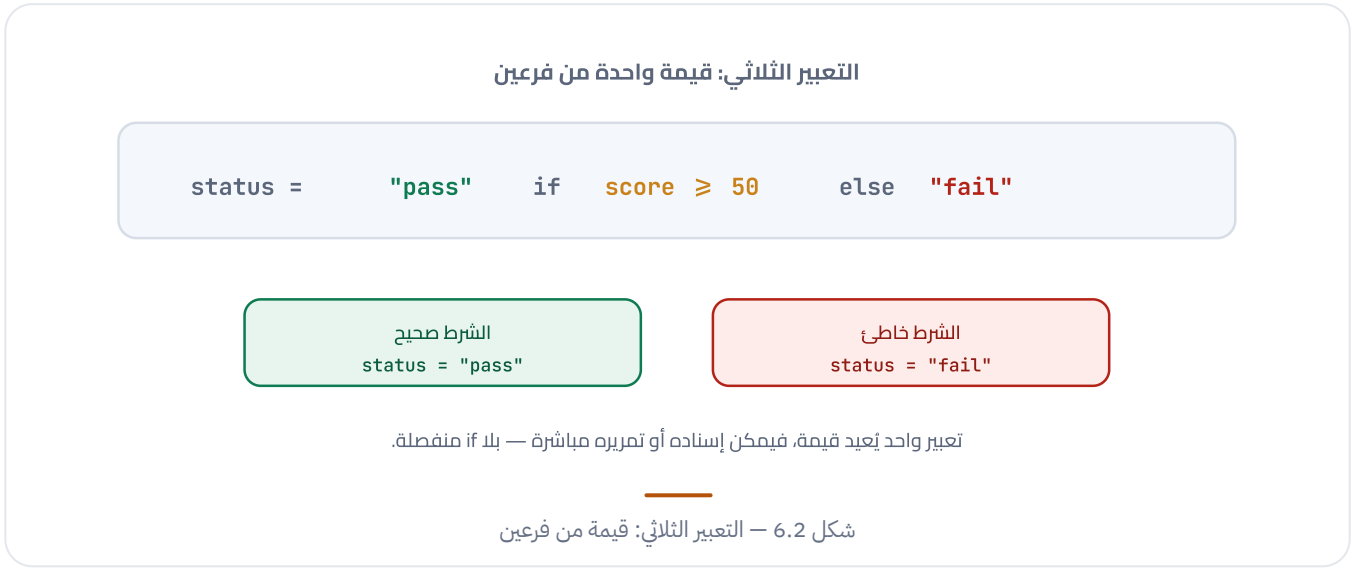
if name:
    print(f"مرحبًا {name}")
else:
    print("مرحبًا أيها الزائر المجهول")
```

⚠️ **تحذير دقيق:** لا تخلط بين `if value:` (فحص Truthy/Falsy) و `if value == True:` (مقارنة صريحة).
الأولى أكثر Pythonic (أسلوب Python الأصيل) والثانية أحيانًا تُسبب سلوكًا غير متوقع:

**Python**

```python
value = 5
print(value == True)  # False!  لأن 5 لا يساوي True رغم أنه Truthy حرفيًا
print(bool(value))    # True   لأن 5 يُقيَّم كـ Truthy
```

## 6.8 الجُمل الشرطية المُدمجة: Ternary Expression




شكل 6.2 — التعبير الثلاثي: قيمة من فرعين

### ✍️ صيغة مختصرة لـ if/else في سطر واحد

```python
age: int = 20

# الطريقة التقليدية
if age ⩾ 18:
    status = "بالغ"
else:
    status = "قاصر"

# الطريقة المختصرة (Ternary / Conditional Expression)
status = "بالغ" if age ⩾ 18 else "قاصر"
```

كلاهما يُعطي نفس النتيجة بالضبط، لكن الصيغة الثانية أقصر وأحيانًا أوضح للحالات البسيطة.

### ✅ متى تستخدم كل أسلوب؟

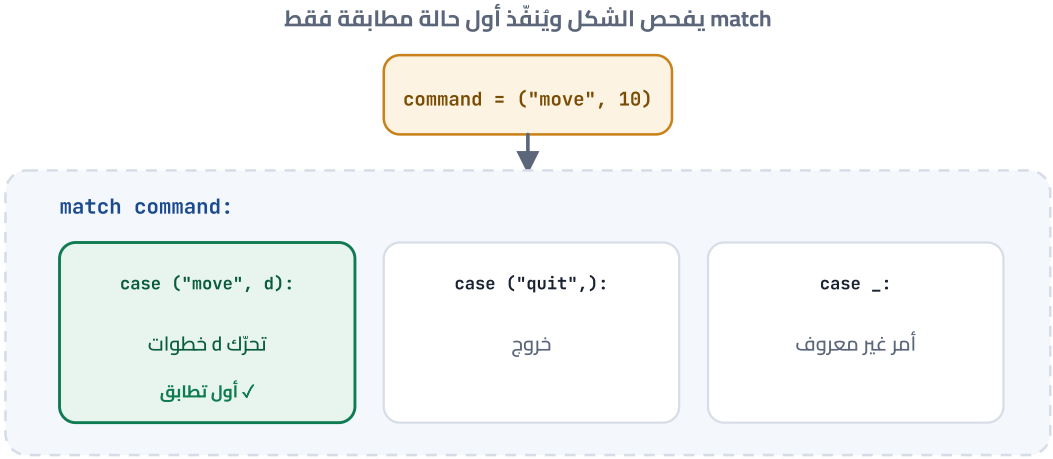
| الحالة | الأسلوب المُفضَّل |
|---|---|
| إسناد قيمة بسيطة بناءً على شرط واحد | Ternary (مختصرة) |
| تنفيذ عدة أسطر أو منطق معقد | if/else التقليدية |
| أكثر من شرط (elif متعددة) | if/elif/else التقليدية فقط |

```python
# ✅ استخدام ternary جيد للـ - بسيط وواضح
discount = 0.2 if is_vip else 0.0

# ❌ استخدام ternary سيء - معقد جدًا وغير مقروء كـ
result = "A" if x > 90 else "B" if x > 80 else "C" if x > 70 else "D"
# في هذه الحالة، استخدم if/elif/else التقليدية بدلاً من هذا!
```

---

## match-case 6.9 : الأسلوب الحديث الثوري (Python 3.10+)




شكل 6.3 — match-case: أول حالة مطابقة

### 🆕 لماذا هذه الميزة مهمة جدًا؟

حتى إصدار Python 3.9، كانت اللغة تفتقر لأداة «تبديل» (Switch) الموجودة في لغات أخرى كثيرة (Java, C++, JavaScript). بدءًا من Python 3.10، ظهرت أداة `match-case` - لكنها أقوى بكثير من أي `switch` تقليدي، لأنها تدعم ما يُسمى **Pattern Matching** (مطابقة الأنماط).

## ✍️ الاستخدام الأساسي

```python
day: str = "Monday"

match day:
    case "Monday":
        print("بداية الأسبوع")
    case "Friday":
        print("نهاية أسبوع العمل")
    case "Saturday" | "Sunday":  # هنا "أو" يعني | عامل
        print("عطلة نهاية الأسبوع")
    case _:
        print("يوم عادي")
```

💡 **ملاحظة مهمة**: الرمز `_` في `case _:` يعني «أي قيمة أخرى» (مشابه لـ `else` في if/elif/else) - يجب أن يكون دائمًا **آخر** حالة في match.

## 🔄 متى تستخدم `match-case` بدلاً من `if/elif`؟

```python
# سيناريو مثالي لـ match-case: فحص قيمة واحدة ضد عدة احتمالات محددة
http_status: int = 404

match http_status:
    case 200:
        print("نجاح")
    case 404:
        print("غير موجود")
    case 500:
        print("خطأ في الخادم")
    case _:
        print("حالة غير معروفة")
```

## 🚀 قوة إضافية: مطابقة الأنماط المتقدمة (Pattern Matching الحقيقي)

هنا تكمن القوة الحقيقية لـ `match-case` - أبعد بكثير من مجرد مقارنة قيم بسيطة:

Python

```python
# مطابقة أنواع البيانات والبنى (سنفهم هذا بعمق أكبر لاحقًا مع القوائم)
def describe_value(value):
    match value:
        case 0:
            return "صفر"
        case int() if value > 0:
            return "رقم صحيح موجب"
        case int() if value < 0:
            return "رقم صحيح سالب"
        case str():
            return "نص"
        case _:
            return "نوع غير معروف"

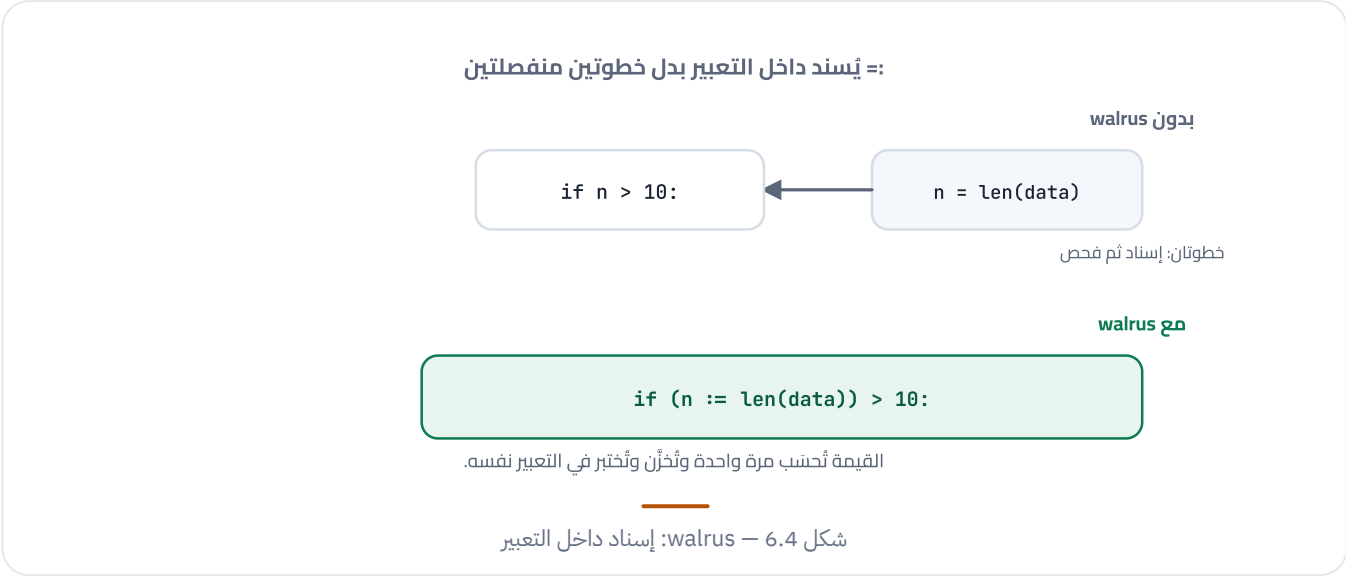
print(describe_value(5))        # رقم صحيح موجب
print(describe_value(-3))       # رقم صحيح سالب
print(describe_value("hi"))     # نص
```

🎯 لا تقلق إن بدت الأمثلة المتقدمة معقدة الآن - هذا طبيعي تمامًا. سنعود لـ `match-case` بعمق أكبر لاحقًا بعد تعلم القوائم والكائنات، حيث ستظهر قوتها الحقيقية بوضوح أكبر.

## 🔄 مقارنة شاملة: متى if/elif ومتى match-case؟

| الحالة | الأسلوب المُفضَّل |
|---|---|
| فحص نطاقات (مثل `score ⩾ 90`) | ✅ `if/elif/else` |
| فحص قيمة واحدة ضد عدة قيم محددة | ✅ `match-case` |
| شروط منطقية معقدة (`and` / `or` متعددة) | ✅ `if/elif/else` |
| مطابقة أنواع بيانات أو بنى معقدة | ✅ `match-case` |

# 6.10 Walrus Operator - الأداة الذكية (Python 3.8+) :=




شكل 6.4 — walrus: إسناد داخل التعبير

## 🤔 المشكلة التي يحلها

أحيانًا تحتاج لحساب قيمة واستخدامها في شرط **ونفس الوقت** تخزينها لاستخدام لاحق، مما يُسبب تكرارًا غير ضروري:

```python
# بدون walrus operator - تكرار الحساب مرتين
import random
number = random.randint(1, 10)

if number > 5:
    print(f"الرقم {number} أكبر من 5")
```

## ✍️ الحل بـ Walrus Operator

```python
import random

if (number := random.randint(1, 10)) > 5:
    print(f"الرقم {number} أكبر من 5")
```

العامل `:=` يُسند القيمة للمتغير **وفي نفس الوقت** يُعيد تلك القيمة لاستخدامها مباشرة في الشرط - كل هذا في سطر واحد.

**متى تستخدمه فعليًا؟** هذا العامل اختياري بالكامل، ويُستخدم بكثرة أكثر في حلقات `while` (سنراها في الفصل القادم) وفي list comprehensions (الفصل الثامن). لا تُفرط باستخدامه في الشروط البسيطة إن كان يُقلّل الوضوح.

⚠ **تحذير من الإفراط**: لا تستخدم `:=` فقط «لأنها أداة جديدة ومثيرة». استخدمها فقط عندما تُحسّن **الوضوح فعليًا** وتتجنب تكرار كود حقيقي.

---

## 6.11 الشروط المتداخلة (Nested Conditions)

```python
age: int = 25
has_id: bool = True

if age ⩾ 18:
    if has_id:
        print("يمكنك الدخول")
    else:
        print("تحتاج إثبات هوية")
else:
    print("عمرك غير كافٍ")
```

⚠ **تحذير احترافي**: الشروط المتداخلة بعمق (أكثر من 2-3 مستويات) تُصعّب قراءة الكود بشكل كبير. غالبًا يمكن تبسيطها بدمج الشروط بـ `and` :

```python
# بدلاً من التداخل العميق
if age ⩾ 18:
    if has_id:
        if not is_banned:
            print("مسموح")

# الأفضل: دمج الشروط
if age ⩾ 18 and has_id and not is_banned:
    print("مسموح")
```

### ✅ تقنية احترافية: Early Return / Guard Clauses

تقنية متقدمة (سنتعمق فيها أكثر مع الدوال في الفصل العاشر) تُجنّبك التداخل العميق:

```python
# بدلاً من هذا (متداخل)
def check_access(age, has_id):
    if age >= 18:
        if has_id:
            return "مسموح"
        else:
            return "تحتاج هوية"
    else:
        return "عمر غير كافٍ"

# استخدم هذا (أوضح - Guard Clauses)
def check_access(age, has_id):
    if age < 18:
        return "عمر غير كافٍ"
    if not has_id:
        return "تحتاج هوية"
    return "مسموح"
```

لا تقلق إن لم تفهم صيغة الدالة بالكامل بعد - النقطة المهمة هنا هي **مبدأ** تقليل التداخل، وسنعود لهذا بعمق في فصل الدوال.

---

## 6.12 أخطاء شائعة ونصائح احترافية

### ⚠️ الخطأ 1: استخدام `=` بدلاً من `==`

```python
age = 18
if age = 18:  # ❌ SyntaxError!
    print("البالغ")

if age == 18:  # ✅ صحيح
    print("البالغ")
```

### ⚠️ الخطأ 2: مقارنة Boolean بشكل زائد

```python
is_active: bool = True

# ❌ غير احترافي (زائد عن الحاجة)
if is_active == True:
    print("نشط")

# ✅ احترافي (Pythonic)
if is_active:
    print("نشط")
```

### ⚠️ الخطأ 3: نسيان `:` بعد الشرط

```python
if age >= 18  # ❌ SyntaxError: expected ':'
    print("بالغ")
```

### ⚠️ الخطأ 4: خلط المسافات البادئة (مسافات مع Tab)

```python
if age >= 18:
    print("بالغ")  # 4 مسافات
    print("مرحبا")  # Tab! ← سيُسبب IndentationError أو سلوكًا غير متوقع
```

### ✅ نصيحة احترافية: تجنّب شروط `== True` و`== False`

```python
# ❌ غير احترافي
if is_valid == False:
    print("غير صالح")

# ✅ احترافي
if not is_valid:
    print("غير صالح")
```

## 6.13 جدول مرجعي شامل للفصل

| الأداة | الاستخدام | مثال |
|---|---|---|
| `if` | شرط بسيط | `if x > 5:` |
| `if/else` | شرط مع بديل | `if x > 5: ... else: ...` |
| `if/elif/else` | شروط متعددة | `if...elif...elif...else` |
| `and` | كلا الشرطين صحيح | `x > 5 and x < 10` |
| `or` | أحد الشرطين على الأقل | `x < 5 or x > 10` |
| `not` | عكس القيمة | `not is_valid` |
| Ternary | شرط مختصر بسطر واحد | `"A" if x else "B"` |
| `match-case` | مطابقة قيمة لعدة احتمالات | `match x: case 1: ...` |
| `:=` | إسناد واستخدام في آن واحد | `if (n := get()) > 5:` |

---

## 6.14 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطلب من المستخدم إدخال عمره، ثم يطبع فئته العمرية باستخدام `if/elif/else`:
- أقل من 13: «طفل»
- 13 إلى 19: «مراهق»
- 20 إلى 59: «بالغ»
- 60 فأكثر: «كبير السن»

### 🟡 تمرين 2 (مستوى متوسط)

اكتب برنامج تسجيل دخول بسيط: يطلب اسم مستخدم وكلمة مرور، ويتحقق من تطابقهما مع قيم محددة مسبقًا في الكود (مثل `"admin"` و `"12345"`). استخدم العوامل المنطقية `and` بشكل صحيح، واطبع رسائل مختلفة حسب الحالة (كلاهما صحيح، اسم المستخدم خطأ فقط، كلمة المرور خطأ فقط، كلاهما خطأ).

### 🔴 تمرين 3 (تحدي)

أعد كتابة الفصل السادس بالكامل - تمرين «تصنيف المثلث»: اطلب من المستخدم أطوال 3 أضلاع، وحدد نوع المثلث:
- إذا كانت الأضلاع الثلاثة متساوية: «متساوي الأضلاع»
- إذا كان ضلعان متساويين فقط: «متساوي الساقين»
- إذا كانت كلها مختلفة: «مختلف الأضلاع»
- تحقّق أولاً إن كانت الأضلاع تُشكّل مثلثًا صالحًا أصلاً (مجموع أي ضلعين يجب أن يكون أكبر من الضلع الثالث)

---

## 6.15 مشروع تطبيقي: «نظام تقييم كلمة المرور الذكي»

### 📋 المهمة

أنشئ ملفًا باسم `password_checker.py` يُقيّم قوة كلمة مرور أدخلها المستخدم بناءً على معايير متعددة:

**المتطلبات:**

1. طلب كلمة مرور من المستخدم
2. التحقق من الشروط التالية باستخدام methods النصوص من الفصل الرابع:
   - الطول 8 أحرف على الأقل
   - تحتوي رقمًا واحدًا على الأقل
   - تحتوي حرفًا كبيرًا واحدًا على الأقل (إن كانت إنجليزية)
   - تحتوي رمزًا خاصًا واحدًا على الأقل (مثل `!@#$%` )
3. حساب «نقاط القوة» بناءً على عدد الشروط المُستوفاة
4. تصنيف القوة: ضعيفة (0-1 شروط) / متوسطة (2-3 شروط) / قوية (4 شروط)
5. طباعة تقرير مفصّل يوضح أي الشروط مُستوفاة وأيها ناقصة

## 💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python password_checker.py

```python
"""نظام تقييم قوة كلمة المرور."""

SPECIAL_CHARS: str = "!@#$%^&*()_+-="

password: str = input("أدخل كلمة مرور لتقييمها: ")

# فحص كل شرط على حدة
has_min_length: bool = len(password) >= 8
has_digit: bool = any(char.isdigit() for char in password)
has_upper: bool = any(char.isupper() for char in password)
has_special: bool = any(char in SPECIAL_CHARS for char in password)

# حساب عدد الشروط المُستوفاة
strength_score: int = sum([has_min_length, has_digit, has_upper, has_special])

# تحديد مستوى القوة
if strength_score <= 1:
    strength_level: str = "ضعيفة ❌"
elif strength_score <= 3:
    strength_level = "متوسطة ⚠️"
else:
    strength_level = "قوية ✅"

# طباعة التقرير
print("\n" + "=" * 40)
print("💪 تقرير تقييم كلمة المرور")
print("=" * 40)
print(f"الطول (8+ أحرف): {'✅' if has_min_length else '❌'}")
print(f"يحتوي رقمًا: {'✅' if has_digit else '❌'}")
print(f"يحتوي حرفًا كبيرًا: {'✅' if has_upper else '❌'}")
print(f"يحتوي رمزًا خاصًا: {'✅' if has_special else '❌'}")
print("-" * 40)
print(f"النتيجة: {strength_score}/4")
print(f"مستوى القوة: {strength_level}")
print("=" * 40)
```

**ملاحظة على الحل:** استخدمنا هنا `any()` مع تعبير يُشبه list comprehension (`for char in password if char.isdigit()`) - هذا تشويق بسيط لمفاهيم ستتعلمها بعمق أكبر في الفصل الثامن (القوائم). لا تقلق إن بدت غامضة الآن؛ لاحظ فقط كيف تتكامل المفاهيم مع تقدمك في الكتاب.

---

## 6.16 اختبر فهمك (Quiz سريع)

1. لماذا تُعتبر المسافة البادئة (Indentation) جزءًا من قواعد Python النحوية، وليست مجرد تنسيق جمالي؟
2. ما الفرق بين `and` و `or` من حيث الشروط اللازمة ليكون الناتج `True` ؟
3. ما معنى "Short-Circuit Evaluation"؟ أعطِ مثالاً عمليًا على فائدته.
4. متى يكون استخدام `match-case` أنسب من `if/elif/else` ؟
5. ما القيم التي تُعتبر "Falsy" ضمنيًا في Python حتى لو لم تكن `False` حرفيًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. لأن Python تعتمد على المسافة البادئة لتحديد أي الأسطر تنتمي لأي كتلة شرطية أو حلقة، بدلاً من استخدام أقواس `{}` كما في لغات أخرى - أي خطأ في المسافة البادئة يُسبب خطأ نحويًا (IndentationError) مباشرة.
2. `and` تتطلب أن يكون **كلا** الشرطين صحيحين ليكون الناتج True، بينما `or` تكتفي بأن يكون **واحد فقط** من الشرطين صحيحًا.
3. يعني أن Python تتوقف عن تقييم بقية الشرط بمجرد معرفة النتيجة النهائية (مثلاً مع `and` ، إن كان أول شرط False، لا داعي لفحص الباقي). فائدته العملية: تجنّب أخطاء محتملة، مثل التحقق من وجود مفتاح في قاموس قبل محاولة الوصول لقيمته في نفس الشرط.
4. عندما تحتاج مقارنة قيمة واحدة ضد عدة احتمالات محددة بوضوح (كمقارنة قيمة HTTP status code، أو يوم من أيام الأسبوع)، أو عند الحاجة لمطابقة أنماط/أنواع بيانات معقدة.
5. `False` , `0` , `0.0` , `""` (نص فارغ), `None` , `[]` (قائمة فارغة), `{}` (قاموس فارغ) - كل هذه القيم تُقيَّم كـ Falsy رغم أنها ليست `False` حرفيًا.

</details>

---

## 6.17 📌 ملخص الفصل (Cheat Sheet)

Python

```python
# الشرط الأساسي
if condition:
    pass
elif other_condition:
    pass
else:
    pass

# العوامل المنطقية
x and y    # كلاهما True
x or y     # أحدهما True
not x      # عكس x

# Ternary Expression
result = "A" if condition else "B"

# match-case (Python 3.10+)
match value:
    case 1:
        pass
    case 2 | 3:  # أو
        pass
    case _:  # الحالة الافتراضية
        pass

# Walrus Operator
if (n := get_value()) > 5:
    print(n)

# Truthy / Falsy
if my_string:  # True إن لم يكن فارغًا
    pass
```

### ⚠️ تذكّر دائمًا

- استخدم `==` للمقارنة، `=` للإسناد فقط
- 4 مسافات دائمًا للمسافة البادئة، لا تخلط مع Tab
- `if is_valid:` أفضل من `if is_valid == True:`
- رتّب شروط `elif` من الأكثر تحديدًا للأقل

## 🎯 ماذا بعد؟

تعلمنا الآن كيف يتخذ برنامجنا قرارات. لكن ماذا لو أردنا تنفيذ نفس الكود عدة مرات؟ تخيل طباعة «مرحبا» 100 مرة - هل ستكتب `print()` مئة مرة؟ بالطبع لا! في **الفصل السابع**، سنتعلم **التكرار (Loops)** - إحدى أقوى الأدوات في البرمجة، شاملة `for` و `while`، مع بناء مشاريع تفاعلية ممتعة مثل لعبة تخمين الأرقام.

فصل 07 / 26 · الجزء الثاني

07

# التكرار (Loops)

7 دقيقة قراءة | 19 قسمًا | 29 مثالًا | 1,496 كلمة

### في هذا الفصل

7.1 مقدمة: لماذا نحتاج التكرار؟
7.2 حلقة `while`: التكرار الشرطي
7.3 ⚠ الفخ الأخطر: الحلقات اللا نهائية (Infinite Loops)
7.4 حلقة `for`: التكرار على عناصر معروفة
7.5 متى تستخدم `for` ومتى تستخدم `while`؟ (قاعدة ذهبية لا تُنسى)
7.6 `break`: الخروج الفوري من الحلقة
7.7 `continue`: تخطّي التكرار الحالي فقط
7.8 `else` في الحلقات: ميزة نادرًا ما تُشرح بوضوح
7.9 الحلقات المتداخلة (Nested Loops)
7.10 `break` و `continue` في الحلقات المتداخلة
7.11 دالة `enumerate()`: الحصول على الفهرس والقيمة معًا
7.12 أداء الحلقات: نصائح احترافية مبكرة
7.13 أخطاء شائعة ونصائح احترافية
7.14 جدول مرجعي شامل للفصل
7.15 تمارين عملية للفصل
7.16 مشروع تطبيقي 1: «لعبة تخمين الأرقام»
7.17 مشروع تطبيقي 2: «جدول الضرب التفاعلي»
7.18 اختبر فهمك (Quiz سريع)
7.19 ملخص الفصل (Cheat Sheet)

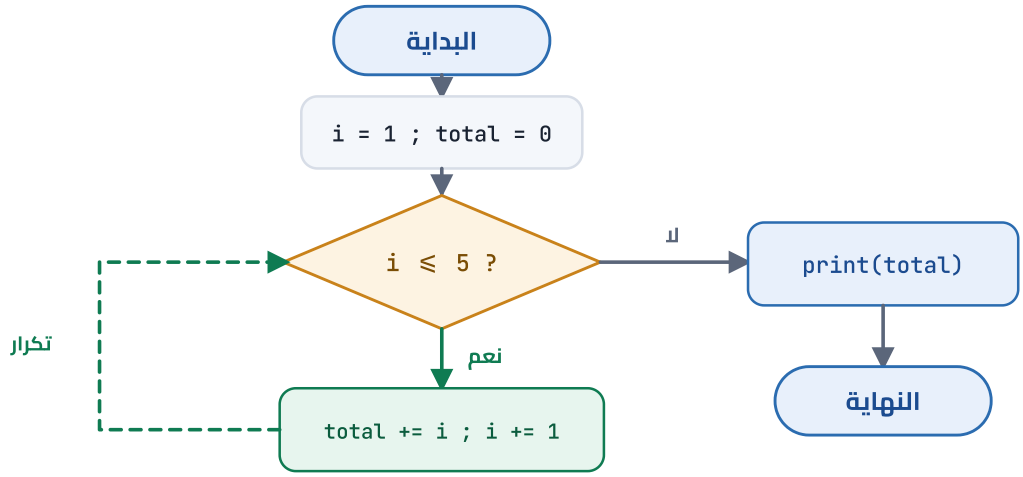
> *«التكرار هو أحد أعظم اختراعات البرمجة - تخيل كتابة سطر واحد من الكود يُنفَّذ آلاف المرات دون أن تكتبه آلاف المرات يدويًا. هذا هو السحر الحقيقي للحوسبة.»*

## 7.1 📖 مقدمة: لماذا نحتاج التكرار؟

تخيل أنك تريد طباعة الأرقام من 1 إلى 1000. هل ستكتب 1000 سطر `print()`؟ بالطبع لا. تخيل أيضًا أنك تريد معالجة بيانات 10,000 عميل في قاعدة بيانات - هل ستكتب كودًا منفصلاً لكل عميل؟ هذا مستحيل عمليًا.

**الحلقات (Loops)** هي الحل: أداة تسمح لك بتكرار تنفيذ كتلة من الكود عدة مرات، إما لعدد محدد من المرات، أو حتى تحقق شرط معين.

# 7.2 حلقة `while` : التكرار الشرطي




إن لم يتغيّر الشرط داخل الجسم (مثل نسيان i += 1) فستدور الحلقة إلى ما لا نهاية.

حلقة while تُعاد طالما بقي الشرط صحيحًا؛ ويجب أن يقرّب الجسم الحالة من الانتهاء.

شكل 7.1 — تدفّق حلقة while

## ✍️ الصيغة الأساسية

**Python**

```
while condition:
    # الكود الذي يتكرر طالما الشرط True
```

**Python**

```
count: int = 1

while count ⩽ 5:
    print(f"العدّ: {count}")
    count += 1

print("انتهى العد!")
```

**الناتج:**

```
العدّ: 1
العدّ: 2
العدّ: 3
العدّ: 4
العدّ: 5
انتهى العد!
```

### 🧹 كيف تعمل `while` خطوة بخطوة؟

```
1. فحص الشرط (count ⩽ 5)
2. إذا True → نفّذ الكود بالداخل → عُد للخطوة 1
3. إذا False → اخرج من الحلقة بالكامل
```

> 💡 **ملاحظة جوهرية:** لاحظ أننا احتجنا `count += 1` **بأنفسنا** داخل الحلقة. `while` لا «تعرف» متى تتوقف تلقائيًا - **أنت** المسؤول عن تحديث الشرط بحيث يُصبح `False` في وقت ما، وإلا...

---

## 7.3 الفخ الأخطر: الحلقات اللا نهائية (Infinite Loops)

### 🔥 المشكلة

```python
count: int = 1

while count ⩽ 5:
    print(f"العدّ: {count}")
    # نسيان count += 1 !
```

هذا الكود سيطبع `"العدّ: 1"` **إلى الأبد** دون توقف، لأن `count` يبقى `1` دائمًا، والشرط `1 ⩽ 5` يبقى `True` للأبد.

### 🚨 كيف تتعامل مع حلقة لا نهائية حدثت فعليًا؟

إذا وقعت في هذا الموقف (سيحدث للجميع في مرحلة ما):

- **في الطرفية:** اضغط `Ctrl + C` لإيقاف البرنامج فورًا
- **في VS Code:** اضغط زر "Stop" (مربع أحمر) في شريط التشغيل

⚠️ **تحذير حرج جدًا:** قبل تشغيل أي حلقة `while`، **تأكد دائمًا** أن هناك مسارًا منطقيًا واضحًا يجعل الشرط `False` في وقت ما. هذا أحد أكثر الأخطاء إحباطًا للمبتدئين - خصوصًا عند نسيان تحديث متغير العدّاد.

### ✅ قائمة تحقق قبل كتابة أي حلقة `while`

```
✓ هل قمت بتهيئة المتغير المُستخدَم في الشرط قبل الحلقة؟
✓ هل يوجد سطر داخل الحلقة يُغيّر قيمة ذلك المتغير؟
✓ هل التغيير يقترب فعليًا من جعل الشرط False؟
```

### 🎯 استخدام متعمّد للحلقة اللانهائية (مع `break`)

أحيانًا، نُريد فعليًا حلقة «لا نهائية» لكن مع شرط خروج داخلي واضح - سنتعلم `break` بعد قليل:

```python
while True:  # لا نهائية عمدًا
    user_input: str = input("اكتب 'خروج' لإنهاء البرنامج: ")
    if user_input == "خروج":
        break  # نخرج من الحلقة هنا
    print(f"كتبت: {user_input}")
```

💡 هذا النمط (`while True` مع `break` داخلي) **شائع جدًا واحترافي تمامًا** في البرمجة الحقيقية - خصوصًا في البرامج التفاعلية التي تنتظر مدخلات المستخدم باستمرار حتى يختار الخروج.

---

## 7.4 حلقة `for`: التكرار على عناصر معروفة

### ✍️ الصيغة الأساسية مع `range()`

```python
for i in range(5):
    print(i)
```

**الناتج:**

```
0
1
2
3
4
```

⚠️ **تحذير مهم:** `range(5)` يُنتج الأرقام من **0 إلى 4** (خمسة أرقام)، **وليس** من 1 إلى 5! هذا يتبع نفس منطق الفهرسة التي تعلمناها (تبدأ من 0، والنهاية غير مُضمَّنة).

## 📊 صيغ `range()` الثلاث

Python

```python
# range(end) - حتى end-1 من 0
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

# range(start, end) - من start حتى end-1
for i in range(2, 6):
    print(i)  # 2, 3, 4, 5

# range(start, end, step) - بقفزات محددة
for i in range(0, 10, 2):
    print(i)  # 0, 2, 4, 6, 8

# step سالب للعد التنازلي
for i in range(10, 0, -1):
    print(i)  # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

## 🔄 `for` على النصوص مباشرة

Python

```python
word: str = "Python"

for letter in word:
    print(letter)
```

**الناتج:**

```
P
y
t
h
o
n
```

> **فهم جوهري مهم:** `for` في Python تختلف عن لغات أخرى كثيرة (مثل C++ أو Java التقليدية) حيث تعتمد على عدّاد رقمي صريح. في Python، `for` تعني حرفيًا: **«لكل عنصر في هذه المجموعة القابلة للتكرار (Iterable)»**. هذا سيصبح أوضح بكثير عندما نتعلم القوائم في الفصل القادم.

---

## 7.5 متى تستخدم `for` ومتى تستخدم `while`؟ (قاعدة ذهبية لا تُنسى)

هذا سؤال يُربك كثيرًا من المبتدئين. إليك القاعدة البسيطة والحاسمة:

| الحالة | استخدم |
|---|---|
| تعرف **بالضبط** عدد مرات التكرار مسبقًا | ✅ `for` |
| التكرار يعتمد على **شرط** قد يتغيّر بشكل غير متوقع | ✅ `while` |
| التكرار على عناصر مجموعة (نص، قائمة...) | ✅ `for` |
| انتظار إدخال معيّن من المستخدم (لا تعرف متى سيأتي) | ✅ `while` |

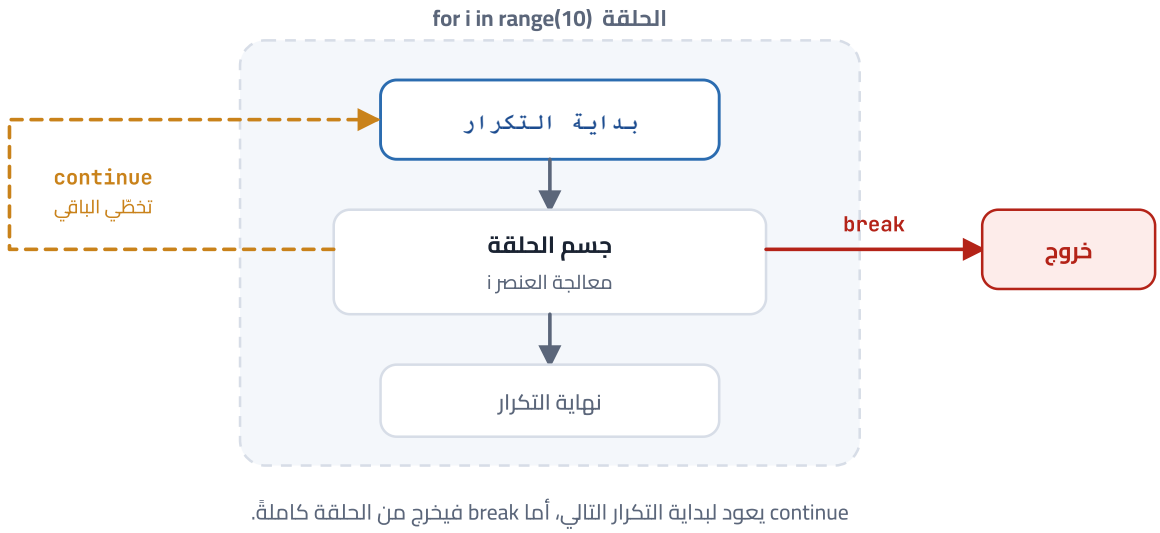
**Python**

```python
# مثال for: نعرف بالضبط أننا نريد 5 تكرارات
for i in range(5):
    print("مرحبا")

# مثال while: لا نعرف متى سيُدخل المستخدم كلمة المرور الصحيحة
password: str = ""
while password != "1234":
    password = input("أدخل كلمة المرور: ")
```

---

## 7.6 break : الخروج الفوري من الحلقة




شكل 7.2 — الفرق بين break وcontinue

Python

```
for number in range(1, 11):
    if number == 5:
        break  # إيقاف الحلقة بالكامل فور الوصول لـ 5
    print(number)
```

**الناتج:**

```
1
2
3
4
```

`break` يُنهي الحلقة **بالكامل** فورًا، بغض النظر عن باقي عدد مرات التكرار المحتملة.

### 🎯 استخدام عملي: البحث عن عنصر والتوقف فور إيجاده

```python
numbers: list[int] = [4, 7, 2, 9, 5, 1]  # سنتعلم القوائم بعمق قريبًا
target: int = 9
found: bool = False

for num in numbers:
    if num == target:
        found = True
        break  # لا داعي لمتابعة البحث بعد إيجاده!

if found:
    print("العنصر موجود!")
```

---

## 7.7 `continue`: تخطّي التكرار الحالي فقط

```python
for number in range(1, 11):
    if number % 2 == 0:  # إذا كان زوجيًا
        continue  # تخطّى باقي كود هذا التكرار، انتقل للتالي مباشرة
    print(number)
```

**الناتج:**

```
1
3
5
7
9
```

**الفرق الجوهري بين `break` و `continue`:**

| الأمر | التأثير |
|---|---|
| `break` | يُنهي الحلقة **بالكامل** فورًا |
| `continue` | يتخطّى **فقط التكرار الحالي**، ويستمر للتكرار التالي |

# 7.8 `else` في الحلقات: ميزة نادرًا ما تُشرح بوضوح

### 🤔 مفهوم فريد في Python

Python تسمح بإضافة `else` بعد حلقة `for` أو `while` - لكن سلوكها مختلف تمامًا عمّا قد تتوقعه:

> ***قاعدة `else` في الحلقات:*** *تُنفَّذ كتلة `else` فقط إذا* ***اكتملت الحلقة بالكامل دون استخدام `break`.***

```python
for number in range(1, 6):
    print(number)
else:
    print("اكتملت الحلقة بنجاح!")
```

**الناتج:**

```
1
2
3
4
5
اكتملت الحلقة بنجاح!
```

لكن إذا استخدمنا `break`:

```python
for number in range(1, 6):
    if number == 3:
        break
    print(number)
else:
    print("اكتملت الحلقة بنجاح!")  # لن تُطبع!
```

**الناتج:**

```
1
2
```

💡 **استخدام عملي حقيقي:** هذا مفيد جدًا في سيناريوهات البحث - «هل وجدت العنصر؟ إن لم أجده بعد فحص الكل، نفّذ شيئًا»:

```python
numbers: list[int] = [1, 3, 5, 7, 9]
target: int = 4

for num in numbers:
    if num == target:
        print("وُجد!")
        break
else:
    print("لم يُوجد العنصر في القائمة")  # ستُطبع هذه، لأن 4 غير موجود
```

⚠️ **تحذير:** هذه الميزة تُربك حتى المبرمجين ذوي الخبرة لأنها غير بديهية (`else` هنا لا تعني «عكس الحلقة» بل «لم يحدث break»). استخدمها بحذر، وتأكد أن فريقك (أو القارئ المستقبلي لكودك) يفهم هذا السلوك إن استخدمتها.

---

## 7.9 الحلقات المتداخلة (Nested Loops)

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

الدورة الخارجية تختار الصف
والداخلية تمرّ على كل عمود فيه

| i \ j | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0,0 | 0,1 | 0,2 |
| 1 | 1,0 | 1,1 | 1,2 |
| 2 | 2,0 | 2,1 | 2,2 |

تسع طباعات = 3 × 3

عدد التكرارات الكلي = حاصل ضرب عددَي دورات الحلقتين.

شكل 7.3 — الحلقات المتداخلة على شبكة 3×3

```python
for i in range(1, 4):
    for j in range(1, 4):
        print(f"i={i}, j={j}")
```

**الناتج:**

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3
```

**كيف تفهم الحلقات المتداخلة؟** لكل تكرار واحد في الحلقة **الخارجية**، تُنفَّذ الحلقة **الداخلية** بالكامل من البداية للنهاية. هنا: الحلقة الخارجية 3 مرات × الحلقة الداخلية 3 مرات = 9 تكرارات إجمالية.

## 🎯 مثال عملي: جدول الضرب الكامل

```python
for i in range(1, 4):
    for j in range(1, 4):
        result: int = i * j
        print(f"{i} × {j} = {result}")
    print("-" * 15)  # فاصل بعد كل جدول فرعي
```

**تحذير أداء:** الحلقات المتداخلة بعمق (3+ مستويات) يمكن أن تُصبح بطيئة جدًا مع البيانات الكبيرة (تعقيد $O(n^2)$ أو أسوأ - مفهوم سنلمسه أكثر لاحقًا في فصل الأداء). كن حذرًا عند التعامل مع مجموعات بيانات كبيرة.

## 7.10 `break` و `continue` في الحلقات المتداخلة

### ⚠️ نقطة دقيقة يجب فهمها جيدًا

```python
for i in range(3):
    for j in range(3):
        if j == 1:
            break  # يُخرج فقط من الحلقة الداخلية!
        print(f"i={i}, j={j}")
```

**الناتج:**

```
i=0, j=0
i=1, j=0
i=2, j=0
```

🎯 **قاعدة مهمة:** `break` و `continue` يؤثران فقط على **الحلقة التي يُكتبان مباشرة بداخلها** (الأقرب لهما)، وليس على كل الحلقات المتداخلة. إذا أردت الخروج من كل الحلقات المتداخلة دفعة واحدة، تحتاج تقنيات أكثر تقدمًا (مثل استخدام متغير علم "flag"، أو تحويل الكود لدالة واستخدام `return` - سنتعلم هذا في الفصل العاشر).

---

## 7.11 دالة `enumerate()`: الحصول على الفهرس والقيمة معًا

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]  # تشويق للفصل القادم

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

**الناتج:**

```
0: تفاح
1: موز
2: برتقال
```

> **استخدام عملي شائع جدًا:** تُستخدم `enumerate()` كثيرًا عندما تحتاج **موضع** العنصر بالإضافة لقيمته - أنيق جدًا مقارنة بإدارة عدّاد يدوي منفصل.

```python
# بدء العدّ من 1 بدلاً من 0
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
```

---

## 7.12 أداء الحلقات: نصائح احترافية مبكرة

### ✅ نصيحة 1: تجنّب الحسابات المتكررة غير الضرورية

```python
import math

numbers: list[int] = [1, 2, 3, 4, 5]

# ❌ يُعيد حساب len() في كل تكرار (رغم أنها سريعة، المبدأ عام) - غير فعّال
for i in range(len(numbers)):
    print(numbers[i])

# ✅ أفضل - أسلوب Pythonic مباشر
for number in numbers:
    print(number)
```

### ✅ نصيحة 2: استخدم `range()` بدلاً من بناء قوائم يدوية للعد

```python
# ❌ غير ضروري وغير فعّال من حيث الذاكرة
numbers_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
for n in numbers_list:
    print(n)

# ✅ أفضل - range لا يُنشئ قائمة فعلية في الذاكرة، بل يُولّد القيم عند الحاجة فقط
for n in range(10):
    print(n)
```

> **ملاحظة متقدمة للفضوليين:** `range()` في Python 3 هي كائن «كسول» (Lazy) - لا تُخزّن كل الأرقام في الذاكرة دفعة واحدة، بل تُولّدها واحدًا تلو الآخر عند الحاجة فعليًا. هذا يجعلها فعّالة جدًا حتى مع نطاقات ضخمة مثل `range(1_000_000_000)` .

---

## 7.13 أخطاء شائعة ونصائح احترافية

### ⚠️ الخطأ 1: الحلقة اللا نهائية بسبب نسيان تحديث المتغير

```python
# ❌ خطأ فادح - حلقة لا نهائية
i = 0
while i < 5:
    print(i)
    # نسيان i += 1 !
```

### ⚠️ الخطأ 2: استخدام `range()` بشكل خاطئ (توقّع تضمين النهاية)

```python
for i in range(1, 5):
    print(i)  # 1, 2, 3, 4 5 ليس - فقط!
```

إذا أردت تضمين 5:

```python
for i in range(1, 6):  # ← 1+ يجب أن تكون النهاية
    print(i)  # 1, 2, 3, 4, 5
```

### ⚠️ الخطأ 3: تعديل متغير العدّاد يدويًا داخل حلقة `for`

```python
for i in range(5):
    print(i)
    i += 10  # ⚠️ هذا لا يُؤثر على تكرارات range() القادمة!
```

> **توضيح مهم:** على عكس `while`، حلقة `for` مع `range()` تتحكم في قيمة `i` تلقائيًا من `range()` نفسها في كل تكرار - تعديلك اليدوي لـ `i` بالداخل لن يُغيّر مسار الحلقة إطلاقًا.

### ✅ نصيحة احترافية: تسمية متغيرات الحلقة بوضوح

```python
# ❌ غامض عند التداخل
for i in range(10):
    for j in range(10):
        pass

# ✅ أوضح عند وجود معنى حقيقي
for row in range(10):
    for column in range(10):
        pass
```

---

## 7.14 جدول مرجعي شامل للفصل

| الأداة | الوظيفة | مثال |
|---|---|---|
| `while` | تكرار حسب شرط | `while x < 5:` |
| `for ... in` | تكرار على عناصر | `for item in items:` |
| `range(n)` | أرقام من 0 إلى n-1 | `range(5)` |
| `range(a,b)` | أرقام من a إلى b-1 | `range(2,8)` |
| `range(a,b,s)` | بقفزة s | `range(0,10,2)` |
| `break` | خروج فوري كامل | - |
| `continue` | تخطّي التكرار الحالي | - |
| `else` (مع حلقة) | يُنفَّذ إن لم يحدث break | - |
| `enumerate()` | فهرس + قيمة معًا | `enumerate(items)` |

---

## 7.15 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطبع جدول الضرب الكامل لرقم يُدخله المستخدم (من 1 إلى 10)، باستخدام حلقة `for`.

### 🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يطلب من المستخدم أرقامًا متتالية (باستخدام `while`)، ويتوقف فقط عندما يُدخل المستخدم الرقم `0`، ثم يطبع: مجموع كل الأرقام المُدخلة (غير شاملة الصفر)، وعددها، ومتوسطها.

### 🔴 تمرين 3 (تحدي)

اكتب برنامجًا يطبع «مثلث النجوم» التالي باستخدام حلقات متداخلة، حيث يُحدد المستخدم عدد الصفوف:

```
*
**
***
****
*****
```

---

## 7.16 مشروع تطبيقي 1: «لعبة تخمين الأرقام»

### 📋 المهمة

أنشئ ملفًا باسم `number_guessing_game.py`:

**المتطلبات:**

1. البرنامج يختار رقمًا عشوائيًا بين 1 و100 (باستخدام `random.randint`)
2. يطلب من المستخدم تخمين الرقم باستمرار (`while True` مع `break`)
3. بعد كل محاولة، يُخبر المستخدم إن كان تخمينه «أكبر من اللازم» أو «أصغر من اللازم»
4. عند التخمين الصحيح، يطبع تهنئة مع عدد المحاولات التي استغرقها
5. (تحدٍ إضافي) حدّد حدًا أقصى لعدد المحاولات (مثلاً 7)، وإن لم ينجح، اطبع «انتهت محاولاتك» مع كشف الرقم الصحيح

## 💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python number_guessing_game.py

```python
"""لعبة تخمين الأرقام - احزر الرقم السري!"""

import random

MAX_ATTEMPTS: int = 7
secret_number: int = random.randint(1, 100)
attempts: int = 0

print("=" * 40)
print("🎯  لعبة تخمين الأرقام")
print("=" * 40)
print(f"لديك {MAX_ATTEMPTS} محاولات لتخمين رقمًا بين 1 و100!")

while attempts < MAX_ATTEMPTS:
    guess_text: str = input("\nخمّن الرقم: ").strip()

    if not guess_text.isdigit():
        print("⚠️ من فضلك أدخل رقمًا صحيحًا فقط!")
        continue

    guess: int = int(guess_text)
    attempts += 1

    if guess == secret_number:
        print(f"\n🎉 مبروك! خمّنت الرقم الصحيح في {attempts} محاولة!")
        break
    elif guess < secret_number:
        print("📈 الرقم السري أكبر من تخمينك")
    else:
        print("📉 الرقم السري أصغر من تخمينك")

    remaining: int = MAX_ATTEMPTS - attempts
    if remaining > 0:
        print(f"محاولاتك المتبقية: {remaining}")
else:
    print(f"\n😢 انتهت محاولاتك! الرقم السري كان: {secret_number}")

print("=" * 40)
```

**ملاحظة على الحل:** لاحظ استخدامنا لـ `else` الخاصة بالحلقة `while` - تُنفَّذ فقط إذا استُنفدت كل المحاولات دون `break` (أي دون تخمين صحيح). هذا تطبيق عملي ممتاز لما تعلمناه في القسم 7.7.

---

## 7.17 مشروع تطبيقي 2: «جدول الضرب التفاعلي»

### 📋 المهمة

أنشئ ملفًا باسم `multiplication_table.py` يطبع جدول ضرب كامل ومنسّق جميل بصريًا (من 1×1 حتى 10×10) بشكل جدول حقيقي باستخدام حلقات متداخلة و f-strings للمحاذاة.

### 💻 حل نموذجي

✅ اضغط لرؤية الحل الكامل

Python multiplication_table.py

```python
"""طباعة جدول ضرب منسّق من 1 إلى 10."""

SIZE: int = 10

# طباعة رأس الجدول
print("    ", end="")
for col in range(1, SIZE + 1):
    print(f"{col:4}", end="")
print()
print("-" * (SIZE * 4 + 4))

# طباعة الجدول الكامل
for row in range(1, SIZE + 1):
    print(f"{row:3} |", end="")
    for col in range(1, SIZE + 1):
        result: int = row * col
        print(f"{result:4}", end="")
    print()  # سطر جديد بعد كل صف
```

**ملاحظة على الحل:** استخدمنا `end=""` في `print()` لمنع الانتقال التلقائي لسطر جديد (الافتراضي)، مما يسمح لنا بطباعة عدة قيم في نفس السطر - تقنية مفيدة جدًا لبناء جداول نصية.

---

## 7.18 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين متى تستخدم `for` ومتى تستخدم `while`؟
2. لماذا `range(1, 5)` يُنتج الأرقام 1 إلى 4 فقط، وليس حتى 5؟
3. ما الفرق بين `break` و `continue`؟
4. متى تُنفَّذ كتلة `else` المرتبطة بحلقة `for` أو `while`؟
5. ما السبب الأكثر شيوعًا لحدوث حلقة لا نهائية؟

### ✅ الإجابات النموذجية

1. `for` تُستخدم عندما تعرف مسبقًا عدد مرات التكرار أو تُكرر على عناصر مجموعة معروفة، بينما `while` تُستخدم عندما يعتمد التكرار على شرط قد يتغيّر بشكل غير متوقع (كانتظار إدخال معيّن من المستخدم).
2. لأن `range()` تتبع نفس منطق الفهرسة في Python حيث نقطة البداية مُضمَّنة والنهاية غير مُضمَّنة، تمامًا كالتقطيع في النصوص.
3. `break` يُنهي الحلقة بالكامل فورًا، بينما `continue` يتخطّى فقط باقي كود التكرار الحالي وينتقل مباشرة للتكرار التالي.
4. تُنفَّذ فقط إذا اكتملت الحلقة بالكامل بشكل طبيعي دون استخدام `break` في أي تكرار.
5. نسيان تحديث المتغير المُستخدَم في شرط حلقة `while` بحيث يبقى الشرط `True` دائمًا دون أي تغيير فعلي يقربه من `False` .

## 7.19 📌 ملخص الفصل (Cheat Sheet)

```python
# while - تكرار شرطي
while condition:
    # الكود
    update_condition()  # لا تنسَ هذا!

# for - تكرار على عناصر معروفة
for item in collection:
    print(item)

for i in range(5):         # 0 إلى 4
for i in range(2, 8):      # 2 إلى 7
for i in range(0, 10, 2): # 2 بقفزة

# التحكم بالحلقة
break      # خروج فوري كامل
continue   # تخطّي التكرار الحالي فقط

# else مع الحلقة (نادر لكن مفيد)
for item in items:
    if condition:
        break
else:
    print("لم يحدث break")

# enumerate - فهرس + قيمة
for index, value in enumerate(items):
    print(index, value)
```

### ⚠️ تذكّر دائمًا

- في `while`: تأكد دائمًا من وجود تحديث يُقرِّب الشرط من `False`
- `range(a, b)` لا يشمل `b` - فقط حتى `b-1`
- `break` يخرج من الحلقة **الأقرب** فقط في التداخل
- استخدم `Ctrl+C` لإيقاف حلقة لا نهائية عالقة

### 🎯 ماذا بعد؟

حتى الآن، تعاملنا مع متغيرات تحمل **قيمة واحدة** فقط (رقم، نص، منطقية). لكن ماذا لو أردنا تخزين **مجموعة كاملة** من القيم معًا - كقائمة بأسماء الطلاب، أو درجاتهم؟ في **الفصل الثامن**، سندخل عالم **هياكل البيانات** بدءًا من أهمها وأكثرها استخدامًا: **القوائم (Lists)** - حيث ستتكامل كل مهارات الحلقات والشروط التي تعلمناها لبناء برامج حقيقية وقوية.

فصل 08 / 26 · الجزء الثاني

# 08 هياكل البيانات - القوائم (Lists)

8 دقيقة قراءة | 18 قسمًا | 45 مثالًا | 1,527 كلمة

### في هذا الفصل

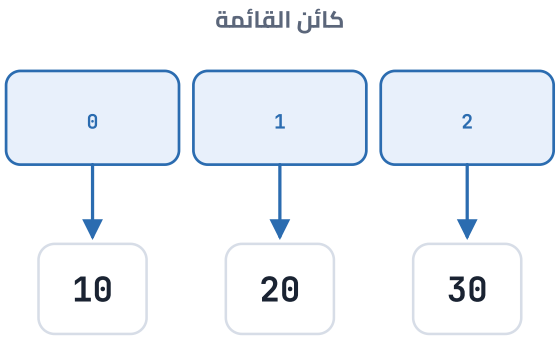
8.1 مقدمة: نقطة تحوّل حقيقية في رحلتك
8.2 ما هي القائمة (List)؟
8.3 الوصول للعناصر: الفهرسة والتقطيع (مألوفة من النصوص!)
8.4 الفرق الجوهري: القوائم Mutable (قابلة للتعديل)!
8.5 أهم Methods القوائم
8.6 القوائم المتداخلة (Nested Lists)
8.7 ⚠️ الفخ الأشهر على الإطلاق: نسخ القوائم (Shallow Copy Trap)
8.8 List Comprehension: الأسلوب الأنيق والـ"Pythonic"
8.9 دوال مُدمجة قوية تعمل مع القوائم
8.10 القوائم والحلقات: دمج كل ما تعلمناه
8.11 🔄 متى تستخدم القوائم؟ (تمهيد لمقارنة أشمل قادمة)
8.12 Tuples: الأخ «غير القابل للتعديل» للقائمة
8.13 أخطاء شائعة ونصائح احترافية
8.14 جدول مرجعي شامل للفصل
8.15 تمارين عملية للفصل
8.16 مشروع تطبيقي: «نظام إدارة قائمة مهام (To-Do List)»
8.17 اختبر فهمك (Quiz سريع)
8.18 ملخص الفصل (Cheat Sheet)

> *«إذا كانت المتغيرات التي تعلمناها حتى الآن أشبه بصناديق فردية، فإن القوائم هي أرفف كاملة يمكنها احتواء عدد غير محدود من هذه الصناديق، منظّمة بترتيب محدد.»*

## 8.1 📖 مقدمة: نقطة تحوّل حقيقية في رحلتك

كل ما تعلمته حتى الآن كان يتعامل مع **قيمة واحدة** في كل متغير. لكن العالم الحقيقي مليء بالمجموعات: قائمة أصدقائك، درجات طلاب فصل كامل، منتجات متجر إلكتروني. في هذا الفصل، ستتعلم أول **هيكل بيانات** حقيقي في Python - والذي سيغيّر جذريًا نوعية البرامج التي يمكنك بناءها.

## 8.2 ما هي القائمة (List)؟




كل خليّة تخزّن مرجعًا (عنوانًا) إلى كائن مستقل، لا القيمة نفسها.

لهذا يمكن للقائمة أن تحتوي أنواعًا مختلفة، وأن تتغيّر عناصرها بحرية.

شكل 8.1 — القائمة مصفوفة مراجع إلى كائنات

### 🎁 التعريف والإنشاء

القائمة هي مجموعة **مُرتَّبة** و**قابلة للتعديل** من العناصر، يمكن أن تحتوي أي نوع بيانات - وحتى أنواعًا مختلفة معًا في نفس القائمة:

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]
numbers: list[int] = [10, 20, 30, 40]
mixed: list = ["25", "محمد", True, 3.14]  # مسموح، لكن نادرًا ما يُستخدم عمليًا

empty_list: list = []  # قائمة فارغة
```

💡 **Type Hint للقوائم**: لاحظ الصيغة `list[str]` - تخبر القارئ والأدوات أن هذه قائمة تحتوي عناصر من نوع `str` تحديدًا. هذا النمط (`list[النوع]`) أصبح قياسيًا في Python الحديثة (3.9+).

# 8.3 الوصول للعناصر: الفهرسة والتقطيع (مألوفة من النصوص!)

## 🔢 نفس منطق فهرسة النصوص بالضبط

```python
fruits: list[str] = ["تفاح", "موز", "برتقال", "عنب"]

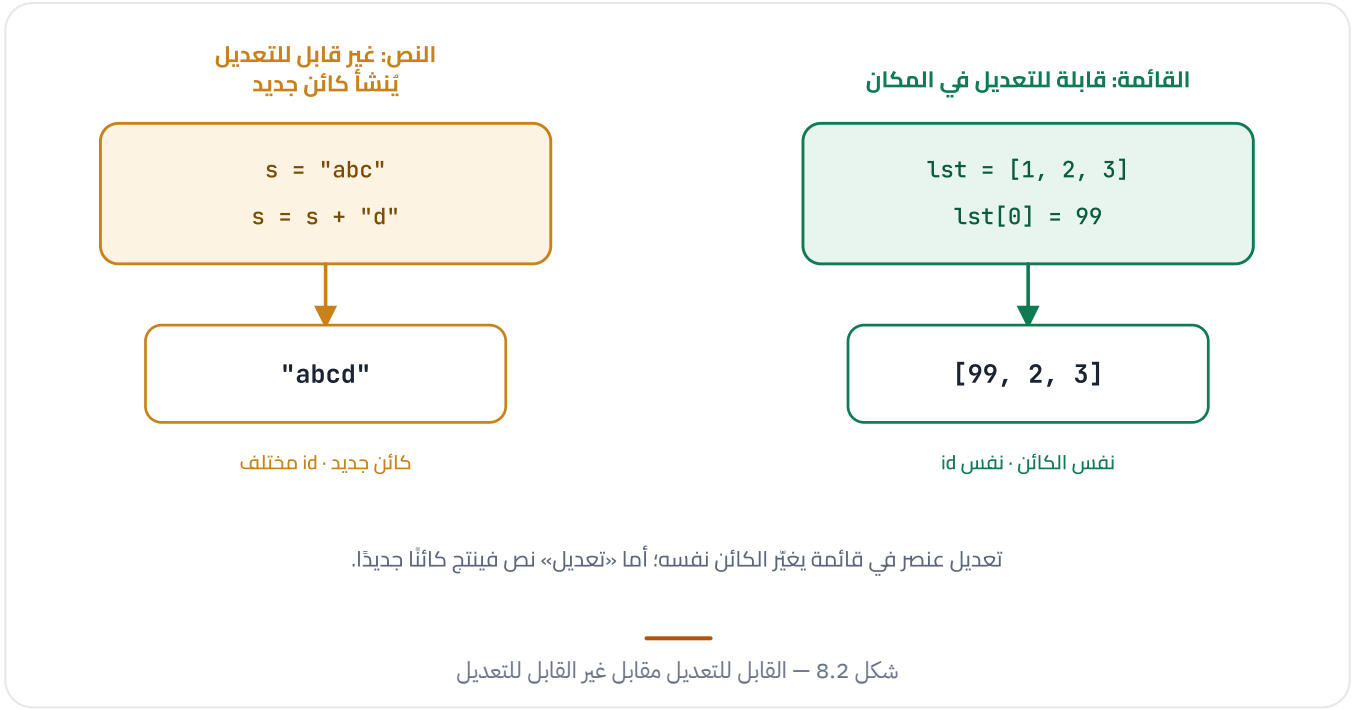
print(fruits[0])    # تفاح  (أول عنصر)
print(fruits[-1])   # عنب  (آخر عنصر)
print(fruits[1:3])  # ['موز', 'برتقال']  (تقطيع - نفس منطق النصوص)
```

🎯 **لاحظ التكامل:** كل ما تعلمته عن الفهرسة والتقطيع في الفصل الرابع (النصوص) ينطبق **حرفيًا** على القوائم. هذا التصميم المتّسق هو أحد أسباب جمال Python.

## ⚠️ تحذير: IndexError لا يزال قائمًا

```python
fruits: list[str] = ["تفاح", "موز"]
print(fruits[5])  # ❌ IndexError: list index out of range
```

# 8.4 الفرق الجوهري: القوائم Mutable (قابلة للتعديل)!




شكل 8.2 — القابل للتعديل مقابل غير القابل للتعديل

## 🔒 أهم فرق بين القوائم والنصوص

تذكر من الفصل الرابع: النصوص **Immutable** (غير قابلة للتعديل المباشر). القوائم **مختلفة تمامًا** - هي **Mutable**:

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]
fruits[0] = "مانجو"  # ✅ يعمل تمامًا! تعديل مباشر
print(fruits)  # ['مانجو', 'موز', 'برتقال']
```

مقارنة بالنصوص:

```python
word: str = "Python"
word[0] = "J"  # ❌ TypeError - النصوص لا تسمح بهذا
```

🎯 **هذا الفرق سيُفسّر سلوكيات مهمة جدًا لاحقًا** - خصوصًا فيما يتعلق بنسخ القوائم (سنتعمق فيه قريبًا في هذا الفصل).

# 8.5 أهم Methods القوائم

## ➕ إضافة عناصر

```python
fruits: list[str] = ["تفاح", "موز"]

# إضافة عنصر واحد في النهاية
fruits.append("برتقال")
print(fruits)  # ['تفاح', 'موز', 'برتقال']

# إدراج عنصر في موضع محدد
fruits.insert(1, "عنب")
print(fruits)  # ['تفاح', 'عنب', 'موز', 'برتقال']

# إضافة عدة عناصر دفعة واحدة
fruits.extend(["مانجو", "أناناس"])
print(fruits)  # ['تفاح', 'عنب', 'موز', 'برتقال', 'مانجو', 'أناناس']
```

⚠ **تحذير شائع:** الخلط بين `.append()` و `.extend()` :

```python
numbers: list[int] = [1, 2, 3]

numbers.append([4, 5])
print(numbers)  # [1, 2, 3, [4, 5]]  ← قائمة متداخلة! ليس هذا المقصود غالبًا

numbers2: list[int] = [1, 2, 3]
numbers2.extend([4, 5])
print(numbers2)  # [1, 2, 3, 4, 5]  ← هذا هو المقصود عادةً
```

## حذف عناصر

```python
fruits: list[str] = ["تفاح", "موز", "برتقال", "موز"]

# حذف بالقيمة (أول ظهور فقط)
fruits.remove("موز")
print(fruits)  # ['تفاح', 'برتقال', 'موز']

# حذف بالفهرس، وإعادة القيمة المحذوفة
removed_item: str = fruits.pop(0)
print(removed_item)  # تفاح
print(fruits)        # ['برتقال', 'موز']

# pop() بدون فهرس يحذف آخر عنصر
last_item: str = fruits.pop()
print(last_item)  # موز

# حذف بالفهرس دون الحاجة للقيمة المحذوفة
del fruits[0]

# إفراغ القائمة بالكامل
fruits.clear()
print(fruits)  # []
```

⚠ **تحذير:** `.remove()` تحذف **أول ظهور فقط** للقيمة، وتُسبب `ValueError` إن لم تكن القيمة موجودة إطلاقًا:

```python
fruits: list[str] = ["تفاح", "موز"]
fruits.remove("مانجو")  # ❌ ValueError: list.remove(x): x not in list
```

## 🔄 الترتيب والعكس

```python
numbers: list[int] = [5, 2, 8, 1, 9]

numbers.sort()
print(numbers)  # [1, 2, 5, 8, 9]  ← عُدّلت القائمة الأصلية مباشرة!

numbers.sort(reverse=True)
print(numbers)  # [9, 8, 5, 2, 1]

numbers.reverse()
print(numbers)  # [1, 2, 5, 8, 9]  ← عكس الترتيب الحالي
```

⚠️ **تحذير حرج:** `.sort()` تُعدّل القائمة **في مكانها مباشرة** وتُعيد `None` (وليس القائمة المُرتَّبة!):

```python
numbers: list[int] = [3, 1, 2]
result = numbers.sort()
print(result)   # None  ← فخ شائع جدًا!
print(numbers)  # [1, 2, 3]  ← هنا التعديل الفعلي
```

## ✅ الحل الاحترافي: دالة `sorted()` (لا تُعدّل الأصل)

```python
numbers: list[int] = [3, 1, 2]
sorted_numbers: list[int] = sorted(numbers)

print(numbers)         # [3, 1, 2]  ← لم تتغيّر!
print(sorted_numbers)  # [1, 2, 3]  ← قائمة جديدة مُرتَّبة
```

🔄 **مقارنة مهمة:** `.sort()` مقابل `sorted()`

| الطريقة | تُعدّل الأصل؟ | تُعيد ماذا؟ |
|---|---|---|
| `list.sort()` | ✅ نعم (in-place) | `None` |
| `sorted(list)` | ❌ لا | قائمة جديدة مُرتَّبة |

✅ **أفضل الممارسات:** استخدم `sorted()` دائمًا إلا إن كنت متأكدًا تمامًا أنك تريد تعديل القائمة الأصلية وتوفير الذاكرة.

### 🔍 البحث والعد

```python
fruits: list[str] = ["عنب" ,"تفاح" ,"موز" ,"تفاح"]

print(fruits.count("تفاح"))   # عدد مرات التكرار ← 2
print(fruits.index("موز"))    # فهرس أول ظهور ← 1
print("عنب" in fruits)        # True
print("مانجو" in fruits)      # False
```

---

## 8.6 القوائم المتداخلة (Nested Lists)

يمكن أن تحتوي القائمة قوائم أخرى بداخلها - مفهوم أساسي للتعامل مع بيانات جدولية (مثل مصفوفة أو شبكة):

```python
matrix: list[list[int]] = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0])      # [1, 2, 3]  ← الصف الأول كاملاً
print(matrix[0][1])   # 2          ← العنصر الثاني في الصف الأول
print(matrix[2][2])   # 9          ← العنصر الأخير
```

### 🎯 مثال عملي: طباعة مصفوفة كاملة

```python
matrix: list[list[int]] = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

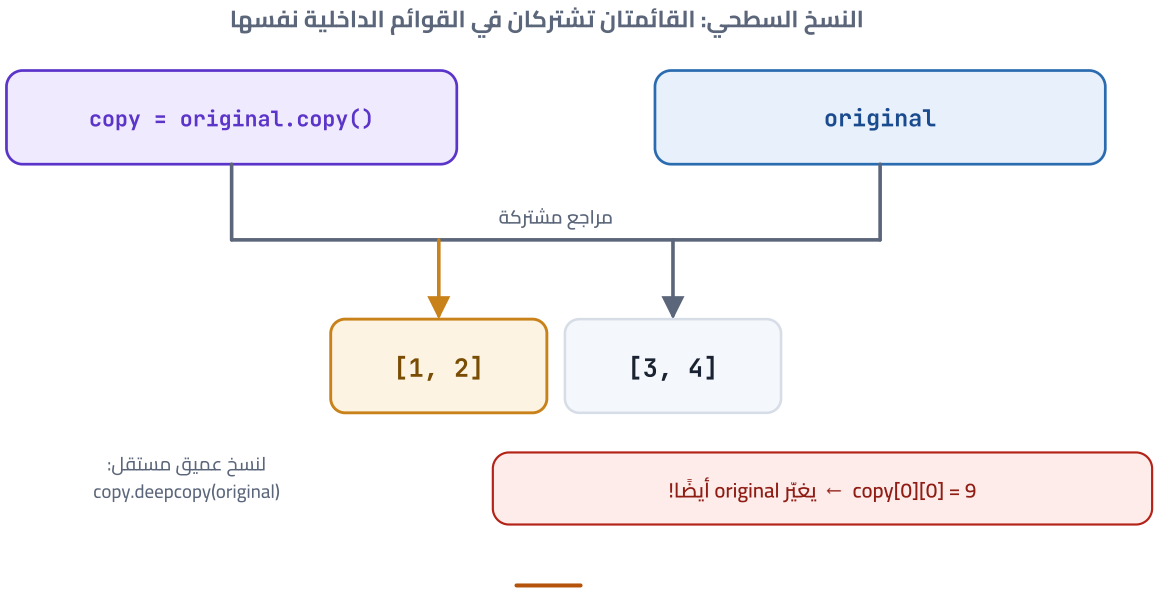
for row in matrix:
    for element in row:
        print(element, end=" ")
    print()  # سطر جديد بعد كل صف
```

**الناتج:**

```
1 2 3
4 5 6
7 8 9
```

---

## 8.7 الفخ الأشهر على الإطلاق: نسخ القوائم (Shallow Copy Trap)




شكل 8.3 — فخ النسخ السطحي: مراجع مشتركة

### 🔥 المشكلة الأكثر إرباكًا لجميع المبتدئين

تذكر من الفصل الثالث: المتغيرات في Python هي «ملصقات» تُشير لكائنات في الذاكرة، وليست «صناديق». مع القوائم (كونها Mutable)، هذا المفهوم يُسبب سلوكًا قد يصدمك تمامًا:

**Python**

```python
original: list[int] = [1, 2, 3]
copy: list[int] = original  # ⚠️ هذا لا ينسخ القائمة!

copy.append(4)

print(original)  # [1, 2, 3, 4]  ← رغم أننا عدّلنا !! تغيّرت أيضًا copy فقط!
print(copy)      # [1, 2, 3, 4]
```

## 😱 لماذا حدث هذا؟

كلا المتغيرين `original` و `copy` يُشيران لنفس القائمة **بالضبط** في الذاكرة - لم يتم إنشاء نسخة جديدة فعليًا، فقط ملصق إضافي يُشير لنفس المكان:

```python
original: list[int] = [1, 2, 3]
copy: list[int] = original

print(id(original))  # 140234567890 :مثلاً
print(id(copy))      # 140234567890 !نفس الرقم بالضبط
```

> 🎯 **هذا بالضبط الفرق الذي حذّرنا منه في الفصل الثالث** بين «الصناديق» و«الملصقات» - والآن ترى تبعاته العملية الحقيقية والمهمة جدًا.

## ✅ الحل الصحيح: إنشاء نسخة فعلية (Shallow Copy)

```python
original: list[int] = [1, 2, 3]

# 3 طرق صحيحة لإنشاء نسخة فعلية مستقلة
copy1: list[int] = original.copy()
copy2: list[int] = original[:]  # تقطيع كامل
copy3: list[int] = list(original)

copy1.append(4)
print(original)  # [1, 2, 3]  ← لم تتأثر! ✅
print(copy1)     # [1, 2, 3, 4]
```

## ⚠️ تحذير متقدم: Shallow Copy لا تكفي دائمًا مع القوائم المتداخلة

```python
original: list[list[int]] = [[1, 2], [3, 4]]
copy: list[list[int]] = original.copy()  # نسخة سطحية فقط!

copy[0].append(99)

print(original)  # [[1, 2, 99], [3, 4]]  ← تأثرت رغم النسخ!
print(copy)      # [[1, 2, 99], [3, 4]]
```

🤔 **لماذا؟** `.copy()` تُنشئ قائمة جديدة، لكنها **لا تنسخ العناصر الداخلية بعمق** - القوائم الداخلية لا تزال نفس الكائنات المُشتركة بين original و copy.

## ✅ الحل الكامل: Deep Copy

```python
import copy

original: list[list[int]] = [[1, 2], [3, 4]]
deep_copy: list[list[int]] = copy.deepcopy(original)

deep_copy[0].append(99)

print(original)   # [[1, 2], [3, 4]]  ← لم تتأثر الآن! ✅
print(deep_copy)  # [[1, 2, 99], [3, 4]]
```

## 🔄 جدول ملخّص: أنواع النسخ

| النوع | الطريقة | يحمي من تأثر الأصل؟ |
|---|---|---|
| لا نسخ (مرجع فقط) | `copy = original` | ❌ أبدًا |
| نسخ سطحي (Shallow) | `.copy()` , `[:]` , `list()` | ✅ للمستوى الأول فقط |
| نسخ عميق (Deep) | `copy.deepcopy()` | ✅ لكل المستويات |

🎯 **قاعدة ذهبية**: إن كانت قائمتك تحتوي فقط أنواعًا بسيطة (أرقام، نصوص)، Shallow Copy كافية تمامًا. إن كانت تحتوي قوائم أو كائنات أخرى متداخلة، استخدم `deepcopy()` .

# List Comprehension 8.8 : الأسلوب الأنيق والـ"Pythonic"




```
[ x * x  for x in nums  if x % 2 == 0 ]
```

الترتيب: النتيجة، ثم المصدر، ثم الشرط — ويُقرأ كحلقة مختصرة.

شكل 8.4 — من الحلقة إلى List Comprehension

## 🤔 المشكلة التقليدية

```python
# الطريقة التقليدية لإنشاء قائمة بمربعات الأرقام
squares: list[int] = []
for number in range(1, 6):
    squares.append(number ** 2)

print(squares)  # [1, 4, 9, 16, 25]
```

## ✨ الحل الأنيق: List Comprehension

```python
squares: list[int] = [number ** 2 for number in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

## 📐 الصيغة العامة

```python
[expression for item in iterable]
```

## 🎯 أمثلة متدرجة الصعوبة

### مثال بسيط:

```python
names: list[str] = ["أحمد" ,"سارة" ,"محمد"]
uppercase_names: list[str] = [name.upper() for name in names]
print(uppercase_names)
```

### مع شرط (Filter):

```python
numbers: list[int] = range(1, 11)
even_numbers: list[int] = [n for n in numbers if n % 2 == 0]
print(even_numbers)  # [2, 4, 6, 8, 10]
```

الصيغة الكاملة مع شرط:

```python
[expression for item in iterable if condition]
```

### مع شرط if/else (مختلف عن الـ filter أعلاه):

```python
numbers: list[int] = range(1, 6)
labels: list[str] = ["زوجي" if n % 2 == 0 else "فردي" for n in numbers]
print(labels)  # ['فردي' ,'زوجي' ,'فردي' ,'زوجي' ,'فردي']
```

⚠️ **تحذير على الصيغة:** لاحظ الفرق الدقيق في الموضع:

```python
# shbr Filter (يُكتب في النهاية)
[x for x in range(10) if x % 2 == 0]

# شرط Ternary (يُكتب قبل for مباشرة)
[x if x % 2 == 0 else -x for x in range(10)]
```

### مع حلقات متداخلة:

```python
matrix: list[list[int]] = [[1, 2], [3, 4], [5, 6]]
flattened: list[int] = [num for row in matrix for num in row]
print(flattened)  # [1, 2, 3, 4, 5, 6]
```

### 🔄 مقارنة الأداء: List Comprehension مقابل الحلقة التقليدية

List comprehension ليست فقط أجمل، بل أيضًا **أسرع تنفيذيًا** غالبًا من الحلقة التقليدية المكافئة، لأن Python تُحسّنها داخليًا بشكل أفضل.

✅ **أفضل الممارسات:** استخدم list comprehension للحالات **البسيطة والواضحة**. إن أصبح التعبير معقدًا جدًا (أكثر من شرط واحد، أو منطق متداخل كثيرًا)، استخدم الحلقة التقليدية لصالح **وضوح القراءة** - القاعدة الذهبية في Python: "Readability counts".

```python
# ❌ list comprehension صعبة القراءة - معقدة جدًا
result = [x**2 if x % 2 == 0 else x**3 for x in range(20) if x > 5 if x < 15]

# ✅ أوضح كحلقة تقليدية في هذه الحالة المعقدة
result = []
for x in range(20):
    if 5 < x < 15:
        if x % 2 == 0:
            result.append(x ** 2)
        else:
            result.append(x ** 3)
```

---

## 8.9 دوال مُدمجة قوية تعمل مع القوائم

```python
numbers: list[int] = [4, 2, 8, 1, 9, 5]

print(len(numbers))    # 6   ← عدد العناصر
print(sum(numbers))    # 29  ← المجموع
print(max(numbers))    # 9   ← أكبر قيمة
print(min(numbers))    # 1   ← أصغر قيمة
```

### 🎯 دالة `zip()` : دمج عدة قوائم معًا

```python
names: list[str] = ["أحمد" ,"سارة" ,"محمد"]
ages: list[int] = [25, 30, 28]

for name, age in zip(names, ages):
    print(f"{name}: {age} سنة")
```

**الناتج:**

```
محمد: 25 سنة
سارة: 30 سنة
أحمد: 28 سنة
```

> 💡 **استخدام عملي شديد الشيوع:** `zip()` مثالية عند التعامل مع بيانات موزّعة على عدة قوائم متوازية (نفس الترتيب والطول).

---

## 8.10 القوائم والحلقات: دمج كل ما تعلمناه

```python
students: list[str] = ["فاطمة" ,"أحمد" ,"سارة" ,"محمد"]
scores: list[int] = [85, 92, 78, 95]

for index, student in enumerate(students):
    score: int = scores[index]
    status: str = "ناجح ✅" if score ⩾ 60 else "راسب ❌"
    print(f"{student}: {score} - {status}")
```

### 🎯 مثال أقوى بـ `zip()` بدلاً من الفهرسة اليدوية

```python
for student, score in zip(students, scores):
    status: str = "ناجح ✅" if score ⩾ 60 else "راسب ❌"
    print(f"{student}: {score} - {status}")
```

✅ **أفضل الممارسات:** استخدم `zip()` بدلاً من الوصول بالفهرس ( `scores[index]` ) عند التكرار المتوازي على عدة قوائم - أوضح وأقل عرضة للأخطاء.

---

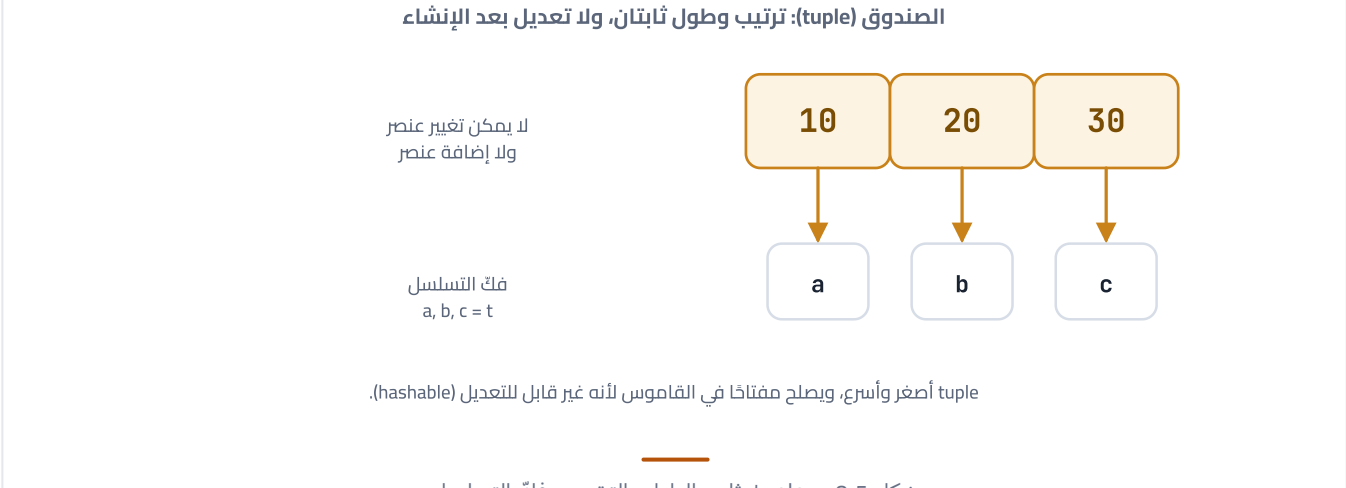
## 8.11 متى تستخدم القوائم؟ (تمهيد لمقارنة أشمل قادمة)

القوائم مثالية عندما:
- تحتاج **ترتيبًا** محددًا للعناصر (الأول، الثاني، إلخ)
- تحتاج **السماح بالتكرار** (نفس القيمة أكثر من مرة)
- تحتاج **تعديل** المحتوى لاحقًا (إضافة/حذف/تغيير)

💡 في الفصل القادم، سنتعرّف على **Sets** و **Dictionaries** - هياكل بيانات أخرى لها استخدامات مختلفة تمامًا، وسنقدّم جدول مقارنة شامل يساعدك على اختيار الهيكل الصحيح لكل مشكلة.

---

## 8.12 Tuples : الأخ «غير القابل للتعديل» للقائمة




شكل 8.5 — tuple: ثابت الطول والترتيب، وفكّ التسلسل

### 🤔 لماذا نتعرّف على Tuples هنا؟

قبل الانتقال للفصل القادم، من المهم التعرّف سريعًا على **Tuple** - هيكل بيانات شبيه جدًا بالقائمة، لكن بفرق جوهري واحد:

```python
coordinates: tuple[int, int] = (10, 20)
colors: tuple[str, ...] = ("أحمر", "أخضر", "أزرق")
```

## 🔒 الفرق الجوهري: Tuples غير قابلة للتعديل (Immutable)

```python
coordinates: tuple[int, int] = (10, 20)
coordinates[0] = 99  # ❌ TypeError: 'tuple' object does not support item assignment
```

## 🤔 لماذا نحتاج Tuple إن كانت القوائم تفعل كل شيء؟

| الخاصية | List | Tuple |
|---|---|---|
| قابلة للتعديل | ✅ نعم | ❌ لا |
| السرعة | أبطأ قليلاً | ⚡ أسرع قليلاً |
| الاستخدام المناسب | بيانات تتغيّر | بيانات ثابتة (كإحداثيات، أو تاريخ ميلاد) |
| يمكن استخدامها كمفتاح Dictionary | ❌ لا | ✅ نعم (سنتعلم هذا قريبًا) |

```python
# استخدام منطقي لـ Tuple: إحداثيات لن تتغيّر أبدًا بهذا الشكل
point: tuple[int, int] = (10, 20)

# استخدام منطقي لـ List: قائمة تسوق قابلة للتعديل باستمرار
shopping_list: list[str] = ["خبز", "حليب"]
shopping_list.append("بيض")  # القوائم تتغيّر - منطقي
```

> 💡 **نصيحة عملية:** استخدم Tuple كلما كانت لديك بيانات تعرف أنها **لن تتغيّر منطقيًا** طوال حياة البرنامج - هذا يُرسل إشارة واضحة لأي قارئ للكود (وأحيانًا يُحسّن الأداء قليلاً).

### ✍️ العمليات الأساسية على Tuples (مشابهة جدًا للقوائم، عدا التعديل)

```python
point: tuple[int, int] = (10, 20)

print(point[0])      # 10  ← الفهرسة تعمل
print(len(point))    # 2   ← len تعمل

x, y = point  # Unpacking - تفكيك القيم مباشرة لمتغيرات!
print(x, y)   # 10 20
```

🎯 **ميزة أنيقة جدًا - Tuple Unpacking**: هذا النمط مفيد جدًا وشائع الاستخدام في Python الاحترافية، وسنراه بكثرة لاحقًا مع الدوال التي تُعيد قيمًا متعددة.

---

## 8.13 أخطاء شائعة ونصائح احترافية

### ⚠️ الخطأ 1: الاعتقاد أن `=` تنسخ القائمة

```python
list_a: list[int] = [1, 2, 3]
list_b: list[int] = list_a  # ⚠️ مرجع فقط، وليس نسخة!
```

### ⚠️ الخطأ 2: نسيان أن `.sort()` تُعيد None

```python
numbers: list[int] = [3, 1, 2]
sorted_list = numbers.sort()
print(sorted_list)  # None ← خطأ شائع جدًا!
```

### ⚠️ الخطأ 3: تعديل قائمة أثناء التكرار عليها مباشرة

```python
numbers: list[int] = [1, 2, 3, 4, 5]

# ❌ خطر! نتائج غير متوقعة
for number in numbers:
    if number % 2 == 0:
        numbers.remove(number)  # تعديل القائمة أثناء المرور عليها!

print(numbers)  # [1, 3, 5] لكن بطريقة غير موثوقة ومحتملة الخطأ
```

⚠️ **تحذير حرج:** تعديل قائمة (بالإضافة أو الحذف) أثناء المرور عليها بـ `for` يُسبب سلوكًا غير متوقع، لأن Python تفقد «مكانها» في التتبّع. **الحل الآمن:**

```python
# ✅ الحل الصحيح: التكرار على نسخة، والتعديل على الأصل
numbers: list[int] = [1, 2, 3, 4, 5]

for number in numbers.copy():  # أو numbers[:]
    if number % 2 == 0:
        numbers.remove(number)

print(numbers)  # [1, 3, 5] ← آمن وصحيح
```

أو الأفضل - استخدام list comprehension لإنشاء قائمة جديدة بدلاً من التعديل المباشر:

```python
numbers: list[int] = [1, 2, 3, 4, 5]
odd_numbers: list[int] = [n for n in numbers if n % 2 != 0]
print(odd_numbers)  # [1, 3, 5]
```

---

## 8.14 جدول مرجعي شامل للفصل

| العملية | الكود | ملاحظة |
|---|---|---|
| إنشاء | `[1, 2, 3]` | - |

| العملية | الكود | ملاحظة |
|---|---|---|
| إضافة في النهاية | `.append(x)` | - |
| إضافة عدة عناصر | `.extend([x, y])` | - |
| إدراج في موضع | `.insert(i, x)` | - |
| حذف بالقيمة | `.remove(x)` | أول ظهور فقط |
| حذف بالفهرس | `.pop(i)` | تُعيد القيمة المحذوفة |
| ترتيب (يُعدّل الأصل) | `.sort()` | تُعيد None |
| ترتيب (نسخة جديدة) | `sorted(list)` | لا تُعدّل الأصل |
| نسخ سطحي | `.copy()` أو `[:]` | مستوى واحد فقط |
| نسخ عميق | `copy.deepcopy()` | كل المستويات |
| List Comprehension | `[x for x in items]` | - |
| دمج قوائم متوازية | `zip(list1, list2)` | - |

---

## 8.15 تمارين عملية للفصل

### 🟢 تمرين 1 (مستوى مبتدئ)

أنشئ قائمة بأسماء 5 مدن تحب زيارتها، ثم: أضف مدينة جديدة في النهاية، أدرج مدينة أخرى في البداية، احذف إحدى المدن بالاسم، واطبع القائمة مُرتَّبة أبجديًا.

### 🟡 تمرين 2 (مستوى متوسط)

لديك قائمة بدرجات طلاب: `scores = [45, 78, 92, 60, 55, 88, 70]`. استخدم list comprehension لإنشاء قائمة جديدة تحتوي فقط الدرجات الناجحة (60 فأكثر)، ثم احسب متوسط الدرجات الناجحة فقط.

### 🔴 تمرين 3 (تحدي)

اكتب برنامجًا يأخذ قائمتين من الأرقام، ويُعيد قائمة ثالثة تحتوي **فقط** العناصر المشتركة بينهما (الموجودة في كلتا القائمتين)، دون تكرار، باستخدام list comprehension فقط (بدون استخدام `set` التي سنتعلمها لاحقًا).

---

# 8.16 مشروع تطبيقي: «نظام إدارة قائمة مهام (To-Do List)»

## 📋 المهمة

أنشئ ملفًا باسم `todo_list.py` يبني نظام قائمة مهام تفاعلي كامل من الطرفية:

**المتطلبات:**

1. قائمة فارغة لحفظ المهام
2. قائمة بخيارات (عرض باستمرار): إضافة مهمة، عرض كل المهام، حذف مهمة، إنهاء البرنامج
3. استخدام `while True` مع `break` للتحكم بدورة حياة البرنامج
4. استخدام `enumerate()` لعرض المهام مع أرقامها
5. معالجة الحالات الاستثنائية (حذف مهمة برقم غير موجود)

## 💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python todo_list.py

```
"""نظام إدارة قائمة مهام تفاعلي بسيط."""

tasks: list[str] = []

def display_menu() -> None:
    print("\n" + "=" * 35)
    print("📋  قائمة المهام")
    print("=" * 35)
    print("1. إضافة مهمة جديدة")
    print("2. عرض كل المهام")
    print("3. حذف مهمة")
    print("4. إنهاء البرنامج")
    print("=" * 35)

while True:
    display_menu()
    choice: str = input("اختر رقمًا: ").strip()

    if choice == "1":
        new_task: str = input("اكتب المهمة الجديدة: ").strip()
        if new_task:
            tasks.append(new_task)
            print(f"✅ تمت إضافة: {new_task}")
        else:
            print("⚠️ لا يمكن إضافة مهمة فارغة!")

    elif choice == "2":
        if not tasks:
            print("📭 لا توجد مهام حاليًا!")
        else:
            print("\n📌 مهامك الحالية:")
            for index, task in enumerate(tasks, start=1):
                print(f"  {index}. {task}")

    elif choice == "3":
        if not tasks:
            print("📭 لا توجد مهام لحذفها!")
        else:
            for index, task in enumerate(tasks, start=1):
                print(f"  {index}. {task}")

            task_number: str = input("أدخل رقم المهمة المراد حذفها: ").strip()
```

```
            if task_number.isdigit() and 1 <= int(task_number) <= len(tasks):
                removed: str = tasks.pop(int(task_number) - 1)
                print(f"🗑️ تم حذف: {removed}")
            else:
                print("⚠️ رقم غير صالح!")

        elif choice == "4":
            print("👋 إلى اللقاء!")
            break

        else:
            print("⚠️ اختيار غير صحيح، حاول مرة أخرى")
```

**ملاحظة على الحل:** لاحظ استخدامنا لأول `def` (تعريف دالة) في هذا الكتاب - `display_menu()`. لا تقلق إن بدت غامضة؛ سنتعمّق في الدوال بالتفصيل الكامل في الفصل العاشر. هذا تشويق بسيط يُظهر كيف ستُنظّم كودك بشكل أفضل قريبًا.

---

## 8.17 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين القوائم (List) والنصوص (String) من حيث قابلية التعديل؟
2. لماذا يُسبب `copy_list = original_list` مشاكل غير متوقعة عند تعديل إحداهما؟
3. ما الفرق بين `.sort()` و `sorted()`؟
4. متى تحتاج `copy.deepcopy()` بدلاً من `.copy()` العادية؟
5. ما الفرق الجوهري بين List و Tuple؟

✅ **الإجابات النموذجية**

1. القوائم Mutable (قابلة للتعديل المباشر بعد الإنشاء)، بينما النصوص Immutable (أي عملية «تعديل» تُنشئ نصًا جديدًا بالكامل بدلاً من تغيير الأصل).
2. لأن عملية الإسناد `=` لا تنسخ القائمة فعليًا، بل تجعل المتغير الجديد «ملصقًا» إضافيًا يُشير لنفس القائمة في الذاكرة - أي تعديل عبر أي من الاسمين يؤثر على نفس الكائن المشترك.
3. `.sort()` method تُعدّل القائمة الأصلية مباشرة (in-place) وتُعيد `None`، بينما `sorted()` دالة مستقلة تُعيد قائمة **جديدة** مُرتَّبة دون التأثير على الأصل.
4. عندما تحتوي القائمة على عناصر متداخلة قابلة للتعديل (كقوائم داخل قوائم)، لأن `.copy()` العادية تنسخ فقط المستوى الأول (Shallow)، بينما العناصر الداخلية تبقى مُشتركة مع الأصل.
5. List قابلة للتعديل (Mutable) بعد الإنشاء، بينما Tuple غير قابلة للتعديل (Immutable) - تُستخدم Tuple عادة للبيانات التي من المنطقي أن تبقى ثابتة طوال حياة البرنامج.

# 8.18 📌 ملخص الفصل (Cheat Sheet)

```python
# الإنشاء
my_list: list[int] = [1, 2, 3]
my_tuple: tuple[int, ...] = (1, 2, 3)

# الإضافة/الحذف
my_list.append(4)
my_list.insert(0, 99)
my_list.remove(2)        # بالقيمة
my_list.pop(0)           # بالفهرس، تُعيد القيمة

# الترتيب
my_list.sort()           # يُعدّل الأصل، يُعيد None
sorted(my_list)          # نسخة جديدة مُرتَّبة

# النسخ (مهم جدًا!)
copy = original          # ⚠️ مرجع فقط، ليس نسخة!
copy = original.copy()   # نسخة سطحية
import copy as cp
deep = cp.deepcopy(original)  # نسخة عميقة كاملة

# List Comprehension
[x**2 for x in range(5)]
[x for x in range(10) if x % 2 == 0]

# دوال مفيدة
len(my_list), sum(my_list), max(my_list), min(my_list)
zip(list1, list2)
enumerate(my_list, start=1)
```

## ⚠️ تذكّر دائمًا

- القوائم Mutable، النصوص والـ Tuples ليست كذلك
- `=` لا تنسخ القوائم - استخدم `.copy()` أو `deepcopy()`
- `.sort()` تُعدّل وتُعيد `None` - `sorted()` تُعيد نسخة جديدة
- لا تُعدّل قائمة أثناء التكرار المباشر عليها

## 🎯 ماذا بعد؟

القوائم رائعة، لكنها ليست الحل الأمثل لكل مشكلة. ماذا لو أردت التأكد من عدم تكرار أي عنصر تلقائيًا؟ أو تخزين بيانات بصيغة «مفتاح → قيمة» (مثل اسم ورقم هاتف)؟ في **الفصل التاسع**، سنتعرّف على **Dictionaries** و **Sets** - هيكلان قويان جدًا، مع جدول مقارنة شامل يوضح متى تستخدم كل هيكل بيانات تعلمناه حتى الآن.

فصل 09 / 26 · الجزء الثاني

# 09 القواميس والمجموعات (Dict & Set)

8 دقيقة قراءة | 9 قسمًا | 32 مثالًا | 1,554 كلمة

في هذا الفصل

9.1 لماذا نحتاج هيكل بيانات جديد؟
9.2 القواميس (Dictionaries)
9.3 المجموعات (Sets)
9.4 جدول المقارنة الشامل: متى تستخدم ماذا؟
9.5 مشروع تطبيقي: نظام إدارة مخزون بسيط
9.6 تمارين
9.7 اختبر فهمك (Quiz)
9.8 ملخص الفصل (Cheat Sheet)
9.9 ماذا بعد؟

## 9.1 لماذا نحتاج هيكل بيانات جديد؟

تخيّل أنك تريد تخزين بيانات طالب: اسمه، عمره، درجاته، تخصصه. باستخدام ما تعلمناه في الفصل الثامن، قد تكتب:

```python
student = ["Ahmed", 22, "Computer Science", 85.5]
```

لكن هذا الحل يحمل مشكلة خطيرة: **كيف تعرف أن** `student[0]` **هو الاسم و** `student[1]` **هو العمر؟** عليك أن تتذكر الترتيب! إذا أراد زميلك في العمل قراءة هذا الكود بعد ستة أشهر، أو إذا أضفت حقلاً جديداً في المنتصف، سينهار كل شيء.

ما نحتاجه هو طريقة لتخزين البيانات **بالاسم** وليس بالموقع. هنا يأتي دور **القاموس (Dictionary)**:

```python
student: dict[str, object] = {
    "name": "Ahmed",
    "age": 22,
    "major": "Computer Science",
    "gpa": 85.5
}

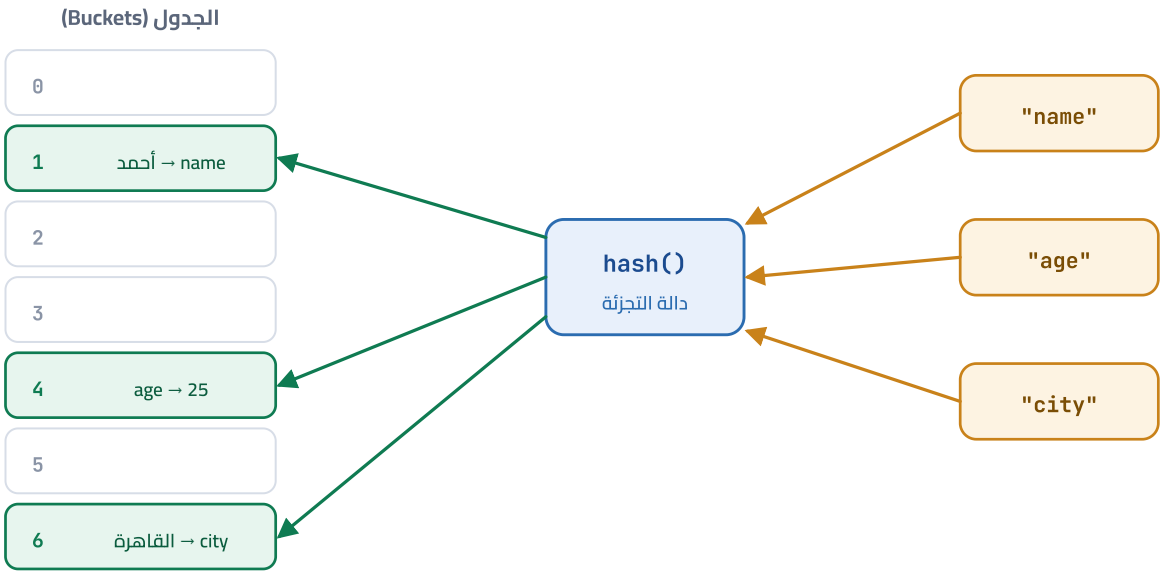
print(student["name"])  # Ahmed - واضح ومباشر!
```

أما المشكلة الثانية التي سنحلها في هذا الفصل: كيف تتأكد أن قائمة أسماء المستخدمين لا تحتوي على تكرار؟ كيف تجد العناصر المشتركة بين قائمتين بسرعة؟ هنا يأتي دور **المجموعة (Set)**.

بنهاية هذا الفصل، ستمتلك الصورة الكاملة لكل هياكل البيانات الأساسية في Python، وستعرف متى تستخدم كل واحد منها بثقة تامة.

---

## 9.2 القواميس (Dictionaries)




شكل 9.1 — كيف يصل القاموس للقيمة عبر التجزئة

### ما هو القاموس؟

القاموس هو مجموعة من أزواج **«مفتاح: قيمة»** (Key: Value). تماماً كقاموس اللغة الحقيقي: تبحث عن **كلمة** (المفتاح) لتجد **معناها** (القيمة).

```python
# قاموس حقيقي
# "Serendipity" → "اكتشاف شيء جميل بالصدفة"

ages: dict[str, int] = {
    "Sara": 25,
    "Omar": 30,
    "Lina": 28
}

print(ages["Sara"])  # 25
```

## خصائص القاموس الجوهرية

| الخاصية | الوصف |
|---|---|
| **غير مرتب منطقياً** لكنه **يحفظ ترتيب الإدخال** | منذ Python 3.7، القواميس تحافظ على ترتيب إضافة العناصر (لكن لا تعتمد عليه منطقياً كترتيب «أول/آخر» بمعنى الفرز) |
| **قابل للتعديل (Mutable)** | يمكنك إضافة/حذف/تعديل العناصر بعد الإنشاء |
| **المفاتيح فريدة** | لا يمكن تكرار نفس المفتاح؛ أي قيمة جديدة لمفتاح موجود تستبدل القديمة |
| **المفاتيح يجب أن تكون Immutable** | str, int, float, tuple مسموحة كمفاتيح؛ list أو dict ممنوعة |
| **القيم يمكن أن تكون أي شيء** | نصوص، أرقام، قوائم، حتى قواميس أخرى |

## إنشاء القواميس

```python
# الطريقة الأولى: الأقواس المعقوفة
empty_dict: dict = {}
person: dict[str, str] = {"name": "Khaled", "city": "Riyadh"}

# الطريقة الثانية: دالة dict()
person2 = dict(name="Noor", city="Cairo")  # بدون علامات تنصيص للمفاتيح هنا!

# الطريقة الثالثة: من قائمة أزواج (tuples)
pairs = [("a", 1), ("b", 2)]
person3 = dict(pairs)
print(person3)  # {'a': 1, 'b': 2}

# قاموس متداخل (Nested Dictionary) - شائع جداً في البيانات الحقيقية
company: dict[str, dict] = {
    "employee_1": {"name": "Sami", "salary": 8000},
    "employee_2": {"name": "Huda", "salary": 9500}
}
print(company["employee_1"]["name"])  # Sami
```

**نصيحة الخبراء:** استخدم `dict[str, int]` كـ Type Hint عندما تعرف أنواع المفاتيح والقيم. إذا كانت القيم متنوعة الأنواع (كما في مثال `student` أعلاه)، استخدم `dict[str, object]` أو الأفضل: استخدم **Dataclass** (سنتعلمها لاحقاً في الفصل 13) بدلاً من قاموس بقيم مختلطة.

## الوصول للقيم والتعامل مع المفاتيح المفقودة

```python
student = {"name": "Ahmed", "age": 22}

# الطريقة الأولى: الأقواس المربعة - تُطلق خطأ إذا المفتاح غير موجود
print(student["name"])  # Ahmed
# print(student["grade"])  # KeyError: 'grade' 💥

# الطريقة الثانية: get() - آمنة، تُعيد None إذا لم يوجد المفتاح
print(student.get("grade"))        # None
print(student.get("grade", "N/A"))  # N/A - قيمة افتراضية مخصصة
```

**تحذير شائع:** المبتدئون يستخدمون `student["key"]` دائماً، فتنهار برامجهم فجأة بـ `KeyError` عند أول بيانات غير متوقعة. **القاعدة الذهبية:** إذا لم تكن متأكداً 100% أن المفتاح موجود، استخدم `.get()` دائماً.

Python

```
# التحقق من وجود مفتاح قبل استخدامه
if "grade" in student:
    print(student["grade"])
else:
    print("لا توجد درجة مسجلة")

# in تبحث في المفاتيح فقط، وليس القيم!
print("name" in student)   # True
print("Ahmed" in student)  # False (هذا بحث في القيم، وهو غير مدعوم مباشرة بـ in)
```

## إضافة وتعديل وحذف العناصر

Python

```
student = {"name": "Ahmed", "age": 22}

# الإضافة والتعديل بنفس الصيغة
student["age"] = 23           # تعديل (المفتاح موجود)
student["major"] = "AI"       # إضافة (المفتاح جديد)
print(student)
# {'name': 'Ahmed', 'age': 23, 'major': 'AI'}

# الحذف بطريقة del - تطلق خطأ إذا المفتاح غير موجود
del student["age"]

# الحذف بطريقة pop() - آمنة، وتُعيد القيمة المحذوفة
major = student.pop("major")
print(major)  # AI

# pop مع قيمة افتراضية لتجنب الخطأ
removed = student.pop("not_exist", "لا يوجد")
print(removed)  # لا يوجد

# حذف كل العناصر
student.clear()
print(student)  # {}
```

## التكرار على القواميس (Looping)

هذه من أهم المهارات العملية:

Python

```python
scores = {"Sara": 90, "Omar": 75, "Lina": 88}

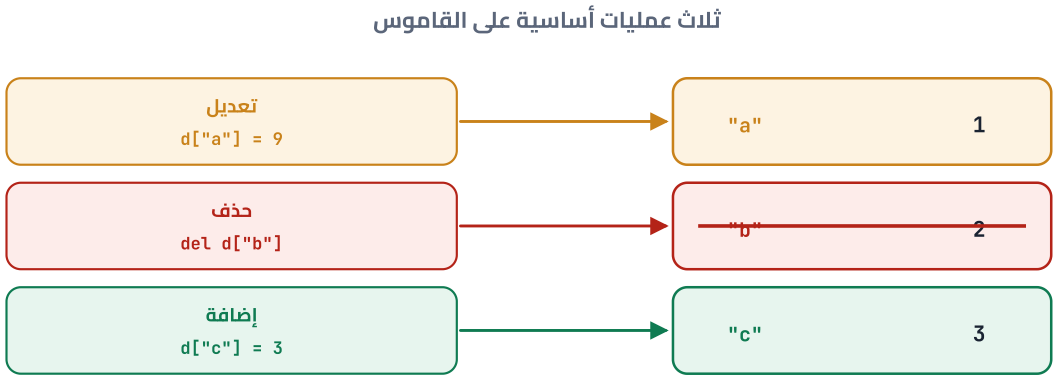
# التكرار على المفاتيح فقط (الافتراضي)
for name in scores:
    print(name)

# التكرار على القيم فقط
for score in scores.values():
    print(score)

# التكرار على المفاتيح والقيم معاً - الأكثر استخداماً عملياً
for name, score in scores.items():
    print(f"{name}: {score}")
```

**مقارنة:** في JavaScript تكتب `for (const [key, value] of Object.entries(obj))`، وفي Python الطريقة أبسط وأوضح: `for key, value in dict.items()`. هذا أحد الأسباب التي تجعل Python محبوبة في تعليم البرمجة.

## أهم Methods القاموس - جدول مرجعي شامل




شكل 9.2 — إضافة وتعديل وحذف في القاموس

| Method | الوصف | مثال |
|---|---|---|
| `.get(key, default)` | إرجاع القيمة أو قيمة افتراضية | `d.get("x", 0)` |
| `.keys()` | إرجاع كل المفاتيح | `d.keys()` |
| `.values()` | إرجاع كل القيم | `d.values()` |
| `.items()` | إرجاع أزواج (مفتاح, قيمة) | `d.items()` |
| `.pop(key, default)` | حذف وإرجاع القيمة | `d.pop("x")` |
| `.update(other_dict)` | دمج قاموس آخر (الكتابة فوق المشترك) | `d.update({"y": 2})` |
| `.setdefault(key, default)` | إرجاع القيمة، وإنشاؤها إن لم توجد | `d.setdefault("x", [])` |
| `.copy()` | نسخة سطحية | `d.copy()` |
| `.clear()` | حذف كل العناصر | `d.clear()` |

Python

```python
# .update() - دمج قاموسين
defaults = {"theme": "dark", "lang": "ar", "font_size": 14}
user_settings = {"lang": "en"}

defaults.update(user_settings)
print(defaults)
# {'theme': 'dark', 'lang': 'en', 'font_size': 14}

# .setdefault() - مفيدة جداً لتجميع البيانات
word_groups: dict[str, list[str]] = {}
words = ["apple", "ant", "banana", "bee", "avocado"]

for word in words:
    first_letter = word[0]
    word_groups.setdefault(first_letter, []).append(word)

print(word_groups)
# {'a': ['apple', 'ant', 'avocado'], 'b': ['banana', 'bee']}
```

**نصيحة الخبراء:** النمط أعلاه (`setdefault` لتجميع عناصر في قوائم فرعية) نمط شائع جداً في معالجة البيانات. هناك بديل أنظف وأسرع له اسم `defaultdict` من مكتبة `collections`:

```python
from collections import defaultdict

word_groups = defaultdict(list)
for word in words:
    word_groups[word[0]].append(word)  # لا حاجة لـ setdefault!

print(dict(word_groups))
```

## Dictionary Comprehension

تماماً مثل List Comprehension، لكن لإنشاء قواميس:

```python
# الصيغة: {key_expr: value_expr for item in iterable}

# مثال 1: تربيع الأرقام
numbers = [1, 2, 3, 4, 5]
squares = {n: n**2 for n in numbers}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# مثال 2: مع شرط
even_squares = {n: n**2 for n in numbers if n % 2 == 0}
print(even_squares)  # {2: 4, 4: 16}

# مثال 3: عكس قاموس (قلب المفاتيح والقيم)
original = {"a": 1, "b": 2, "c": 3}
reversed_dict = {value: key for key, value in original.items()}
print(reversed_dict)  # {1: 'a', 2: 'b', 3: 'c'}

# مثال 4: تحويل قائمتين متوازيتين إلى قاموس باستخدام zip
names = ["Sara", "Omar", "Lina"]
scores = [90, 75, 88]
result = {name: score for name, score in zip(names, scores)}
print(result)  # {'Sara': 90, 'Omar': 75, 'Lina': 88}
```

⚠ **خطأ شائع**: استخدام قيمة قابلة للتعديل (mutable) كمفتاح:

```python
# d = {[1, 2]: "value"}  # TypeError: unhashable type: 'list'
d = {(1, 2): "value"}  # صحيح! tuple غير قابلة للتعديل، لذا مسموحة كمفتاح
```

السبب التقني: المفاتيح يجب أن تكون **Hashable** (قابلة للتحويل إلى قيمة hash ثابتة)، والقيم القابلة للتعديل (list, dict, set) لا تملك hash ثابتاً لأن محتواها قد يتغير.

---

## 9.3 المجموعات (Sets)




شكل 9.3 — اتحاد وتقاطع وفرق المجموعات

### ما هي المجموعة؟

المجموعة هي تجميعة من عناصر **فريدة** (بدون تكرار) **وغير مرتبة**. فكّر بها كصندوق تضع فيه أشياء، لكن لا يمكن أن يحتوي على نفس الشيء مرتين، ولا يهمك ترتيبه.

```python
fruits: set[str] = {"apple", "banana", "apple", "orange"}
print(fruits)
# {'apple', 'banana', 'orange'}  -- لاحظ التكرار اختفى تلقائياً!
```

هذه هي الاستخدام الأول والأشهر للمجموعات: **إزالة التكرار بسهولة**.

```python
numbers = [1, 2, 2, 3, 4, 4, 4, 5]
unique_numbers = list(set(numbers))
print(unique_numbers)  # [1, 2, 3, 4, 5] (الترتيب قد يختلف)
```

⚠ **تحذير:** المجموعات **غير مرتبة**، فلا تعتمد على ترتيب معين عند تحويل قائمة إلى set ثم إعادتها إلى list. إذا احتجت الحفاظ على الترتيب مع إزالة التكرار، استخدم:

```python
numbers = [3, 1, 2, 3, 1, 4]
unique_ordered = list(dict.fromkeys(numbers))
print(unique_ordered)  # [3, 1, 2, 4] - الترتيب محفوظ!
```

## إنشاء المجموعات

```python
# الأقواس المعقوفة (بدون نقطتين - هذا ما يميزها عن dict)
s1: set[int] = {1, 2, 3}

# دالة set() - ضرورية للمجموعة الفارغة!
s2 = set()

# ⚠ تحذير حرج
empty = {}         # هذا قاموس فارغ وليس مجموعة فارغة!
print(type(empty))  # <class 'dict'>

# set() من قائمة
s3 = set([1, 2, 2, 3])
print(s3)  # {1, 2, 3}
```

⚠ **خطأ شائع جداً عند المبتدئين:** كتابة `{}` معتقدين أنها مجموعة فارغة، بينما Python تفسرها كقاموس فارغ دائماً. **لإنشاء مجموعة فارغة استخدم `set()` فقط.**

## العمليات الأساسية على المجموعات

```python
colors: set[str] = {"red", "green", "blue"}

# إضافة عنصر واحد
colors.add("yellow")

# إضافة عدة عناصر
colors.update(["purple", "pink"])

# حذف عنصر - يطلق خطأ إذا غير موجود
colors.remove("red")

# حذف آمن - لا يطلق خطأ
colors.discard("not_exist")  # لا شيء يحدث

# حذف عنصر عشوائي وإرجاعه
item = colors.pop()

# التحقق من وجود عنصر - سريع جداً (انظر الأداء أدناه)
print("blue" in colors)
```

## العمليات الرياضية على المجموعات (القوة الحقيقية)

هنا يكمن السر الحقيقي لأهمية المجموعات: **عمليات نظرية المجموعات الرياضية**.

**Python**

```
python_devs = {"Ahmed", "Sara", "Omar", "Lina"}
js_devs = {"Omar", "Lina", "Khaled", "Noor"}

# الاتحاد (Union): كل العناصر من الاثنين بدون تكرار
all_devs = python_devs | js_devs
# أو: python_devs.union(js_devs)
print(all_devs)
# {'Ahmed', 'Sara', 'Omar', 'Lina', 'Khaled', 'Noor'}

# التقاطع (Intersection): العناصر المشتركة فقط
both_devs = python_devs & js_devs
# أو: python_devs.intersection(js_devs)
print(both_devs)
# {'Omar', 'Lina'}

# الفرق (Difference): في الأولى وليس في الثانية
only_python = python_devs - js_devs
# أو: python_devs.difference(js_devs)
print(only_python)
# {'Ahmed', 'Sara'}

# الفرق التماثلي (Symmetric Difference): في أحدهما فقط وليس الاثنين
exclusive = python_devs ^ js_devs
# أو: python_devs.symmetric_difference(js_devs)
print(exclusive)
# {'Ahmed', 'Sara', 'Khaled', 'Noor'}
```

**تطبيق عملي واقعي**: تخيل نظام توصيات في موقع تسوق إلكتروني:

**Python**

```
user_a_purchases = {"laptop", "mouse", "keyboard"}
user_b_purchases = {"mouse", "keyboard", "monitor", "webcam"}

# ماذا اشترى B ولم يشتره A؟ (توصية محتملة لـ A)
recommendations_for_a = user_b_purchases - user_a_purchases
print(recommendations_for_a)  # {'monitor', 'webcam'}
```

```python
# فحص العلاقات بين المجموعات
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}

print(a.issubset(b))      # True -- a مجموعة جزئية من b
print(b.issuperset(a))    # True -- b تحتوي كل عناصر a
print(a.isdisjoint({9, 10}))  # True -- لا يوجد أي عنصر مشترك
```

## لماذا المجموعات سريعة جداً؟

**مقارنة الأداء - نظرة تحت غطاء المحرك**

| العملية | List | Set |
|---|---|---|
| البحث عن عنصر ( `x in collection` ) | O(n) - يفحص كل عنصر تباعاً | O(1) - وصول مباشر تقريباً |
| إضافة عنصر | O(1) في الآخر | O(1) |
| إزالة التكرار | تحتاج كود يدوي | تلقائي بالتعريف |

```python
import time

big_list = list(range(1_000_000))
big_set = set(range(1_000_000))

start = time.perf_counter()
999_999 in big_list
print(f"List: {time.perf_counter() - start:.6f} ثانية")

start = time.perf_counter()
999_999 in big_set
print(f"Set: {time.perf_counter() - start:.6f} ثانية")

# النتيجة التقريبية: List أبطأ بآلاف المرات!
```

السبب التقني: القوائم تبحث عنصراً تلو الآخر (Linear Search)، بينما المجموعات والقواميس تستخدم بنية بيانات تسمى **Hash Table** تحسب «عنوان» كل عنصر مباشرة، فتصل إليه فوراً تقريباً بغض النظر عن حجم المجموعة.

**نصيحة الخبراء:** إذا كان كودك يحتوي على سطر مثل `if x in my_list:` داخل حلقة تتكرر آلاف المرات، وكانت `my_list` كبيرة، **حوّلها إلى set أولاً**:

```python
# بطيء مع قوائم كبيرة
allowed_ids = [1, 5, 10, 15, ..., 50000]  # قائمة ضخمة
for user_id in incoming_ids:
    if user_id in allowed_ids:  # بحث خطي بطيء كل مرة!
        process(user_id)

# سريع جداً
allowed_ids_set = set(allowed_ids)  # تحويل مرة واحدة
for user_id in incoming_ids:
    if user_id in allowed_ids_set:  # بحث فوري!
        process(user_id)
```

### Set Comprehension

```python
numbers = [1, 2, 2, 3, 4, 4, 5]
squared_unique = {n**2 for n in numbers}
print(squared_unique)  # {1, 4, 9, 16, 25}
```

### Frozenset: المجموعة غير القابلة للتعديل

```python
# frozenset مثل set لكنها immutable - يمكن استخدامها كمفتاح قاموس!
frozen = frozenset([1, 2, 3])
# frozen.add(4)  # AttributeError - لا توجد طريقة للتعديل

cache: dict[frozenset, str] = {
    frozenset([1, 2]): "combo A",
    frozenset([3, 4]): "combo B"
}
```

---

## 9.4 جدول المقارنة الشامل: متى تستخدم ماذا؟

هذا هو الجدول الذي وعدناك به في نهاية الفصل السابق. ادرسه جيداً، وارجع إليه كلما ترددت في اختيار هيكل البيانات المناسب.

| الخاصية | List `[]` | Tuple `()` | Dict `{k: v}` | Set |
|---|---|---|---|---|
| **قابل للتعديل؟** | ✅ نعم | ❌ لا | ✅ نعم | ✅ |
| **مرتب (يحفظ الترتيب)؟** | ✅ نعم | ✅ نعم | ✅ نعم (منذ 3.7) | ❌ |
| **يسمح بالتكرار؟** | ✅ نعم | ✅ نعم | المفاتيح: ❌ / القيم: ✅ | ❌ |
| **الوصول للعنصر** | بالموقع `[0]` | بالموقع `[0]` | بالمفتاح `["key"]` | لا يـ |
| **سرعة البحث `in`** | بطيء O(n) | بطيء O(n) | سريع جداً O(1) | سرـ |
| **الاستخدام المثالي** | تسلسل عناصر قد يتغير (قائمة مهام، نتائج) | بيانات ثابتة لا تتغير (إحداثيات، تاريخ ميلاد) | بيانات مرتبطة بأسماء/معرّفات (سجل مستخدم، إعدادات) | ضـ مجـ |
| **مثال واقعي** | `cart_items = ["pen", "book"]` | `point = (3, 4)` | `user = {"name": "Ali"}` | `'}` |
| **Comprehension** | `[x for x in ...]` | غير مباشر (generator) | `{k: v for ...}` | `}` |

## 🧭 خريطة قرار سريعة

```
هل البيانات ثابتة لن تتغير أبداً؟
├── نعم → Tuple
└── لا
    ├── هل أحتاج الوصول بالاسم/المفتاح وليس بالرقم؟
    │   └── نعم → Dict
    └── لا
        ├── هل يهمني منع التكرار أو عمل عمليات رياضية (تقاطع/اتحاد)؟
        │   └── نعم → Set
        └── لا → List (الخيار الافتراضي الأكثر مرونة)
```

> **مقارنة مع لغات أخرى:** في JavaScript، `Object` يقابل `dict`، و `Array` يقابل `list`، و `Set` موجود بنفس الاسم والمفهوم تقريباً. في Java، `HashMap` يقابل `dict`، و `HashSet` يقابل `set`. المفاهيم عالمية؛ فقط الأسماء والصياغة تختلف.

---

## 9.5 مشروع تطبيقي: نظام إدارة مخزون بسيط

سنبني نظاماً لإدارة مخزون متجر صغير، يستخدم القواميس للمنتجات، والمجموعات لتتبع الفئات الفريدة.

**المتطلبات:**

1. تخزين المنتجات: الاسم، السعر، الكمية، الفئة
2. إضافة منتج جديد
3. تحديث كمية منتج (بيع/شراء)
4. عرض كل المنتجات التي كميتها أقل من حد معين (تنبيه نفاد المخزون)
5. عرض كل الفئات الفريدة الموجودة في المخزون
6. حساب القيمة الإجمالية للمخزون

Python

```
inventory: dict[str, dict[str, object]] = {}

def add_product(name: str, price: float, quantity: int, category: str) → None:
    """إضافة منتج جديد إلى المخزون."""
    if name in inventory:
        print(f"⚠️ المنتج '{name}' موجود مسبقاً. استخدم update_quantity للتعديل.")
        return
    inventory[name] = {
        "price": price,
        "quantity": quantity,
        "category": category
    }
    print(f"✅ تمت إضافة '{name}' بنجاح.")

def update_quantity(name: str, change: int) → None:
    """تحديث كمية منتج (موجب للشراء، سالب للبيع)."""
    if name not in inventory:
        print(f"❌ المنتج '{name}' غير موجود.")
        return

    new_quantity = inventory[name]["quantity"] + change
    if new_quantity < 0:
        print(f"❌ لا توجد كمية كافية من '{name}'.")
        return

    inventory[name]["quantity"] = new_quantity
    print(f"✅ كمية '{name}' الآن: {new_quantity}")

def low_stock_alert(threshold: int = 5) → list[str]:
    """إرجاع قائمة بأسماء المنتجات التي كميتها أقل من الحد المسموح."""
    low_items = [
        name for name, info in inventory.items()
        if info["quantity"] < threshold
    ]
    return low_items

def unique_categories() → set[str]:
    """إرجاع كل الفئات الفريدة الموجودة في المخزون."""
```

```
    return {info["category"] for info in inventory.values()}

def total_inventory_value() -> float:
    """حساب القيمة الإجمالية لكل المخزون (السعر × الكمية)."""
    total = sum(
        info["price"] * info["quantity"]
        for info in inventory.values()
    )
    return total

def print_report() -> None:
    """طباعة تقرير شامل عن حالة المخزون."""
    print("\n" + "=" * 40)
    print("📊 تقرير المخزون")
    print("=" * 40)

    for name, info in inventory.items():
        print(f"- {name}: {info['quantity']} قطعة | "
              f"{info['price']:.2f} ر.س | الفئة: {info['category']}")

    print("-" * 40)
    print(f"💰 القيمة الإجمالية: {total_inventory_value():,.2f} ر.س")
    print(f"🏷️  الفئات المتاحة: {', '.join(unique_categories())}")

    low_items = low_stock_alert(threshold=5)
    if low_items:
        print(f"⚠️  تنبيه نفاد المخزون: {', '.join(low_items)}")
    else:
        print("✅ لا توجد منتجات على وشك النفاد.")
    print("=" * 40)

# --- تشغيل تجريبي للنظام ---
if __name__ == "__main__":
    add_product("لابتوب", 3500.0, 10, "إلكترونيات")
    add_product("ماوس", 50.0, 3, "إلكترونيات")
    add_product("كرسي مكتب", 450.0, 8, "أثاث")
    add_product("طاولة", 700.0, 2, "أثاث")

    update_quantity("لابتوب", -2)   # بيع لابتوبين
    update_quantity("ماوس", 10)     # شراء 10 ماوسات جديدة

    print_report()
```

## الناتج المتوقع:

```
✅ تمت إضافة 'لابتوب' بنجاح.
✅ تمت إضافة 'ماوس' بنجاح.
✅ تمت إضافة 'كرسي مكتب' بنجاح.
✅ تمت إضافة 'طاولة' بنجاح.
✅ كمية 'لابتوب' الآن: 8
✅ كمية 'ماوس' الآن: 13

========================================
📊 تقرير المخزون
========================================
- لابتوب: 8 قطعة | 3500.00 ر.س | الفئة: إلكترونيات
- ماوس: 13 قطعة | 50.00 ر.س | الفئة: إلكترونيات
- كرسي مكتب: 8 قطعة | 450.00 ر.س | الفئة: أثاث
- طاولة: 2 قطعة | 700.00 ر.س | الفئة: أثاث
----------------------------------------
💰 القيمة الإجمالية: 32,250.00 ر.س
🏷️ الفئات المتاحة: إلكترونيات, أثاث
⚠️ تنبيه نفاد المخزون: طاولة
========================================
```

### ✅ تحدي إضافي: دالة مقارنة مخزونين (الحل)

**Python**

```python
def compare_inventories(inv_a: dict, inv_b: dict) -> dict[str, set]:
    """مقارنة منتجات مخزونين مختلفين (مثلاً فرعين لنفس المتجر)."""
    products_a = set(inv_a.keys())
    products_b = set(inv_b.keys())

    return {
        "only_in_a": products_a - products_b,
        "only_in_b": products_b - products_a,
        "in_both": products_a & products_b
    }

branch_1 = {"لابتوب": {}, "ماوس": {}, "كرسي": {}}
branch_2 = {"ماوس": {}, "كرسي": {}, "طابعة": {}}

result = compare_inventories(branch_1, branch_2)
print(result)
# {'only_in_a': {'لابتوب'}, 'only_in_b': {'طابعة'}, 'in_both': {'ماوس', 'كرسي'}}
```

# 9.6 تمارين

## 🟢 مستوى مبتدئ

**تمرين 1:** أنشئ قاموساً باسم `car` يحتوي على: `brand` , `model` , `year` . ثم اطبع كل قيمة باستخدام f-string.

**تمرين 2:** لديك القائمة `visitors = ["Ali", "Sara", "Ali", "Omar", "Sara", "Ali"]` . استخدم set لمعرفة عدد الزوار الفريدين (بدون تكرار) واطبع العدد.

**تمرين 3:** أنشئ قاموساً فارغاً، ثم أضف إليه 3 عناصر باستخدام `[]` ، ثم احذف عنصراً واحداً باستخدام `.pop()` .

✅ الحل

Python

```python
# تمرين 1
car = {"brand": "Toyota", "model": "Corolla", "year": 2023}
print(f"الماركة: {car['brand']}")
print(f"الموديل: {car['model']}")
print(f"السنة: {car['year']}")

# تمرين 2
visitors = ["Ali", "Sara", "Ali", "Omar", "Sara", "Ali"]
unique_visitors = set(visitors)
print(f"عدد الزوار الفريدين: {len(unique_visitors)}")  # 3

# تمرين 3
my_dict = {}
my_dict["a"] = 1
my_dict["b"] = 2
my_dict["c"] = 3
removed = my_dict.pop("b")
print(my_dict)  # {'a': 1, 'c': 3}
```

## 🟡 مستوى متوسط

**تمرين 4:** اكتب دالة `count_words(text: str) → dict[str, int]` تأخذ نصاً وتُرجع قاموساً بعدد تكرار كل كلمة فيه (تجاهل حالة الأحرف).

**تمرين 5:** لديك قائمتان:

```python
team_a = {"Ali", "Sara", "Omar"}
team_b = {"Omar", "Lina", "Khaled"}
```

اطبع: من في الفريقين معاً، من في A فقط، من في B فقط، ومن في فريق واحد فقط (وليس الاثنين).

**تمرين 6:** اكتب دالة `merge_dicts(d1: dict, d2: dict) → dict` تدمج قاموسين، وفي حال وجود مفتاح مشترك، تجمع القيم (بافتراض أنها أرقام).

✅ الحل

Python

```python
# تمرين 4
def count_words(text: str) -> dict[str, int]:
    words = text.lower().split()
    counts: dict[str, int] = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts

result = count_words("Python is great and Python is fun")
print(result)
# {'python': 2, 'is': 2, 'great': 1, 'and': 1, 'fun': 1}

# تمرين 5
team_a = {"Ali", "Sara", "Omar"}
team_b = {"Omar", "Lina", "Khaled"}

print("في الفريقين:", team_a & team_b)
print("في A فقط:", team_a - team_b)
print("في B فقط:", team_b - team_a)
print("في فريق واحد فقط:", team_a ^ team_b)

# تمرين 6
def merge_dicts(d1: dict, d2: dict) -> dict:
    result = d1.copy()
    for key, value in d2.items():
        result[key] = result.get(key, 0) + value
    return result

a = {"apples": 5, "bananas": 3}
b = {"bananas": 2, "oranges": 7}
print(merge_dicts(a, b))
# {'apples': 5, 'bananas': 5, 'oranges': 7}
```

## 🔴 تحدي

**تمرين 7**: اكتب برنامجاً يقرأ قائمة طلبات (كل طلب عبارة عن قاموس فيه `product` و `quantity` و `price`)، ويحسب:
- إجمالي المبيعات لكل منتج
- المنتج الأكثر مبيعاً (بالكمية)
- استخدم `defaultdict` من `collections` في الحل.

### ✅ الحل

Python

```python
from collections import defaultdict

orders = [
    {"product": "قلم", "quantity": 10, "price": 2.0},
    {"product": "دفتر", "quantity": 5, "price": 8.0},
    {"product": "قلم", "quantity": 7, "price": 2.0},
    {"product": "ممحاة", "quantity": 20, "price": 1.0},
    {"product": "دفتر", "quantity": 3, "price": 8.0},
]

sales_total: dict[str, float] = defaultdict(float)
quantity_total: dict[str, int] = defaultdict(int)

for order in orders:
    product = order["product"]
    sales_total[product] += order["quantity"] * order["price"]
    quantity_total[product] += order["quantity"]

print("💰 إجمالي المبيعات لكل منتج:")
for product, total in sales_total.items():
    print(f"  {product}: {total:.2f} ر.س")

best_seller = max(quantity_total, key=quantity_total.get)
print(f"\n🏆 المنتج الأكثر مبيعاً: {best_seller} ({quantity_total[best_seller]} قطعة)")
```

**الناتج:**

```
💰 إجمالي المبيعات لكل منتج:
  قلم: 34.00 ر.س
  دفتر: 64.00 ر.س
  ممحاة: 20.00 ر.س

🏆 المنتج الأكثر مبيعاً: ممحاة (20 قطعة)
```

💡 ملاحظة: `max(dict, key=dict.get)` نمط احترافي شائع جداً للعثور على المفتاح صاحب أكبر قيمة في القاموس.

# 9.7 اختبر فهمك (Quiz)

**السؤال 1: ما الفرق بين `{}` و `set()`؟**

**الإجابة:** `{}` تُنشئ **قاموساً فارغاً** دائماً في Python، بينما `set()` هي الطريقة الوحيدة لإنشاء **مجموعة فارغة**. هذا من أكثر الأخطاء شيوعاً عند المبتدئين.

**السؤال 2: لماذا لا يمكن استخدام list كمفتاح في dict؟**

**الإجابة:** لأن المفاتيح يجب أن تكون **Hashable** (قابلة لحساب قيمة hash ثابتة لا تتغير). القوائم قابلة للتعديل (mutable)، فلو سمحت Python باستخدامها كمفتاح، ثم عدّلتها، سيختل موقعها في بنية الـ Hash Table الداخلية. لهذا فقط الأنواع غير القابلة للتعديل (str, int, float, tuple, frozenset) مسموحة كمفاتيح.

**السؤال 3: ما ناتج الكود التالي؟**

```python
a = {1, 2, 3}
b = {3, 4, 5}
print(a ^ b)
```

**الإجابة:** `{1, 2, 4, 5}` — هذا هو الفرق التماثلي (Symmetric Difference): كل العناصر الموجودة في أحد المجموعتين فقط، وليس في كليهما.

**السؤال 4: متى تفضل استخدام .get() بدلاً من []؟**

**الإجابة:** عندما لا تكون متأكداً 100% من وجود المفتاح في القاموس. `[]` تطلق `KeyError` إذا لم يوجد المفتاح وتُوقف البرنامج، بينما `.get()` تُرجع `None` (أو قيمة افتراضية تحددها أنت) بأمان دون توقف البرنامج.

**❓ السؤال 5: ما الفرق بين البحث عن عنصر في list مقابل set من ناحية الأداء؟**

**الإجابة:** البحث في list هو O(n) — أي أن Python تفحص العناصر واحداً تلو الآخر حتى تجد المطلوب أو تصل للنهاية، فكلما كبرت القائمة طال وقت البحث. البحث في set (وكذلك dict) هو O(1) تقريباً — أي وقت شبه ثابت بغض النظر عن حجم المجموعة، بفضل بنية Hash Table الداخلية.

---

## 9.8 ملخص الفصل (Cheat Sheet)

```python
# ============ DICT ============
d = {"key": "value"}                # إنشاء
d["key"]                            # وصول (يطلق KeyError إن لم يوجد)
d.get("key", "default")             # وصول آمن
d["new_key"] = "value"              # إضافة/تعديل
del d["key"]  /  d.pop("key")       # حذف
"key" in d                          # فحص وجود مفتاح
for k, v in d.items(): ...          # تكرار على المفاتيح والقيم
{k: v for k, v in ...}              # Dict Comprehension
d.update(other_dict)                # دمج قاموسين
d.setdefault(k, default)            # إرجاع أو إنشاء افتراضي

from collections import defaultdict
dd = defaultdict(list)              # قاموس بقيمة افتراضية تلقائية

# ============ SET ============
s = {1, 2, 3}                       # إنشاء (وحدها {} :لاحظ = dict فارغ!)
empty_set = set()                   # مجموعة فارغة - الطريقة الوحيدة
s.add(4)                            # إضافة عنصر
s.remove(1)  /  s.discard(1)        # حذف (remove يطلق خطأ، discard آمن)
a | b   # union() - اتحاد
a & b   # intersection() - تقاطع
a - b   # difference() - فرق
a ^ b   # symmetric_difference() - فرق تماثلي
{x for x in ...}                    # Set Comprehension
```

| المهمة | الحل الأمثل |
|---|---|
| بيانات مرتبطة بأسماء (user, config) | `dict` |
| إزالة التكرار من قائمة | `set(my_list)` |
| بحث متكرر وسريع عن عناصر | `set` بدلاً من `list` |
| تجميع عناصر حسب خاصية | `dict` + `setdefault` أو `defaultdict` |
| مقارنة مجموعتين (مشترك/فريد) | عمليات `set`: `& - \| ^` |

---

## 9.9 ماذا بعد؟

الآن تمتلك ترسانة كاملة من هياكل البيانات: **List** للتسلسلات المرنة، **Tuple** للبيانات الثابتة، **Dict** للبيانات المرتبطة بالأسماء، و**Set** لضمان التفرد والعمليات الرياضية.

لكن حتى الآن، كل برامجنا كانت عبارة عن كود «مسطّح» يُنفَّذ من أعلى لأسفل. ماذا لو أردنا إعادة استخدام نفس الكود عدة مرات دون نسخه ولصقه في كل مرة؟ ماذا لو أردنا تقسيم برنامج كبير إلى قطع صغيرة منظمة وسهلة الفهم والاختبار؟

هذا بالضبط ما ستتعلمه في **الفصل العاشر: الدوال - تنظيم الكود**. ستتعلم كيف تكتب كودك مرة واحدة وتستخدمه ألف مرة، وكيف يفكر المبرمجون المحترفون في «تقسيم المشكلة» إلى دوال صغيرة قابلة لإعادة الاستخدام — وهي من أهم المهارات التي تفصل المبرمج المبتدئ عن المحترف.

فصل 10 / 26 · الجزء الثاني

10

# الدوال - تنظيم الكود

9 دقيقة قراءة | 13 قسمًا | 41 مثالًا | 1,750 كلمة

في هذا الفصل

10.1 لماذا تُعتبر الدوال نقطة تحوّل في رحلتك البرمجية؟
10.2 أساسيات تعريف واستدعاء الدالة
10.3 المعاملات (Parameters) بعمق
10.4 القيمة المُرجعة (Return Values) بعمق
10.5 النطاق (Scope): أين «تعيش» المتغيرات؟
10.6 تمرير المعاملات: القيمة مقابل المرجع
10.7 دوال من الدرجة الأولى (First-Class Functions) - مقدمة
10.8 Type Hints المتقدمة في الدوال
10.9 مشروع تطبيقي: حاسبة علمية بالدوال
10.10 تمارين
10.11 اختبر فهمك (Quiz)
10.12 ملخص الفصل (Cheat Sheet)
10.13 ماذا بعد؟

## 10.1 لماذا تُعتبر الدوال نقطة تحوّل في رحلتك البرمجية؟

تخيّل أنك تكتب برنامجاً لحساب مساحة دائرة في خمسة أماكن مختلفة من الكود. إذا احتجت لاحقاً تعديل طريقة الحساب (مثلاً لتحسين الدقة)، ستحتاج للبحث والتعديل في خمسة أماكن! وإذا نسيت مكاناً واحداً، سيظهر خطأ غامض يصعب تتبعه.

هذا بالضبط ما تحله **الدوال (Functions)**: بدلاً من تكرار نفس الكود، تكتبه **مرة واحدة** بداخل دالة، ثم «تستدعيها» (تطلب تنفيذها) من أي مكان تحتاجه، بقدر ما تشاء من المرات.

```python
# بدون دالة - تكرار مزعج وخطير
radius1 = 5
area1 = 3.14159 * radius1 ** 2
print(f"المساحة: {area1}")

radius2 = 8
area2 = 3.14159 * radius2 ** 2
print(f"المساحة: {area2}")

# مع دالة - اكتب مرة، استخدم دائماً
def calculate_circle_area(radius: float) -> float:
    return 3.14159 * radius ** 2

print(f"المساحة: {calculate_circle_area(5)}")
print(f"المساحة: {calculate_circle_area(8)}")
```

الدوال ليست مجرد «اختصار لتقليل الكتابة» — إنها أداة تفكير. المبرمج المحترف ينظر لأي مشكلة كبيرة ويسأل: «كيف أقسّمها إلى قطع أصغر، كل قطعة تقوم بمهمة واحدة واضحة؟» هذا المبدأ يُسمى **Separation of Concerns** (فصل الاهتمامات)، وهو حجر الأساس لكل كود احترافي، صغيراً كان أو ضخماً بحجم أنظمة شركات عملاقة.

بنهاية هذا الفصل، ستكتب دوالاً احترافية، تفهم كيف تتدفق البيانات داخل وخارج الدالة، وتتقن أحد أهم المفاهيم في Python: **النطاق (Scope)**.

# 10.2 أساسيات تعريف واستدعاء الدالة




كل استدعاء يضيف إطارًا أعلى المكدّس، وكل return يحذفه.

شكل 10.1 — المكدّس أثناء الاستدعاءات المتداخلة

## الصيغة الأساسية

Python

```
def function_name(parameters) -> return_type:
    """Docstring: وصف مختصر لما تفعله الدالة."""
    # جسم الدالة
    return value
```

Python

```
def greet(name: str) -> str:
    """تُرجع رسالة ترحيب للمستخدم."""
    return f"أهلاً بك، {name}!"

message = greet("سارة")
print(message)  # أهلاً بك، سارة!
```

### تشريح الدالة:

| الجزء | الوصف |
|---|---|
| `def` | كلمة مفتاحية تُعلن بداية تعريف دالة |
| `greet` | اسم الدالة (نفس قواعد تسمية المتغيرات: snake_case) |
| `(name: str)` | المعاملات (Parameters) مع Type Hints |
| `→ str` | Type Hint لنوع القيمة المُرجعة |
| `"""..."""` | Docstring - توثيق وظيفة الدالة |
| `return` | القيمة التي تُعاد للمستدعي |

**نصيحة الخبراء:** اختر أسماء دوال تبدأ بفعل يصف ما تفعله: `calculate_total()`, `is_valid()`, `get_user_data()`, `send_email()`. اسم الدالة الجيد يجعل الكود «يُقرأ كجملة إنجليزية» دون الحاجة لتعليقات إضافية.

## Docstrings: التوثيق الاحترافي

Python

```python
def calculate_bmi(weight_kg: float, height_m: float) → float:
    """
    حساب مؤشر كتلة الجسم (BMI).

    المعاملات:
        weight_kg: الوزن بالكيلوغرام
        height_m: الطول بالمتر

    تُرجع:
        قيمة BMI كرقم عشري
    """
    return weight_kg / (height_m ** 2)

# يمكنك قراءة التوثيق في أي وقت باستخدام:
print(calculate_bmi.__doc__)
help(calculate_bmi)
```

**نصيحة الخبراء:** في VS Code، عندما تكتب docstring بصيغة منظمة (كما أعلاه)، ستظهر هذه المعلومات تلقائياً كـ Tooltip عند استدعاء الدالة في أي مكان بالمشروع — هذا يوفر وقتاً هائلاً لك ولزملائك.

### دالة بدون `return`

```python
def print_separator() -> None:
    """تطبع خطاً فاصلاً. لا ترجع أي قيمة."""
    print("-" * 40)

result = print_separator()
print(result)  # None
```

⚠ **تحذير شائع:** إذا لم تكتب `return` صراحة، الدالة تُرجع `None` تلقائياً. هذا سبب شائع لخطأ "لماذا متغيري أصبح `None`؟" — المبرمج يستدعي دالة تطبع نتيجة لكن ينسى أنها لا «تُرجع» شيئاً فعلياً.

```python
# خطأ شائع
def add(a: int, b: int) -> None:
    print(a + b)  # يطبع فقط، لا يُرجع!

result = add(3, 5)  # result = None وليس 8!
print(result + 1)  # 💥 TypeError: unsupported operand type(s)
```

```python
# الصح
def add(a: int, b: int) -> int:
    return a + b

result = add(3, 5)  # result = 8
print(result + 1)  # 9
```

---

## 10.3 المعاملات (Parameters) بعمق

### Parameters مقابل Arguments

مصطلحان كثيراً ما يُخلط بينهما:

- **Parameter (معامل)**: الاسم المُعرَّف في تعريف الدالة
- **Argument (وسيط)**: القيمة الفعلية المُمرَّرة عند الاستدعاء

```python
def greet(name: str) -> str:  # name = parameter
    return f"Hello, {name}"

greet("Ahmed")  # "Ahmed" = argument
```

## المعاملات الموضعية والمسماة (Positional & Keyword Arguments)

```python
def create_profile(name: str, age: int, city: str) -> str:
    return f"{name}, {age} سنة، من {city}"

# موضعية (Positional) - الترتيب مهم جداً
print(create_profile("Ahmed", 25, "Riyadh"))

# مسماة (Keyword) - الترتيب لا يهم
print(create_profile(city="Riyadh", name="Ahmed", age=25))

# مزيج - الموضعية أولاً، ثم المسماة
print(create_profile("Ahmed", city="Riyadh", age=25))
```

⚠ **تحذير:** لا يمكن وضع معامل موضعي بعد معامل مسمى:

```python
# print(create_profile(name="Ahmed", 25, "Riyadh"))
# 💥 SyntaxError: positional argument follows keyword argument
```

💡 **نصيحة الخبراء:** عندما تستدعي دالة لها أكثر من 3 معاملات، استخدم الصيغة المسماة دائماً لتحسين القراءة:

```python
# هنا؟ True و False غامض - ما معنى
send_email("user@test.com", "Subject", "Body", True, False)

# واضح تماماً
send_email(
    to="user@test.com",
    subject="Subject",
    body="Body",
    is_urgent=True,
    send_copy=False
)
```

## القيم الافتراضية (Default Parameters)

```python
def greet(name: str, greeting: str = "أهلاً") -> str:
    return f"{greeting}, {name}!"

print(greet("سارة"))              # أهلاً, سارة!
print(greet("عمر", "مرحباً"))     # مرحباً, عمر!
print(greet("لينا", greeting="صباح الخير"))  # صباح الخير, لينا!
```

⚠ **تحذير حرج - فخ القيم الافتراضية القابلة للتعديل (Mutable Default Argument)**

هذا من أخطر وأشهر الأخطاء في Python، حتى بين المبرمجين ذوي الخبرة:

```python
# ❌ خطأ خطير جداً
def add_item(item: str, cart: list = []) -> list:
    cart.append(item)
    return cart

cart1 = add_item("apple")
print(cart1)  # ['apple']

cart2 = add_item("banana")
print(cart2)  # ['apple', 'banana']  💥 لماذا تذكرت العنصر السابق؟!
```

**السبب:** القيمة الافتراضية `[]` تُنشأ **مرة واحدة فقط** عند تعريف الدالة، وليس في كل استدعاء! لذا كل الاستدعاءات التي لا تمرر `cart` تشترك في **نفس القائمة بالذاكرة**.

```python
# ✅ الحل الصحيح: استخدم None كقيمة افتراضية
def add_item(item: str, cart: list | None = None) -> list:
    if cart is None:
        cart = []  # قائمة جديدة في كل استدعاء
    cart.append(item)
    return cart

cart1 = add_item("apple")
print(cart1)  # ['apple']

cart2 = add_item("banana")
print(cart2)  # ['banana']  ✅ صحيح الآن!
```

**مقارنة:** هذا السلوك مرتبط بكون Python تُنشئ كائنات القيم الافتراضية مرة واحدة عند **تعريف** الدالة (وقت التحميل)، وليس عند **استدعائها**. لغات أخرى مثل JavaScript لا تملك هذا الفخ لأن طريقة التعامل مع المعاملات الافتراضية تختلف جوهرياً. **القاعدة الذهبية في Python**: لا تستخدم أبداً `list` , `dict` , أو `set` كقيمة افتراضية مباشرة — استخدم `None` دائماً كبديل آمن.

## `*args` : عدد غير محدد من المعاملات الموضعية

```python
def calculate_sum(*numbers: float) -> float:
    """تجمع أي عدد من الأرقام الممررة."""
    total = 0
    for num in numbers:
        total += num
    return total

print(calculate_sum(1, 2, 3))          # 6
print(calculate_sum(1, 2, 3, 4, 5))    # 15
print(calculate_sum())                  # 0

# داخل الدالة، numbers تصبح tuple
def show_type(*args):
    print(type(args))  # <class 'tuple'>

show_type(1, 2, 3)
```

### `**kwargs` : عدد غير محدد من المعاملات المسماة

```python
def create_user(**details: str) -> dict:
    """تنشئ قاموس مستخدم من أي عدد من الحقول المسماة."""
    return details

user = create_user(name="Ahmed", age=25, city="Jeddah")
print(user)
# {'name': 'Ahmed', 'age': 25, 'city': 'Jeddah'}

# داخل الدالة، kwargs تصبح dict
def show_type(**kwargs):
    print(type(kwargs))  # <class 'dict'>

show_type(a=1, b=2)
```

## ترتيب المعاملات الكامل

```python
def full_example(
    positional: str,
    default: int = 10,
    *args: int,
    keyword_only: str = "default_value",
    **kwargs: str
) -> None:
    print(f"positional: {positional}")
    print(f"default: {default}")
    print(f"args: {args}")
    print(f"keyword_only: {keyword_only}")
    print(f"kwargs: {kwargs}")

full_example("A", 20, 1, 2, 3, keyword_only="X", extra="Y")
```

### الناتج:

```
positional: A
default: 20
args: (1, 2, 3)
keyword_only: X
kwargs: {'extra': 'Y'}
```

| الترتيب الإلزامي | الوصف |
|---|---|
| 1. Positional parameters | معاملات عادية بدون/مع قيمة افتراضية |
| 2. `*args` | معاملات موضعية إضافية غير محددة العدد |
| 3. Keyword-only parameters | يجب تمريرها بالاسم (بعد `*args` أو `*`) |
| 4. `**kwargs` | معاملات مسماة إضافية غير محددة العدد |

## Positional-Only Arguments (Python 3.8+) g Keyword-Only

Python

```python
# استخدام * لإجبار ما بعدها أن تكون keyword-only
def create_order(product: str, *, quantity: int = 1, urgent: bool = False) -> str:
    return f"{quantity}x {product} {'(عاجل)' if urgent else ''}"

# print(create_order("Laptop", 2))
# 💥 TypeError: يجب تمرير quantity بالاسم

print(create_order("Laptop", quantity=2, urgent=True))  # ✅ صحيح

# استخدام / لإجبار ما قبلها أن تكون positional-only
def divide(a: float, b: float, /) -> float:
    return a / b

print(divide(10, 2))      # ✅ صحيح
# print(divide(a=10, b=2))
# 💥 TypeError: يجب تمريرهما بالموضع فقط a و b
```

**نصيحة الخبراء:** استخدم `*` لإجبار الوضوح في المعاملات التي قد يُساء فهمها (مثل `urgent=True` بدلاً من `True` غامضة)، واستخدم `/` عندما يكون اسم المعامل تفصيلاً داخلياً لا يهم المستخدم (شائع في مكتبات built-in مثل `len(obj, /)`).

# 10.4 القيمة المُرجعة (Return Values) بعمق

## إرجاع قيم متعددة

```python
def get_min_max(numbers: list[float]) -> tuple[float, float]:
    """تُرجع أصغر وأكبر قيمة في القائمة."""
    return min(numbers), max(numbers)

low, high = get_min_max([5, 2, 9, 1, 7])
print(f"الأصغر: {low}, الأكبر: {high}")  # الأصغر: 1, الأكبر: 9
```

في الحقيقة، Python تُرجع **tuple واحدة** هنا ( `(1, 9)` ) ثم يحدث **Unpacking** تلقائي إلى المتغيرين.

## الخروج المبكر (Early Return / Guard Clauses)

تعلمنا هذا المفهوم في الفصل السادس؛ الدوال هي المكان المثالي لتطبيقه:

```python
def calculate_discount(price: float, is_member: bool) -> float:
    """حساب السعر بعد الخصم."""
    if price <= 0:
        return 0.0  # خروج مبكر - لا داعي لمتابعة الحساب

    if not is_member:
        return price  # خروج مبكر - لا خصم

    return price * 0.85  # متعدد الكود الرئيسي، بدون تداخل if
```

**مقارنة:** بدون Guard Clauses، سيبدو الكود هكذا مع تداخل مزعج:

```python
def calculate_discount_messy(price: float, is_member: bool) -> float:
    if price > 0:
        if is_member:
            return price * 0.85
        else:
            return price
    else:
        return 0.0
```

النمط الأول أسهل بكثير للقراءة والصيانة، خاصة مع تعقّد الشروط لاحقاً.

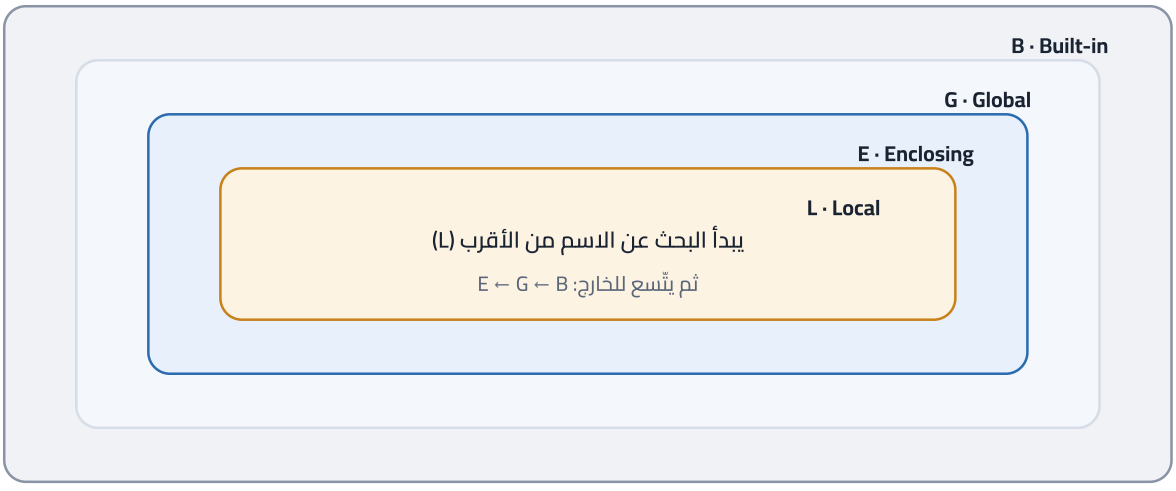
### دوال بدون return صريح مقابل `return None`

```python
def log_message(message: str) -> None:
    """تسجل رسالة. لا ترجع شيئاً (None صراحة)."""
    print(f"[LOG]: {message}")
    return None  # اختياري لكنه أوضح من حيث القصد

def validate_age(age: int) -> bool | None:
    """ترجع True/False؛ تتحقق من صحة العمر. إذا كانت القيمة غير منطقية None."""
    if age < 0 or age > 150:
        return None
    return age >= 18
```

---

## 10.5 النطاق (Scope): أين «تعيش» المتغيرات؟




شكل 10.2 — ترتيب البحث عن الأسماء: LEGB

هذا من أهم المفاهيم التي يسيء المبتدئون فهمها. **النطاق** يحدد أين يمكن الوصول لمتغير معين في الكود.

## Global Scope g Local Scope

```python
x = 10  # متغير عام (Global)

def my_function():
    y = 20  # متغير محلي (Local) - يعيش فقط داخل الدالة
    print(x)  # يمكن قراءة المتغير العام من داخل الدالة
    print(y)

my_function()
print(x)  # 10 - متاح
# print(y)
# 💥 NameError: name 'y' is not defined -- y لا توجد خارج الدالة!
```

⚠ **تحذير شائع**: المبتدئون يستغربون لماذا متغير عرّفوه داخل دالة «يختفي» خارجها. هذا ليس خطأ — إنه التصميم المقصود للنطاق، ويمنع الفوضى في البرامج الكبيرة.

## لماذا لا يمكن تعديل متغير عام من داخل دالة مباشرة؟

```python
counter = 0

def increment():
    counter += 1  # 💥 UnboundLocalError!

increment()
```

**لماذا يحدث هذا الخطأ؟** لأن Python عندما ترى `counter += 1` داخل الدالة، تفترض أن `counter` متغير **محلي** جديد (لأنك تحاول الكتابة إليه)، لكنك تحاول قراءته قبل تعريفه محلياً، فتحدث مشكلة.

## الكلمة المفتاحية `global`

```python
counter = 0

def increment():
    global counter  # أخبر Python: استخدم المتغير العام، لا تنشئ واحداً محلياً
    counter += 1

increment()
increment()
print(counter)  # 2
```

⚠ **تحذير احترافي:** استخدام `global` يُعتبر من أفضل الممارسات **السيئة** (Anti-pattern) في البرمجة الحديثة! لأنه يجعل الدوال «تعتمد» على حالة خارجية خفية، مما يصعّب تتبع الأخطاء وفهم الكود. **البديل الأفضل دائماً:**

```python
# ✅ الطريقة الاحترافية: مرر القيمة وأعد النتيجة
def increment(counter: int) -> int:
    return counter + 1

counter = 0
counter = increment(counter)
counter = increment(counter)
print(counter)  # 2
```

🔄 **مقارنة:** في البرمجة الكائنية (الفصل 13)، سنرى حلاً أنظف بكثير لهذه المشكلة باستخدام `self.counter` داخل الكائنات (Objects)، مما يلغي الحاجة لـ `global` نهائياً في الكود الاحترافي.

## قاعدة LEGB

Python تبحث عن أي متغير بالترتيب التالي:

| المستوى | الاسم | الوصف |
|---|---|---|
| 1 | **L**ocal | داخل الدالة الحالية |
| 2 | **E**nclosing | داخل أي دالة «محيطة» (دوال متداخلة - الفصل 12) |
| 3 | **G**lobal | على مستوى الملف كله |
| 4 | **B**uilt-in | الأسماء المدمجة في Python مثل `len` , `print` |

Python

```
x = "global x"

def outer():
    x = "enclosing x"

    def inner():
        x = "local x"
        print(x)  # local x - توجد في المستوى المحلي، تتوقف هنا

    inner()
    print(x)  # enclosing x

outer()
print(x)  # global x
```

**نصيحة الخبراء:** كلما قلّت اعتمادية دالتك على متغيرات خارجية (Global)، وكلما اعتمدت فقط على معاملاتها (Parameters)، أصبحت أسهل للاختبار، وأسهل لإعادة الاستخدام، وأقل عرضة للأخطاء الغامضة. هذا المبدأ يُسمى **Pure Functions** (الدوال النقية): الدالة التي تُعطي نفس النتيجة دائماً لنفس المدخلات، ولا تُغيّر أي شيء خارج نطاقها.

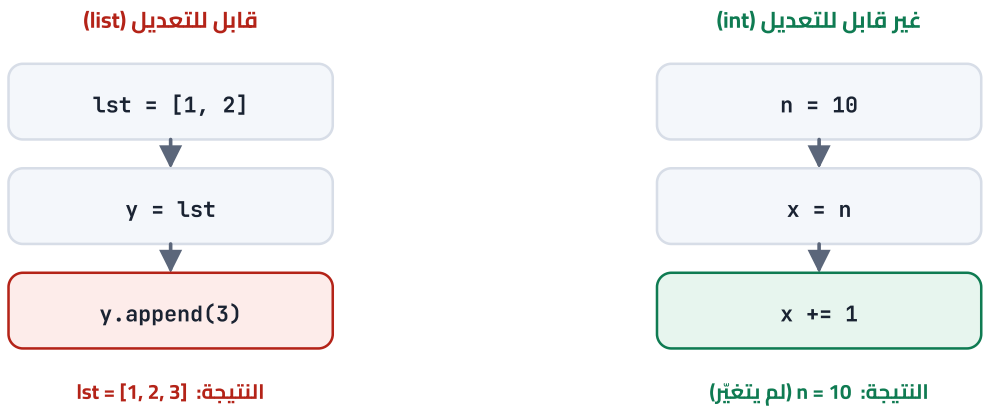
Python

```
# دالة نقية (Pure Function) - يُفضّل هذا النمط
def add_tax(price: float, tax_rate: float) -> float:
    return price * (1 + tax_rate)

# دالة غير نقية - تعتمد على وتُعدّل حالة خارجية
total = 0

def add_to_total(amount: float) -> None:
    global total
    total += amount
```

# 10.6 تمرير المعاملات: القيمة مقابل المرجع




تمرير المعامل يمرّر المرجع؛ لكن إعادة الإسناد داخل الدالة لا تمسّ المتغيّر الأصلي.

شكل 10.3 — تمرير المعاملات: القابل للتعديل وغير القابل

هذا المفهوم يُربك الكثير من المبرمجين القادمين من لغات أخرى.

```python
def modify_number(n: int) -> None:
    n += 10
    print(f"داخل الدالة: {n}")

x = 5
modify_number(x)
print(f"خارج الدالة: {x}")

# الناتج:
# داخل الدالة: 15
# خارج الدالة: 5  -- لم يتغير!
```

لكن مع القوائم (كائنات قابلة للتعديل):

```python
def modify_list(lst: list) -> None:
    lst.append(100)
    print(f"داخل الدالة: {lst}")

my_list = [1, 2, 3]
modify_list(my_list)
print(f"خارج الدالة: {my_list}")

# الناتج:
# داخل الدالة: [1, 2, 3, 100]
# خارج الدالة: [1, 2, 3, 100] -- تغيّرت فعلاً!
```

**السبب**: Python تُمرر دائماً «مرجع» (Reference) للكائن، لكن التأثير يختلف حسب نوع الكائن:
- الأنواع **غير القابلة للتعديل** (int, float, str, tuple): أي عملية كـ `n += 10` تُنشئ كائناً **جديداً** بالكامل، فالمتغير الأصلي خارج الدالة لا يتأثر.
- الأنواع **القابلة للتعديل** (list, dict, set): العمليات مثل `.append()` تُعدّل **نفس الكائن** في الذاكرة، فالتغيير يظهر خارج الدالة أيضاً.

⚠ **تحذير حرج**: هذا السلوك مصدر شائع جداً للأخطاء الغامضة. إذا مررت قائمة لدالة ولم تقصد تعديلها، انسخها أولاً:

```python
def process_safely(lst: list) -> list:
    lst_copy = lst.copy()  # اعمل على نسخة
    lst_copy.append(999)
    return lst_copy
```

🔄 **مقارنة مع لغات أخرى**: هذا المفهوم يُسمى أحياناً "Pass by Object Reference"، وهو مختلف عن "Pass by Value" الصِرف (كما في C عند تمرير أنواع بسيطة) و"Pass by Reference" الصِرف (كما في بعض حالات C++). Python تقف في منتصف الطريق: المرجع نفسه يُمرر بالقيمة، لكن الكائن المُشار إليه يبقى نفسه.

---

## 10.7 دوال من الدرجة الأولى (First-Class Functions) - مقدمة

في Python، الدوال هي «مواطنون من الدرجة الأولى» (First-Class Citizens) — يمكن التعامل معها كأي قيمة أخرى: تخزينها في متغير، تمريرها كمعامل لدالة أخرى، أو إرجاعها من دالة.

```python
def square(x: int) -> int:
    return x ** 2

# تخزين الدالة في متغير (بدون أقواس استدعاء!)
my_func = square
print(my_func(5))  # 25

# تمرير دالة كمعامل لدالة أخرى
def apply_function(func, value: int) -> int:
    return func(value)

print(apply_function(square, 6))  # 36
```

مثال عملي باستخدام `sorted()` مع معامل `key` (دالة):

```python
students = [
    {"name": "Ahmed", "score": 85},
    {"name": "Sara", "score": 92},
    {"name": "Omar", "score": 78}
]

def get_score(student: dict) -> int:
    return student["score"]

sorted_students = sorted(students, key=get_score, reverse=True)
for s in sorted_students:
    print(f"{s['name']}: {s['score']}")

# الناتج:
# Sara: 92
# Ahmed: 85
# Omar: 78
```

هذا تمهيد لمفهوم **Lambda Functions** و**Higher-Order Functions** الذي سنتعمق فيه بالكامل في **الفصل الثاني عشر**.

## 10.8 Type Hints المتقدمة في الدوال

Python

```python
# استخدام | للسماح بأكثر من نوع (Python 3.10+)
def format_value(value: int | float | str) -> str:
    return f"القيمة: {value}"

# استخدام Optional (أو | None) لمعامل قد يكون غائباً
def find_user(user_id: int) -> dict | None:
    users = {1: {"name": "Ahmed"}, 2: {"name": "Sara"}}
    return users.get(user_id)

# Callable: Type Hint لمعامل يكون دالة
from collections.abc import Callable

def apply_twice(func: Callable[[int], int], value: int) -> int:
    return func(func(value))

def add_five(x: int) -> int:
    return x + 5

print(apply_twice(add_five, 10))  # 20
```

**نصيحة الخبراء:** استخدم أداة `mypy` (سنتعمق فيها في الفصل 15) للتحقق التلقائي من أن كل استدعاءات دوالك تتوافق مع الـ Type Hints المُعلنة. هذا يكشف أخطاءً كثيرة **قبل** تشغيل الكود فعلياً.

---

## 10.9 مشروع تطبيقي: حاسبة علمية بالدوال

سنبني حاسبة علمية مُنظّمة بالكامل بدوال، تدعم العمليات الأساسية والمتقدمة، مع معالجة آمنة للأخطاء الشائعة.

Python

```python
import math

def add(a: float, b: float) -> float:
    """جمع رقمين."""
    return a + b

def subtract(a: float, b: float) -> float:
    """طرح رقمين."""
    return a - b
```

```
def multiply(a: float, b: float) -> float:
    """ضرب رقمين."""
    return a * b

def divide(a: float, b: float) -> float | None:
    """قسمة رقمين، مع حماية من القسمة على صفر."""
    if b == 0:
        print("❌ خطأ: لا يمكن القسمة على صفر.")
        return None
    return a / b

def power(base: float, exponent: float) -> float:
    """رفع رقم لأس معين."""
    return base ** exponent

def square_root(number: float) -> float | None:
    """الجذر التربيعي، مع حماية من الأرقام السالبة."""
    if number < 0:
        print("❌ خطأ: لا يمكن حساب الجذر التربيعي لرقم سالب.")
        return None
    return math.sqrt(number)

def percentage(part: float, whole: float) -> float | None:
    """حساب النسبة المئوية لـ part من whole."""
    if whole == 0:
        print("❌ خطأ: القيمة الكلية لا يمكن أن تكون صفراً.")
        return None
    return (part / whole) * 100

# --- قاموس يربط الرمز بالدالة المقابلة (نمط احترافي) ---
OPERATIONS: dict[str, tuple] = {
    "+": (add, 2),
    "-": (subtract, 2),
    "*": (multiply, 2),
    "/": (divide, 2),
    "^": (power, 2),
    "sqrt": (square_root, 1),
    "%": (percentage, 2),
}

def display_menu() -> None:
    """طباعة قائمة العمليات المتاحة."""
    print("\n" + "=" * 40)
    print("🧮 الحاسبة العلمية")
    print("=" * 40)
    print("العمليات المتاحة:")
    print("  +     : جمع")
    print("  -     : طرح")
    print("  *     : ضرب")
    print("  /     : قسمة")
    print("  ^     : أس (قوة)")
```

```
    print("  sqrt  : جذر تربيعي")
    print("  %     : نسبة مئوية")
    print("  q     : خروج")
    print("=" * 40)

def run_calculator() → None:
    """تشغيل الحاسبة في حلقة تفاعلية."""
    display_menu()

    while True:
        choice = input("\nاختر العملية: ").strip()

        if choice == "q":
            print("👋 إلى اللقاء!")
            break

        if choice not in OPERATIONS:
            print("❌ عملية غير معروفة. حاول مجدداً.")
            continue

        func, num_args = OPERATIONS[choice]

        try:
            if num_args == 1:
                x = float(input("أدخل الرقم: "))
                result = func(x)
            else:
                x = float(input("أدخل الرقم الأول: "))
                y = float(input("أدخل الرقم الثاني: "))
                result = func(x, y)

            if result is not None:
                print(f"✅ النتيجة: {result}")

        except ValueError:
            print("❌ خطأ: الرجاء إدخال أرقام صحيحة فقط.")

if __name__ == "__main__":
    run_calculator()
```

### مثال تشغيل:

```
==========================================
🧮 الحاسبة العلمية
==========================================
العمليات المتاحة:
  +      : جمع
  -      : طرح
  *      : ضرب
  /      : قسمة
  ^      : أس (قوة)
  sqrt   : جذر تربيعي
  %      : نسبة مئوية
  q      : خروج
==========================================

اختر العملية: +
أدخل الرقم الأول: 5
أدخل الرقم الثاني: 3
✅ النتيجة: 8.0

اختر العملية: sqrt
أدخل الرقم: 16
✅ النتيجة: 4.0

اختر العملية: /
أدخل الرقم الأول: 10
أدخل الرقم الثاني: 0
❌ خطأ: لا يمكن القسمة على صفر.

اختر العملية: q
👋 إلى اللقاء!
```

💡 **ملاحظة تصميمية مهمة**: لاحظ استخدام قاموس `OPERATIONS` لربط الرموز بالدوال المقابلة بدلاً من سلسلة طويلة من `if/elif`. هذا النمط يُسمى **Dispatch Table**، وهو أسلوب احترافي شائع لتنظيم الكود عندما يكون لديك عدة خيارات تؤدي لاستدعاء دوال مختلفة.

✅ تحدي إضافي: إضافة سجل العمليات (الحل)

Python

```python
calculation_history: list[str] = []

def log_calculation(operation: str, result: float) -> None:
    """تسجل كل عملية حساب في سجل."""
    calculation_history.append(f"{operation} = {result}")

def show_history() -> None:
    """تعرض كل العمليات المحسوبة سابقاً."""
    if not calculation_history:
        print("لا توجد عمليات سابقة.")
        return

    print("\n📜 سجل العمليات:")
    for entry in calculation_history:
        print(f"  {entry}")

# يمكن استدعاء log_calculation بعد كل نتيجة ناجحة في run_calculator
```

## 10.10 تمارين

### 🟢 مستوى مبتدئ

**تمرين 1:** اكتب دالة `is_even(number: int) → bool` تُرجع `True` إذا كان الرقم زوجياً.

**تمرين 2:** اكتب دالة `greet_user(name: str, title: str = "الأستاذ/ة") → str` تُرجع تحية مع لقب افتراضي.

**تمرين 3:** اكتب دالة `average(*numbers: float) → float` تحسب متوسط أي عدد من الأرقام الممررة.

✅ الحل

Python

```python
# تمرين 1
def is_even(number: int) -> bool:
    return number % 2 == 0

print(is_even(4))  # True
print(is_even(7))  # False

# تمرين 2
def greet_user(name: str, title: str = "الأستاذ/ة") -> str:
    return f"مرحباً {title} {name}"

print(greet_user("سمير"))                    # مرحباً الأستاذ/ة سمير
print(greet_user("نورة", title="الدكتورة"))  # مرحباً الدكتورة نورة

# تمرين 3
def average(*numbers: float) -> float:
    if not numbers:
        return 0.0
    return sum(numbers) / len(numbers)

print(average(10, 20, 30))  # 20.0
```

## 🟡 مستوى متوسط

**تمرين 4:** اكتب دالة `is_valid_password(password: str) → tuple[bool, str]` تتحقق أن كلمة المرور: طولها 8 أحرف على الأقل، تحتوي رقماً واحداً على الأقل، وحرفاً كبيراً واحداً على الأقل. تُرجع `(True, "صالحة")` أو `(False, "سبب الرفض")`.

**تمرين 5:** اكتب دالة `filter_products(products: list[dict], **criteria) → list[dict]` تُرجع المنتجات المطابقة لأي معايير مُمرَّرة (مثلاً `category="إلكترونيات"`).

✅ الحل

Python

```python
# تمرين 4
def is_valid_password(password: str) -> tuple[bool, str]:
    if len(password) < 8:
        return False, "يجب أن تكون 8 أحرف على الأقل"
    if not any(char.isdigit() for char in password):
        return False, "يجب أن تحتوي على رقم واحد على الأقل"
    if not any(char.isupper() for char in password):
        return False, "يجب أن تحتوي على حرف كبير واحد على الأقل"
    return True, "صالحة"

valid, message = is_valid_password("abc123")
print(f"{valid}: {message}")  # False: يجب أن تكون 8 أحرف على الأقل

valid, message = is_valid_password("Abcd1234")
print(f"{valid}: {message}")  # True: صالحة

# تمرين 5
def filter_products(products: list[dict], **criteria) -> list[dict]:
    result = []
    for product in products:
        if all(product.get(key) == value for key, value in criteria.items()):
            result.append(product)
    return result

products = [
    {"name": "لابتوب", "category": "إلكترونيات", "price": 3500},
    {"name": "كرسي", "category": "أثاث", "price": 450},
    {"name": "ماوس", "category": "إلكترونيات", "price": 50},
]

electronics = filter_products(products, category="إلكترونيات")
print(electronics)
# [{'name': 'لابتوب',...  }, {'name': 'ماوس',...  }]
```

## 🔴 تحدي

**تمرين 6:** اكتب دالة `memoize(func)` تأخذ دالة أخرى كمعامل، وتُرجع نسخة «محسّنة» منها تُخزّن (تتذكر) نتائج الاستدعاءات السابقة بنفس المعاملات، بدلاً من إعادة حسابها (هذا تمهيد لمفهوم Decorators في الفصل 12).

✅ الحل

Python

```python
def memoize(func):
    """تُرجع نسخة من func السابقة النتائج تخزّن."""
    cache: dict = {}

    def wrapper(*args):
        if args in cache:
            print(f"(من الذاكرة المؤقتة لـ {args})")
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result

    return wrapper

def slow_square(n: int) -> int:
    import time
    time.sleep(1)  # محاكاة عملية بطيئة
    return n ** 2

fast_square = memoize(slow_square)

print(fast_square(5))  # 25 :يأخذ ثانية واحدة، النتيجة
print(fast_square(5))  # ((,5) لـ (من الذاكرة المؤقتة !فوري
print(fast_square(7))  # 49 :يأخذ ثانية واحدة، النتيجة
```

ملاحظة: مكتبة Python القياسية توفر هذا جاهزاً عبر `functools.lru_cache` — سنتعرف عليها في الفصل 12 عند دراسة Decorators بعمق.

## 10.11 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين Parameter و Argument؟

**الإجابة:** Parameter هو الاسم المُعرَّف داخل أقواس تعريف الدالة (`def func(name):`)، أما Argument هو القيمة الفعلية التي تُمرر عند استدعاء الدالة (`func("Ahmed")`).

### ❓ السؤال 2: لماذا يُعتبر استخدام list كقيمة افتراضية لمعامل خطأً خطيراً؟

**الإجابة:** لأن القيمة الافتراضية تُنشأ **مرة واحدة فقط** عند تعريف الدالة، وليس في كل استدعاء. فإذا عدّلتها (مثل `.append()`) في أحد الاستدعاءات، ستبقى معدّلة وتؤثر على كل الاستدعاءات اللاحقة التي لا تمرر قيمة صريحة لهذا المعامل. الحل: استخدم `None` كقيمة افتراضية، وأنشئ القائمة الفعلية داخل جسم الدالة.

### ❓ السؤال 3: ما ناتج الكود التالي ولماذا؟

```python
def modify(x, lst):
    x += 1
    lst.append(x)

a = 10
b = [1, 2, 3]
modify(a, b)
print(a, b)
```

**الإجابة:** `10 [1, 2, 3, 11]`. المتغير `a` (int، غير قابل للتعديل) لم يتأثر لأن `x += 1` أنشأ كائناً جديداً داخل الدالة فقط. أما `b` (list، قابلة للتعديل) تأثرت فعلياً لأن `.append()` عدّلت نفس الكائن في الذاكرة الذي يُشير إليه `b` خارج الدالة أيضاً.

### ❓ السؤال 4: لماذا يُعتبر استخدام global ممارسة سيئة غالباً؟

**الإجابة:** لأنه يجعل الدالة تعتمد على حالة خارجية خفية غير واضحة من توقيع الدالة نفسها، مما يصعّب فهم الدالة بمعزل عن باقي الكود، ويصعّب اختبارها، ويزيد احتمال حدوث أخطاء غير متوقعة عند تعديل الكود لاحقاً. البديل الأفضل: تمرير القيم كمعاملات وإرجاع النتائج بدلاً من تعديل متغيرات خارجية مباشرة.

### ❓ السؤال 5: ما الفرق بين *args و **kwargs؟

**الإجابة:** `*args` تجمع أي عدد من المعاملات **الموضعية** الإضافية في tuple، بينما `**kwargs` تجمع أي عدد من المعاملات **المسماة** الإضافية في dict.

## 10.12 ملخص الفصل (Cheat Sheet)

Python

```python
# تعريف دالة أساسية
def func_name(param: type) -> return_type:
    """Docstring."""
    return value

# قيم افتراضية
def func(a: int, b: int = 10) -> int: ...

# ⚠️ لا تستخدم mutable كقيمة افتراضية
def bad(lst: list = []): ...            # ❌
def good(lst: list | None = None):      # ✅
    if lst is None:
        lst = []

# *args و **kwargs
def func(*args, **kwargs): ...
func(1, 2, 3, key="value")

# Keyword-only و Positional-only
def func(a, b, /, c, d, *, e, f): ...
#         ^positional-only  ^keyword-only

# إرجاع قيم متعددة (tuple unpacking)
def minmax(nums: list) -> tuple:
    return min(nums), max(nums)
low, high = minmax([1, 5, 3])

# Scope
x = 10                  # Global
def f():
    y = 20              # Local
    global x; x = 99    # تعديل Global (تجنّبه قدر الإمكان)

# الدوال كقيم (First-Class Functions)
my_func = some_function   # بدون ()
sorted(data, key=my_func) # تمرير كمعامل
```

| المفهوم | القاعدة الذهبية |
|---|---|
| القيم الافتراضية | لا تستخدم list/dict/set مباشرة، استخدم None |
| التعديل على الحالة الخارجية | تجنّب global، مرر وأعد القيم بدلاً منه |
| وضوح الاستدعاء | استخدم Keyword Arguments مع أكثر من 3 معاملات |
| عدد غير محدد من المدخلات | `*args` للموضعية، `**kwargs` للمسماة |
| الدوال النقية (Pure Functions) | يُفضّل دائماً: نفس المدخل = نفس المخرج، بدون آثار جانبية |

---

## 10.13 ماذا بعد؟

تهانينا! لقد أتممت الآن **الجزء الأول من الكتاب بالكامل**: الأساسيات المتينة في Python. تمتلك الآن كل الأدوات الأساسية: المتغيرات، النصوص، الأرقام، التحكم بالتدفق، الحلقات، هياكل البيانات الأربعة، والدوال.

لكن حتى الآن، تعاملنا مع الأخطاء بطريقة بدائية: برنامجك ينهار فوراً عند حدوث أي خطأ غير متوقع (`KeyError`, `ValueError`, `ZeroDivisionError`...). في العالم الحقيقي، البرامج الاحترافية **تتوقع** الأخطاء وتتعامل معها بأناقة دون أن تنهار بالكامل.

في **الفصل الحادي عشر: معالجة الأخطاء**، ستتعلم أسلوب `try/except/else/finally`، كيف «ترمي» أخطاءك المخصصة، وكيف تكتب برامج قوية تصمد أمام المدخلات غير المتوقعة والظروف الاستثنائية — مهارة تفصل البرامج الهشة عن البرامج الاحترافية الجاهزة للإنتاج.